Building a Processor

CSE112 MAJOR TASK PHASE 1 & 2

Presented to:

Dr. Karim Emara
Dr. Tamer Mostafa
Eng. Aya
Eng. Yasmine

Presented by:
Team **19**

Date of publication APRIL 21, 2024, date of current version APRIL 24, 2024.

Building a Processor
(Register File, ALU, Controller, PC, Sign Extender and Shifter)

Author's Name: Youssef Moustafa Elsayed
ID: 22P0047

Author's Name: Nour Eldeen Ahmed Ali Ahmed Rehab
ID: 22P0040

Author's Name: Ibrahim Shaker Hammad Shaker
ID: 22P0056

Author's Name: Ahmed Hazem Abdul Rahman Khalil Ibrahim
ID: 22P0226

Author's Name: Mohamed Yehia Zakaria Abdelhamid
ID: 22P0064

Contribution in the project

Phase 1

Youssef & Ibrahim contributed in (Register File)

Mohamed & Nour contributed in (ALU)

Ahmed & Youssef contributed in (data path)

Every one contributed in his part in the (Report)

Phase 2

Youssef & Ibrahim contributed in (datapath)

Mohamed & Nour contributed in (controller)

Ahmed & Youssef contributed in (mips & main)

Every one contributed in his part in the (Report)

Table of Contents

Contribution in the project 2
List of Figures 4
Abstract 5
Introduction 6
Project statement & description 9
Controller 23
Instruction Memory 28
Data Memory 28
Updated DataPath 29
VHDL coding of blocks with synthesized RTL schematic 31
Testing Codes 64
Simulation of blocks 69
Conclusion 79
References 80

List of Figures

Figure 1: Final processor shape 9
Figure 2: D - Flip Flop 12
Figure 3: 4-bit Flop Register 12
Figure 4: Writing in Register File 14
Figure 5: Reading from the registers 15
Figure 6: Register File 16
Figure 7: ALU specifications 17
Figure 8: 1 bit ALU 17
Figure 9: Subtraction in the ALU 19
Figure 10: Zero Flag 19
Figure 11: ALU 20
Figure 12: Instruction 22

Abstract

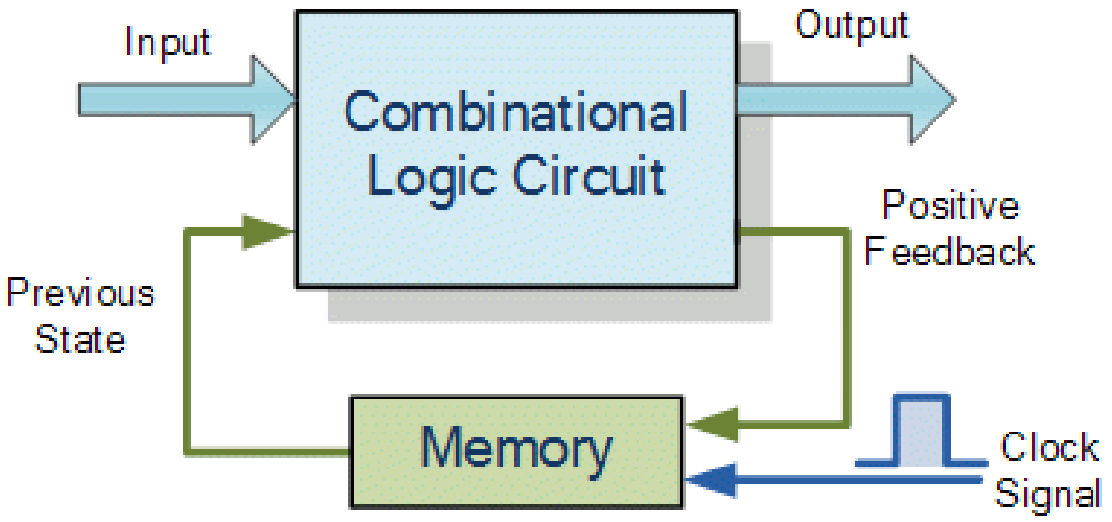
The project part I is designing a processor mainly containing Register File and ALU. the project part II is designing a processor mainly containing PC, Controller, Sign Extender and Left Shifter that's for milestone 2. Designing a project like this need a combination of combinational & sequential circuits. Most important step is analyzing the project and dividing it into smaller components to build the processor.

Introduction

In the project, as mentioned before our objective is to design a processor containing Register File, ALU, Controller, PC, Sign Extender and Bit Sifter and all of them will be explained later on. But, before we go on with analyzing and discussing we must know about VHDL. VHDL is hardware Description Language, It can be tested on software simulation before hardware implementation, then translated into a hardware circuit. Finally, there are lists of background information need to be known.

1- Sequential circuits & how to use them in VHDL

Sequential circuits are a special type of circuits but consists of a series of various inputs and outputs. Here, the outputs depend on a combination of both the present inputs as well as the previous outputs. This previous output gets treated in the form of the present state. They depend on clock where each clock cycle it causes an effect on the output. It must be enclosed in sequential block which guarantees sequential execution.



PROCESSES, FUNCTIONS, and PROCEDURES are the only sections of code that are executed sequentially. There are statements that allowed only inside sequential code like IF and CASE statements.

PROCESS

To construct a synchronous circuit, monitoring a signal is necessary (for example clk) A common way of detecting a signal change

(clk'EVENT AND clk='1')... -- EVENT attribute

IF STATEMENT

IF/ELSE Creates priority-encoded logic.

```
ARCHITECTURE myarch OF myentity IS
BEGIN
  PROCESS (clk, rst)
  BEGIN
    IF (rst = '1') THEN
      . . .
    ELSIF (clk'EVENT AND clk = '1') THEN
      . . .
    ELSE
      . . .
    END IF;
  END PROCESS;
END myarch;
```

2- Declaring components

Declaring components is performing a certain circuit like flop registers or multiplexers then we declare them in a package where this package can be included in our main module, this helps in achieving reusability as for example when you need to use in your main module 32 flop register you just implement it once and add its component in a package then include it in the main module, after that you can make 32 flop register in the main by just declaring it not by repeating the same code of flop register 32 times.

Project statement & description

Part I

I have mentioned before several times that our aim is to build a processor containing Register File and ALU and connected together as shown in the following figure and designing this data path between Register File & ALU is going to be the final step in our project.

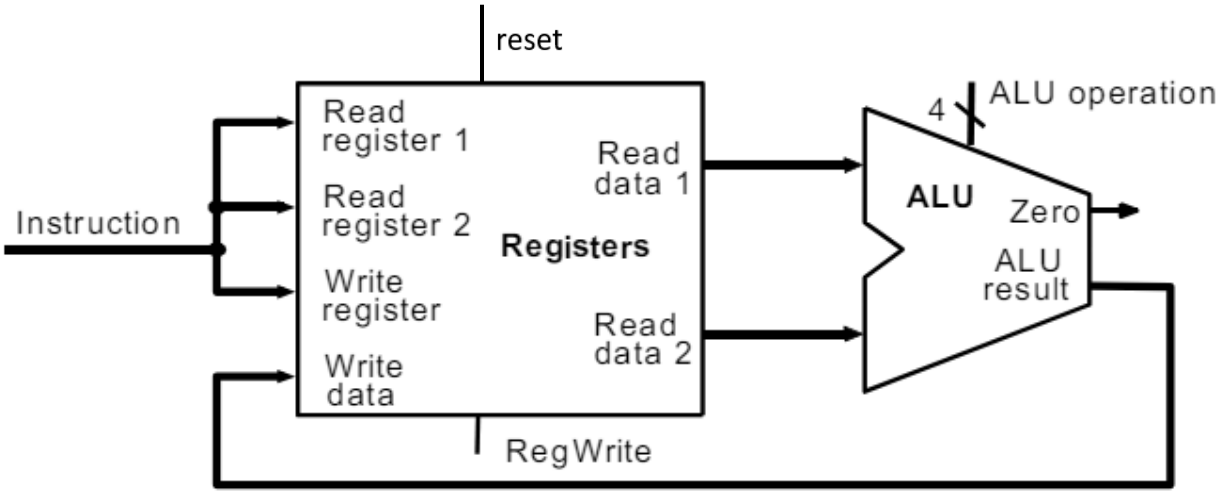


Figure 1: Phase I – processor shape

Phase 2

First of all dividing and conquering is a vital way of implementing our project so we must think about how to divide the project.

The project is divided to the following steps:

1- Register File
   - I- decoder
   - II- enabling registers
   - III- registers
   - IV- 2 MUXs
2- ALU
3- Data path of the processor

Part II

Our aim is to complete the processor to be able to deal with all Instruction formats such as R-Format, I-Format and J-Format. To do that we need controller to handle all those formats. Also, PC connected to MUXs, adders, shifter and sign extender to handle J-Formats. ALU Control is vey much needed to select the type of operation.

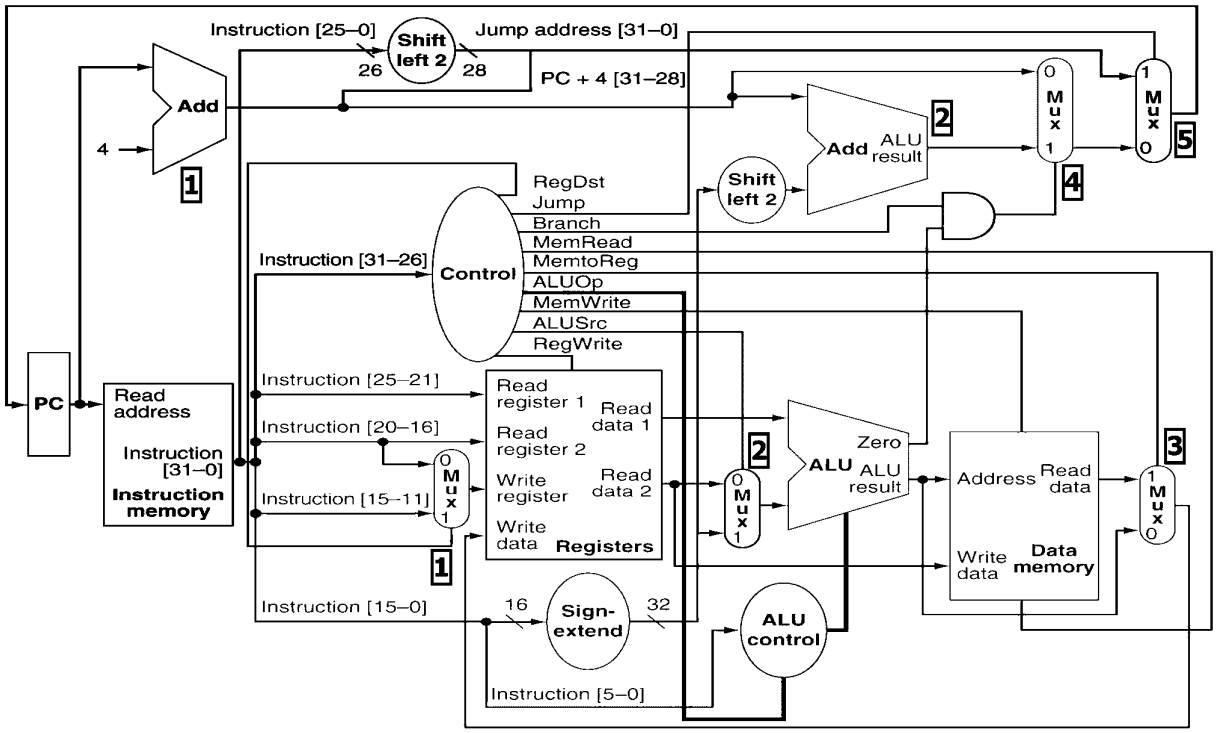


Figure 2: Final processor shape

First of all dividing and conquering is a vital way of implementing our project so we must think about how to divide the project.

The project is divided to the following steps: According to Code!

1- Main
    a. MIPS
        i. Controller
            1. ALU Control
            2. ALU Decoder
        ii. Data Path
            1. Register File
            2. ALU
            3. PC
            4. Sign Extender
            5. 2 Adders
            6. Shifter (Left 2)
    b. Instruction Data memory
    c. Data Memory

Register File

First the Register File consists of several Flop registers which are mainly used for storing data inside of them. Flop registers are simply like a D-Flip Flop but can store more than 1 bit unlike d flip flop, so if we made a 4-bit Flop register we can imagine it to be formed of 4 D-Flip Flops beside each other each one store a bit inside it.

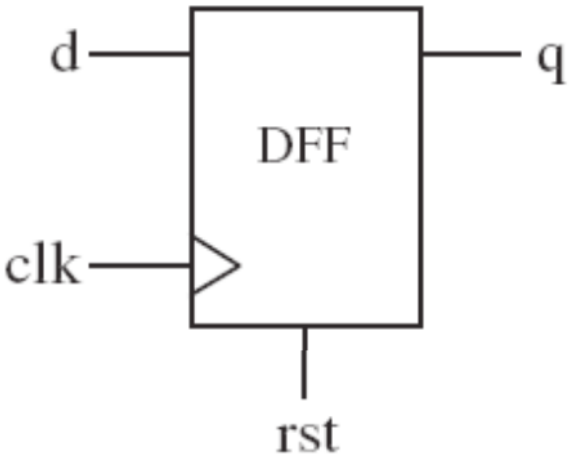


Figure 3: D - Flip Flop

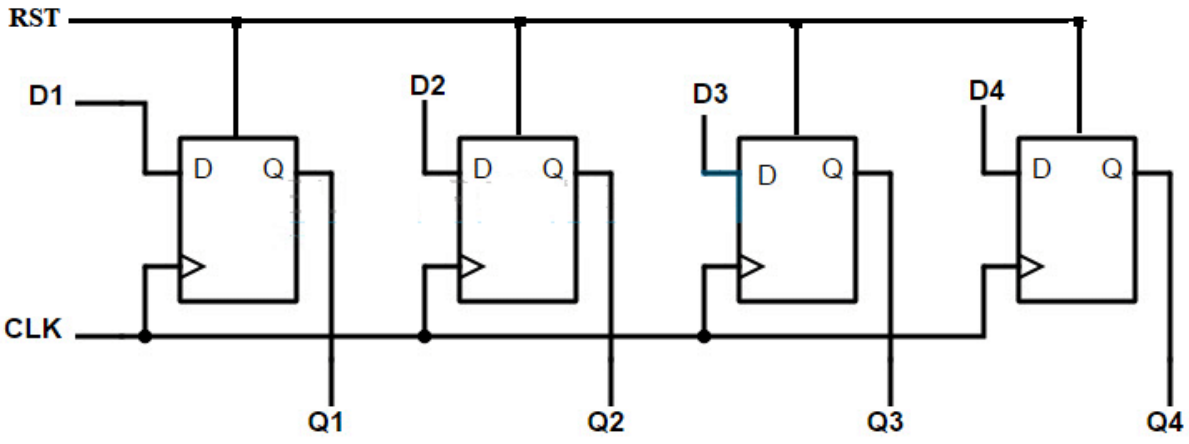


Figure 4: 4-bit Flop Register

Similarly we can make our 32-bit Flop Register.

In Register File we will add 32 from our 32-bit Flop Register we mentioned. Moreover; the Register File add many features to this Flop Registers which are writing and reading data from this registers.

Writing data on the registers

writing data simply can be done by passing the data input into the 32 flop registers in their d so with the next clock trigger it is passed to the its q which is the output. But with this we have a problem we don't want to pass the same data to all the 32 registers so we must make a way to choose which register to pass this data to it and this can be achieved by making a decoder.

Decoder has n inputs and then it controls 2 to the power of n outputs, so we need 32 outputs entering the register enable so we must use 5 to 32 decoder. From this 5 input we consider it as binary write enable selection that controls which one of the 32 outputs has 1 and the other 31 bits have 0 so by this we can choose which one of the 32 register will work.

Finally, in case we don't want any of the registers to work so we must use another input called write enable if it is 1 we can after that choose which of the registers we want to work by the decoder otherwise if it is 0 no register will work. This can be achieved by making the write enable enter 32 AND gates each time with one of the 32 outputs of the decoder then the output of the 32 AND gates can finally be the inputs of the enable of the 32 flop registers.

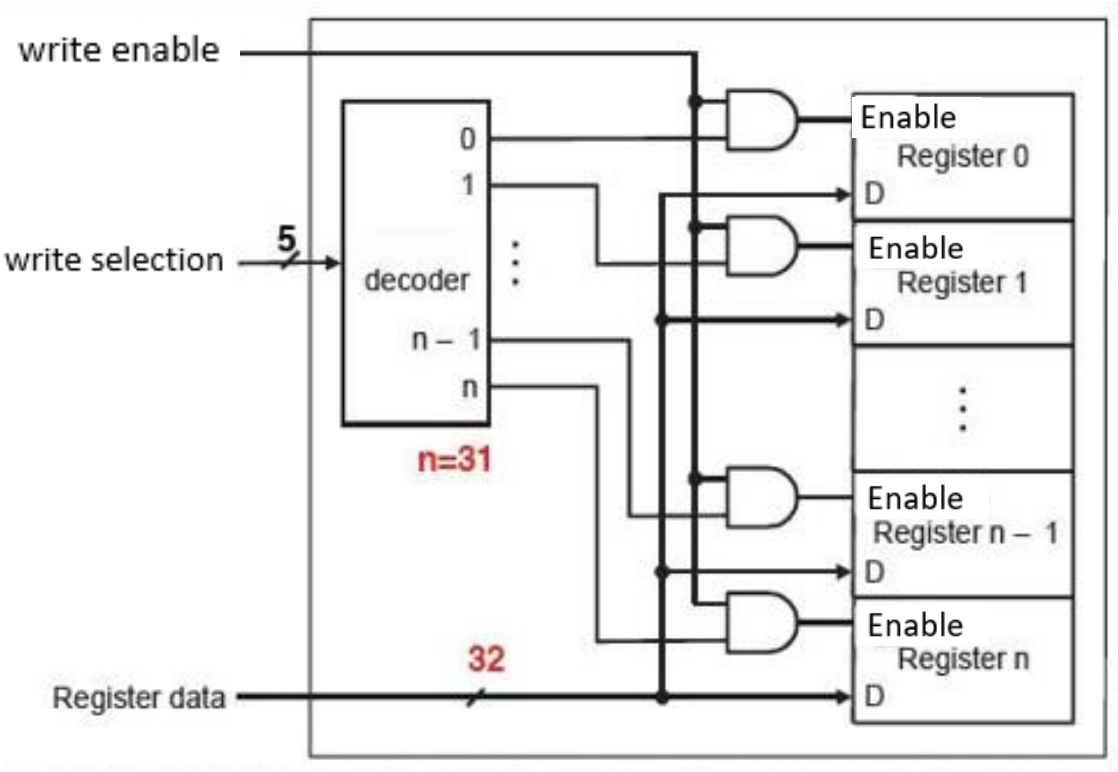


Figure 5: Writing in Register File

Reading data from registers

We want to read from the registers we wrote in and we have 32 outputs from the registers. So we need to choose only 2 registers from the 32 as we will use them in performing R type operation like add, or, and,...etc. Logically we need to select the 2 registers we previously filled them with data in order to read them in this step. In order to choose 2 only from the 32 outputs we will need to make 2 MUXs.

Multiplexers has n inputs with log n with base 2 selectors and 1 output only. So we will make the inputs of the mux to be the 32 q (output) of the registers and the 5 bit selector to choose which one of the outputs of the registers to appear on the output.

We want to choose 2 outputs from the 32 registers at the same time so we will make 2 MUXs with 2 different read selectors.

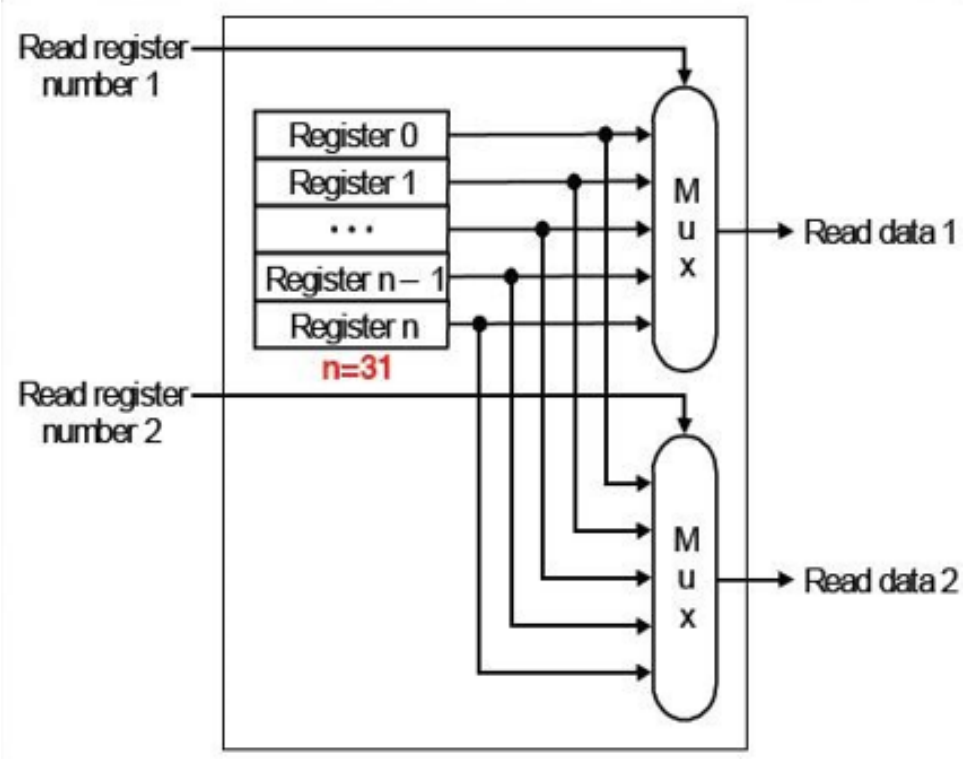


Figure 6: Reading from the registers

Finally we make a trigger which is a clock that performs the previous operation with every clock cycle.

We chose to make the Register File work at every falling edge of the clock (when clock equals 0).

After we finished our Register File, we now can consider it only a block and it doesn't matter now that it is made of decoder, Flop Registers and MUXs.
We consider it a block with following inputs and outputs shown in figure and that achieves abstract concept.

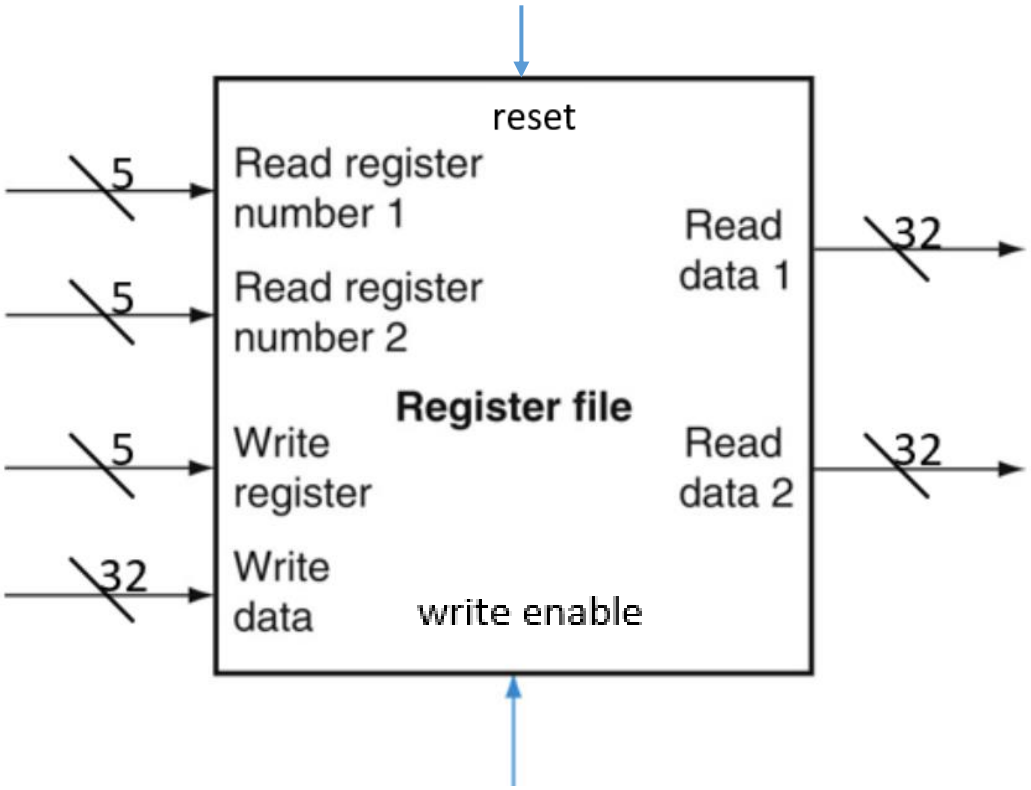


Figure 7: Register File

ALU

Now we move to the second aspect which is the 32 bit ALU. We want to make the ALU which will be a combination of MUXs and the following figure will show its required specification.

ALUOp	Function
0000	AND
0001	OR
0010	ADD
0110	SUB
1100	NOR

We added some assumed specifications to make it more detailed.

ALUOP	FUNCTION
0000	A AND B
0001	A OR B
0010	A+B
0011	not used
0100	A AND B’
0101	A OR B’
0110	A-B
0111	SLT
1000	A’ AND B
1001	A’ OR B
1010	B – A
1011	not used
1100	NOR = A’ AND B’
1101	not used
1110	not used
1111	not used

Figure 8: ALU specifications

The ALU has a main mux which chooses between AND, OR, ADD.

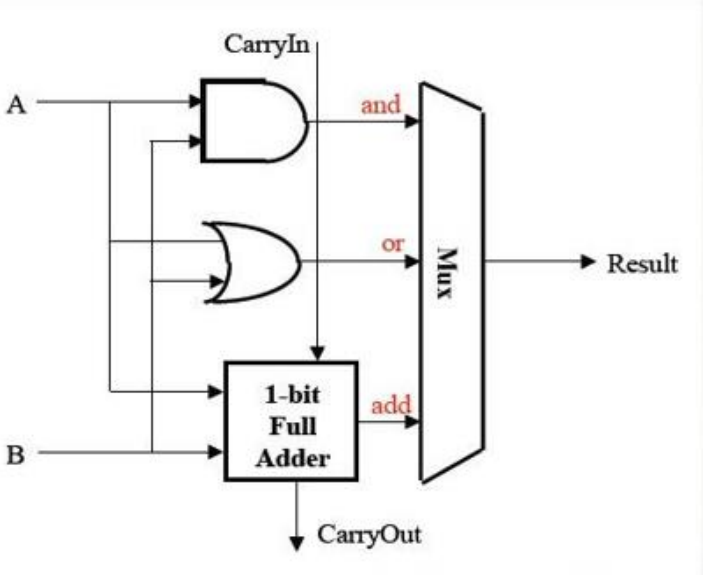


Figure 9: 1 bit ALU

If we make it for example a 4 bit ALU its carry out will be passed to the following 1-bit Full adder but in this project we don't care about the carry out or in and we can imagine it the carry in is always 0.

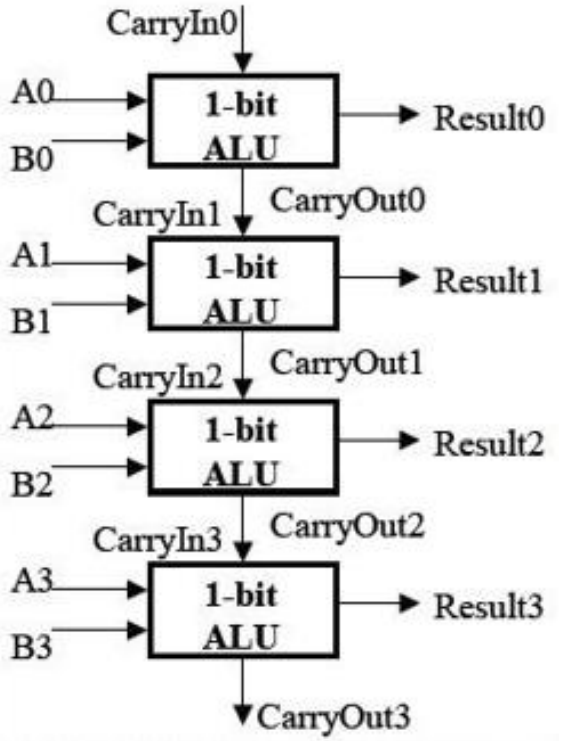


Figure 10: 4 bit ALU

Similarly we can make our 32 bit ALU.

Now we want to add the subtraction feature mainly to make A – B, so we make an initial mux to choose whether its addition or subtraction between A – B and the select of the MUX will be the third bit which will be a representative for whether the second input is remained as it is or it will be complemented. If we chose it to be subtraction then we will make the carry input with 1 and that's how subtraction work as it is can be represented by A - NOT(B) + 1.

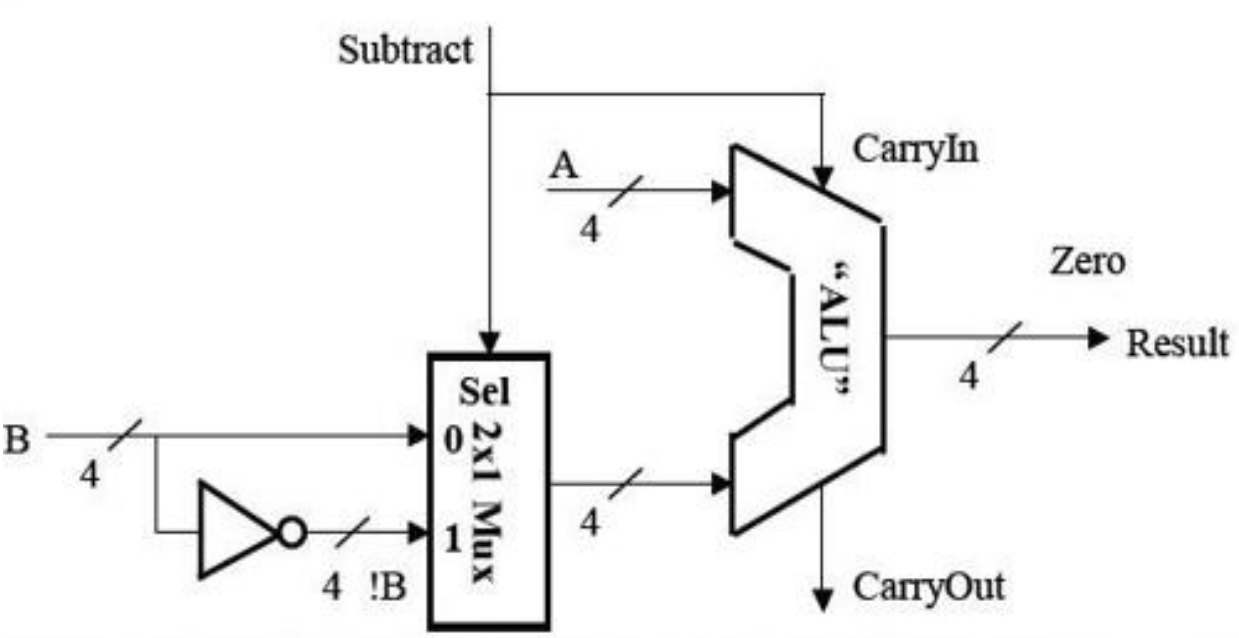


Figure 11: Subtraction in the ALU

similarly we can add another MUX for the A input to make B-A.

Also we have the zero flag which has output 1 when the result is equal to zero and that can be performed by adding all the results bit with nor. It can be implemented on VHDL much easier.

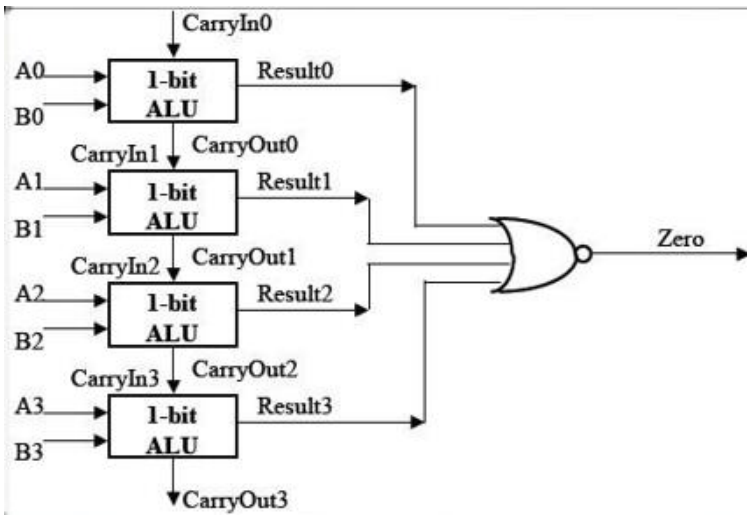


Figure 12: Zero Flag

Finally the other cases will be discussed in the coding better.

Now we will work with abstract model of ALU as shown in the figure.

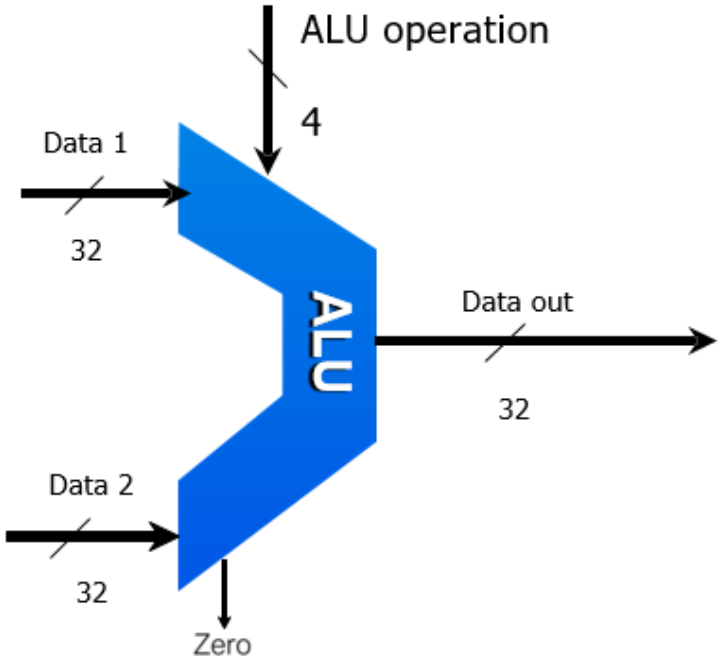
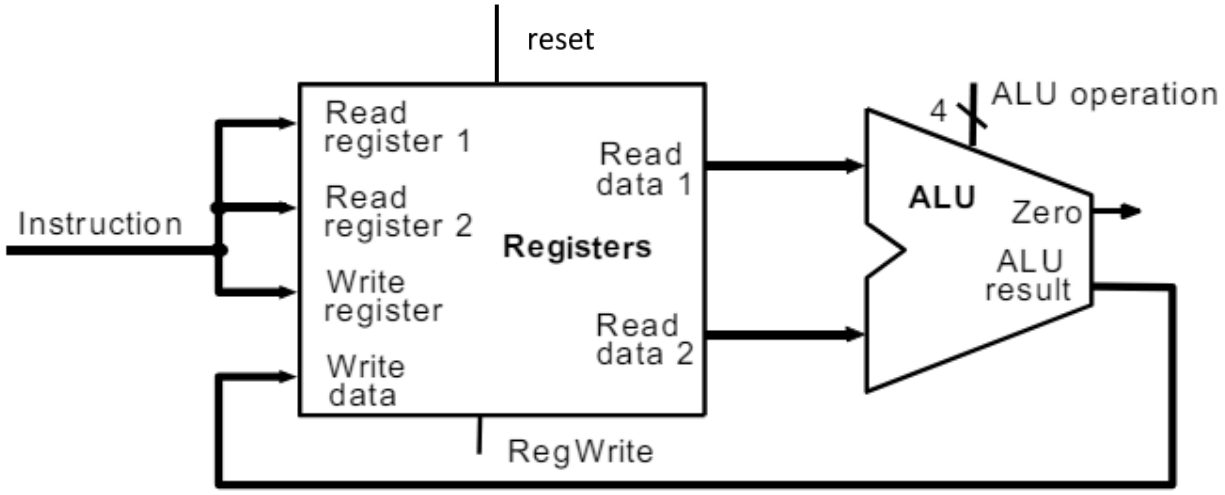


Figure13: ALU

Data path of the processor (phase 1)

In the data path we will connect the Register File with ALU like the following figure which we have shown earlier.



The 2 outputs from the Register File which are the (Read data 1 & Read data 2) will be the Inputs in the ALU as to perform the desired operations on them such as add, and , or,....etc. Those 2 outputs from the Register file are the 2 registers which carry the data we want to perform operations on them so they enter the ALU as inputs.

The output of ALU will enter the Register File in the Write data input. As the output from the operations done in the ALU will be written inside a certain register in the Register File.

Finally the Instruction input which will be 32 bit, they represent the R-format & I-format & J-format. But, in this milestone we only use the R-format instructions so the opcode is always zero so we don't need the most significant 6 bits.
Also we will choose which R-format operation we want from the ALU operation so we don't need the least significant 5 bits.
We don't perform any Shift operation so we don't need the shamt bits which are the bits (10-6).

In the end we need only the bits of rs,rt,rd from the instruction input.

rs bits which are (25-21) will be passed to the Read register 1.
rt bits which are (20-16) will be passed to the Read register 1.
they will select which registers to read from.

rd bits which are (15-11) will be passed to the Write register.
this input will choose which register to write in it.

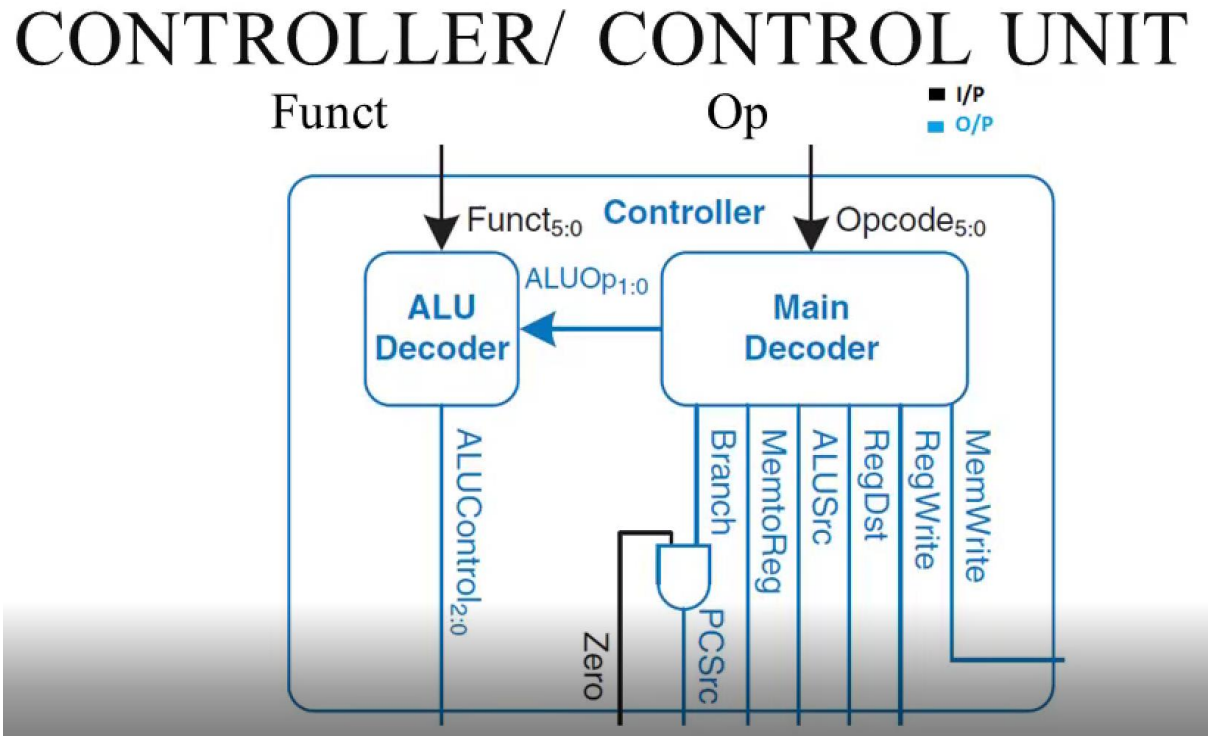
Field bit positions	31-26	25-21	20-16	15-11	10-6	5-0
No. of bits	6	5	5	5	5	6
R-format	op	rs	rt	rd	shamt	funct
I-format	op	rs	rt	16- bit immediate/address		
J-format	op	26-bit address				

Figure 14: Instruction

Controller

The Control Unit (CU) is a component of computer's central processing unit (CPU) that directs the operation of the processor. It tells the computer's memory, ALU and others how to respond to the instruction that have been sent to the processor. The Control Unit (CU) is separated to 2 parts:

1. Main Decoder
2. ALU Decoder

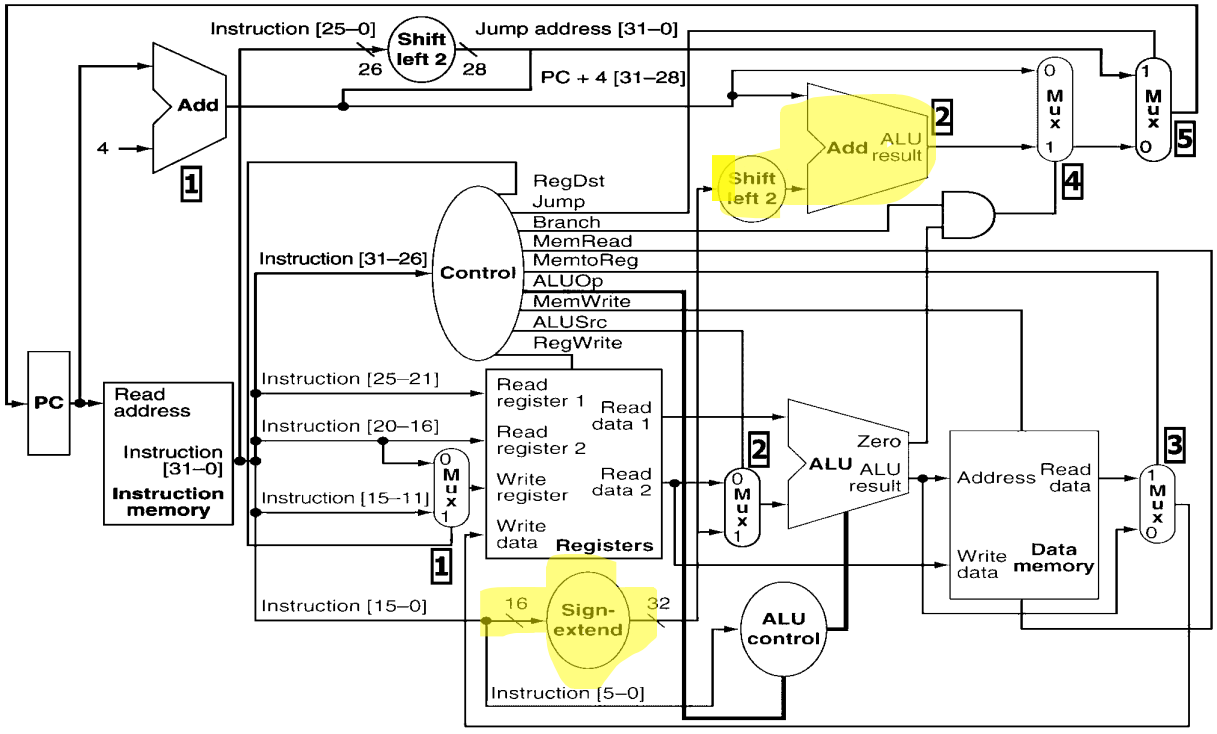


1. Main Decoder

Signal Name	Effect when de-asserted (0)	Effect when asserted (1)
Regwrite	None	The register on the write register input is written with the value on the write data input
Regdst	The register destination number for the write register comes from the rt field (bits 20:16)	The register destination number for the write register comes from the rd field (bits 15:11)
Alusrc	The second ALU operand comes from the second register file output(read data 2)	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
Branch	PC+4 to be placed in PC	Target address of the branch instruction (PC+4 +address×4) is placed in PC
Memwrite	None	Activates write operation to data memory.
MemtoReg	The value fed to register write dat input comes from ALU PC+4) is placed into PC	The value fed to the register write data input comes from date memory
Jump	The output of the branch MUX (target address or PC+4) is placed into PC	The target address of the jump (PC [31-28]: address*4 is placed into PC)
ALUop	00 for lw (load), sw (store), add | 01 for beq | 10 for R-Format	

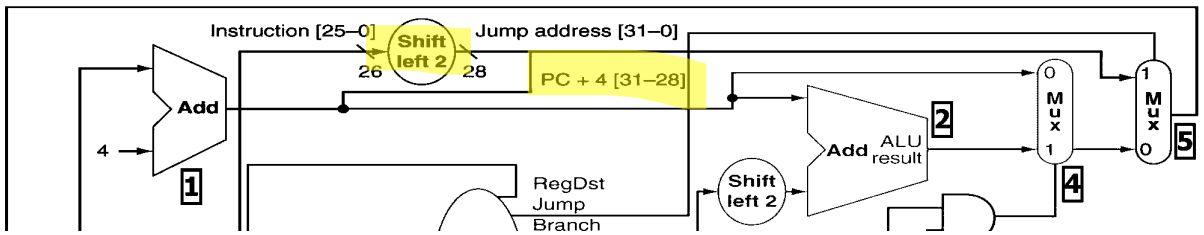
OPCode is the code that tells that CPU that this instruction is either R-Format or I-Format or J-Format

I-Format (Branch Instruction)



In I-Fomat there is 16-bit address to preform this instruction we sign extend this 16-bit address to 32-bit address then we multiply by 4 (Left 2) and add program counter + 4 to get the desired address.

J-Format



In J-Format there is 26-bit address the CPU contaminate extra 2-bit using shift left by 2, lastly we take the remaining 4-bits from the program counter.

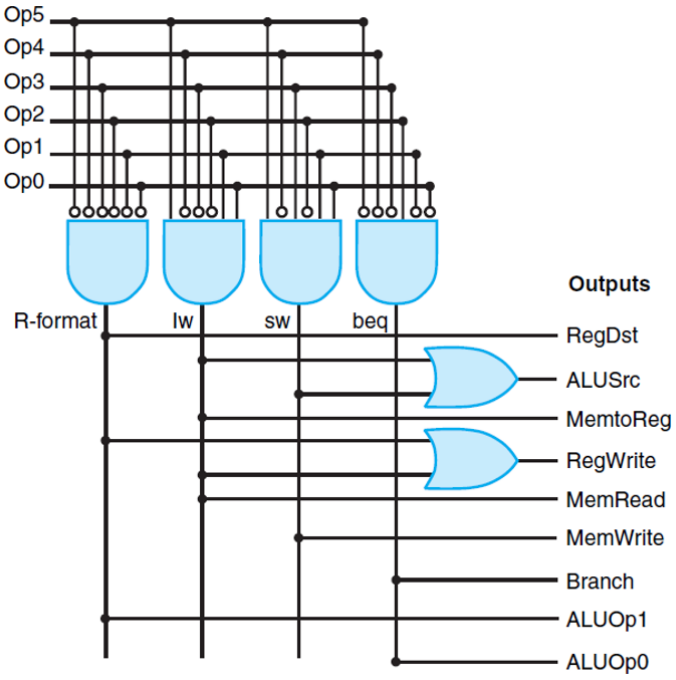
In case of normal PC + 4 to fetch next instruction line both MUXs will be Zero to pass PC + 4.

Main Decoder Function Table

Input or output	Signal name	R-format	lw	sw	beq
Inputs	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
Outputs	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Hardware Design

This is the hardware design to the Control Unit (CU), It manipulates the data flow according to type of inst. It used a group of ANDs and ORs to get the data. Then the outputs signals lines selects / controls some MUXs.



ALU Decoder

ALU Decoder Decides which operation will be performed by the ALU.

Input1: ALUOp, which is a 2-bit control signal indicating a 00 (add for loads and stores), a 01 (subtract for branches), and a 10 (use the *funct.* field).

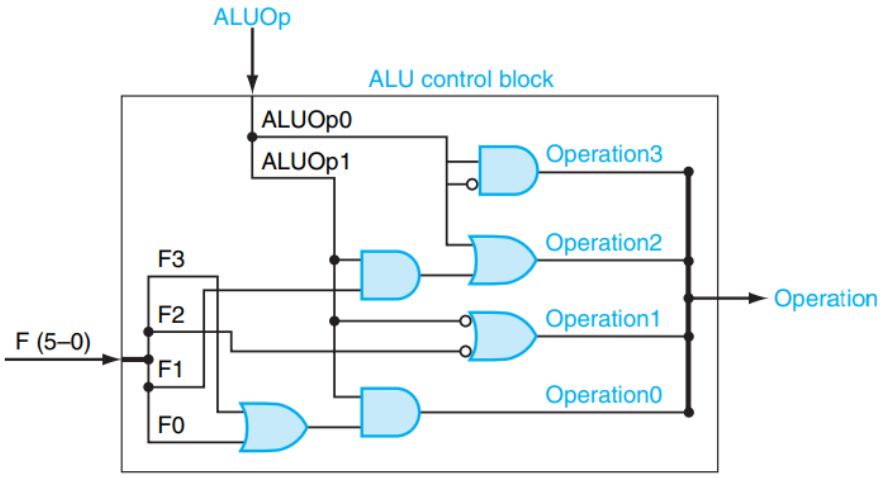
Input2: *funct.* field. Remember that for R-type instructions, the opcode field is always zero and the *funct.* field is used to determine the type of operation to perform.

The output of the ALU control unit is a 3-bit field that is fed into the ALU to select the operation to be performed.

ALU Decoder Function Table

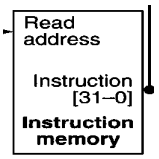
Instruction opcode	ALUOp	Instruction operation	Funct field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	010
SW	00	store word	XXXXXX	add	010
Branch equal	01	branch equal	XXXXXX	subtract	110
R-type	10	add	100000	add	010
R-type	10	subtract	100010	subtract	110
R-type	10	AND	100100	and	000
R-type	10	OR	100101	or	001
R-type	10	set on less than	101010	set on less than	111

Hardware Design



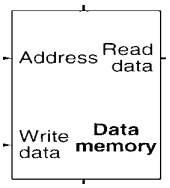
Instruction Memory

It is a memory that holds the instruction that is going to be fetched, decoded and executed. It has 2 ports. First port read the address and the other output the 32-bit instruction code.



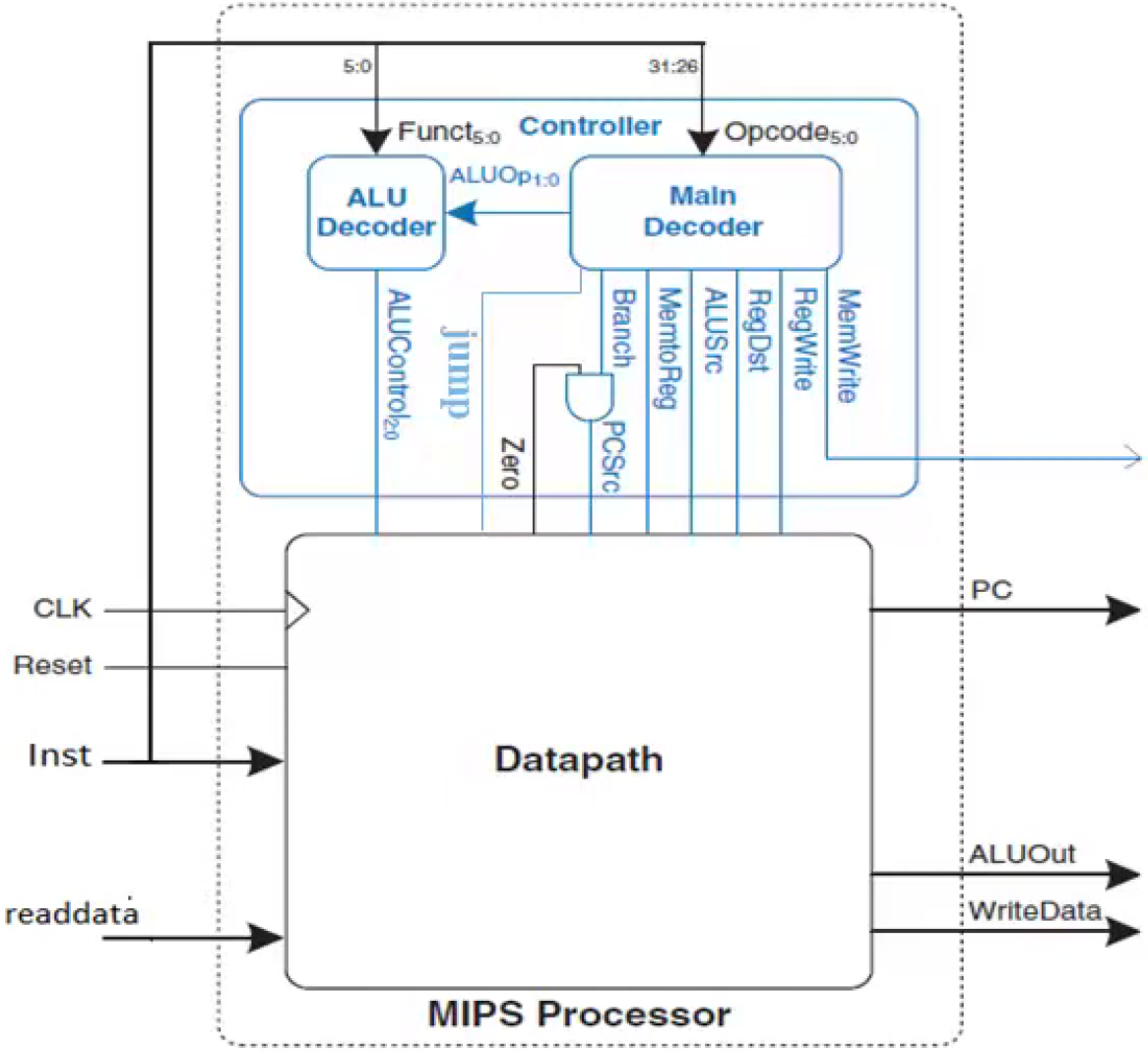
Data Memory

It is a memory that holds the data inside the computer. It has 5 ports. First port take the instruction address another 2 ports either to read the data or to write overt the data. Lastly 2 ports to enable reading or writing.

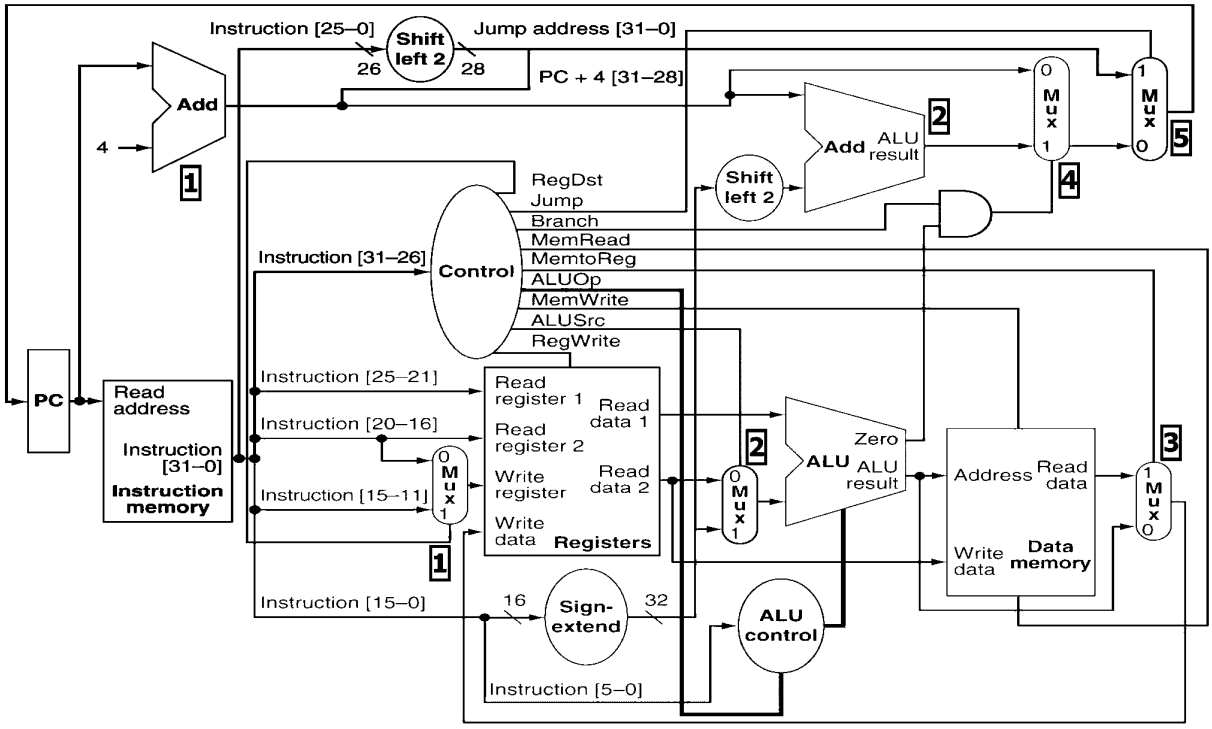


Updated DataPath

In this stage we connect all component together. This stage might seems easy but it is very crucial part because there are some data manipulation in this stage also connecting all these different types of components might seems overwhelming work.



We create a package then introduce each component / module. Afterwards, we connect the ports with each other as seen in code preview.



PC is connected to instruction memory to give it the instruction address that is going to be fetched. Also this data line has a branch connected to an adder that adds only 4 which is the address that is going to fetched (Next Step). Instruction memory is connected to register file to write/read data. The control unit (CU) detects which instruction type this current instruction is and the instruction is decoded and executed accordingly.

- If R-Format: opcode is read and ALU control decides what type of operation to prefomed.
- If I-Format: The 16-bit address is extended to 32-bit address then it is multipled by 4 finally, we add the PC + 4.
- If J-Format: The 26-bit address is contaminated with an extra 2-bit, lastly we take the remaining 4-bits from the program counter. After, operating this we have the address we want to jump to.

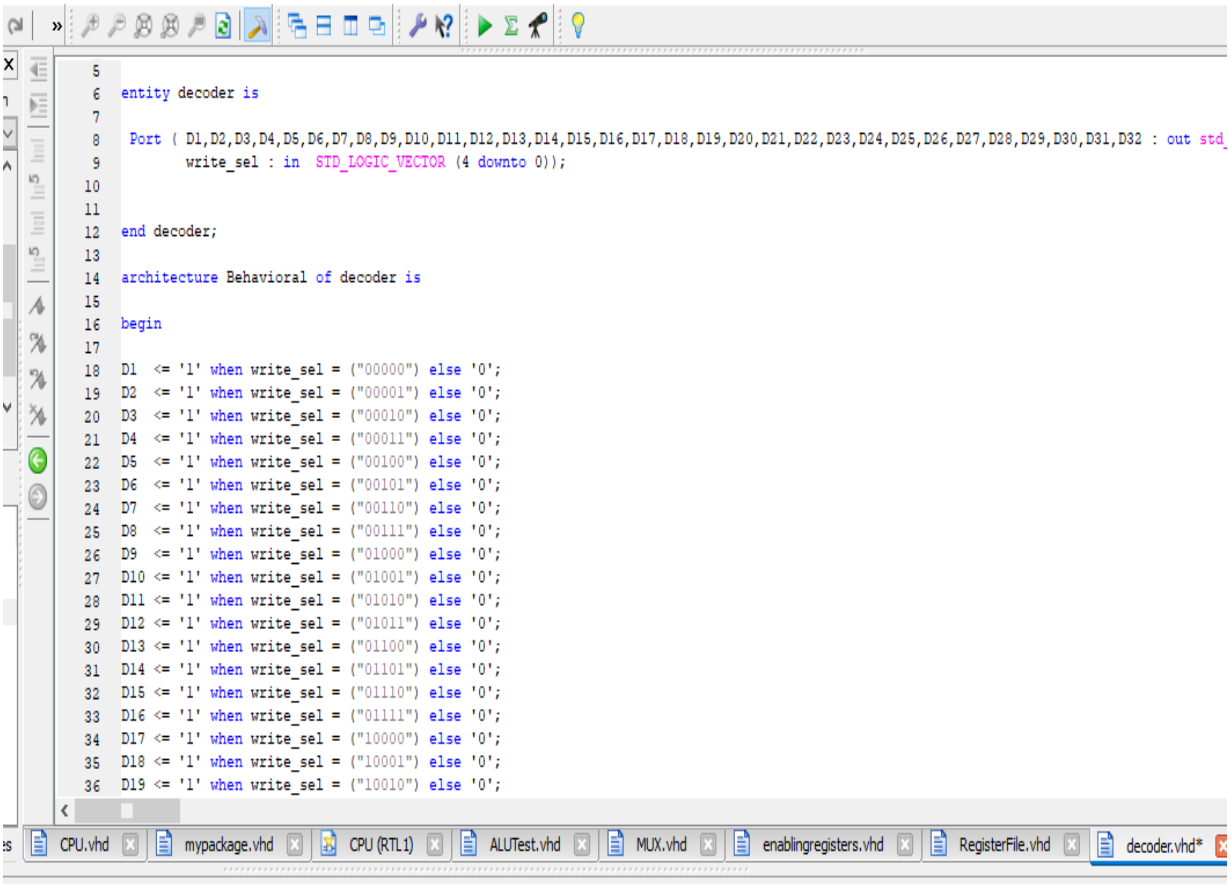
Finally, ALU make changes to data memory.

VHDL coding of blocks with synthesized RTL schematic

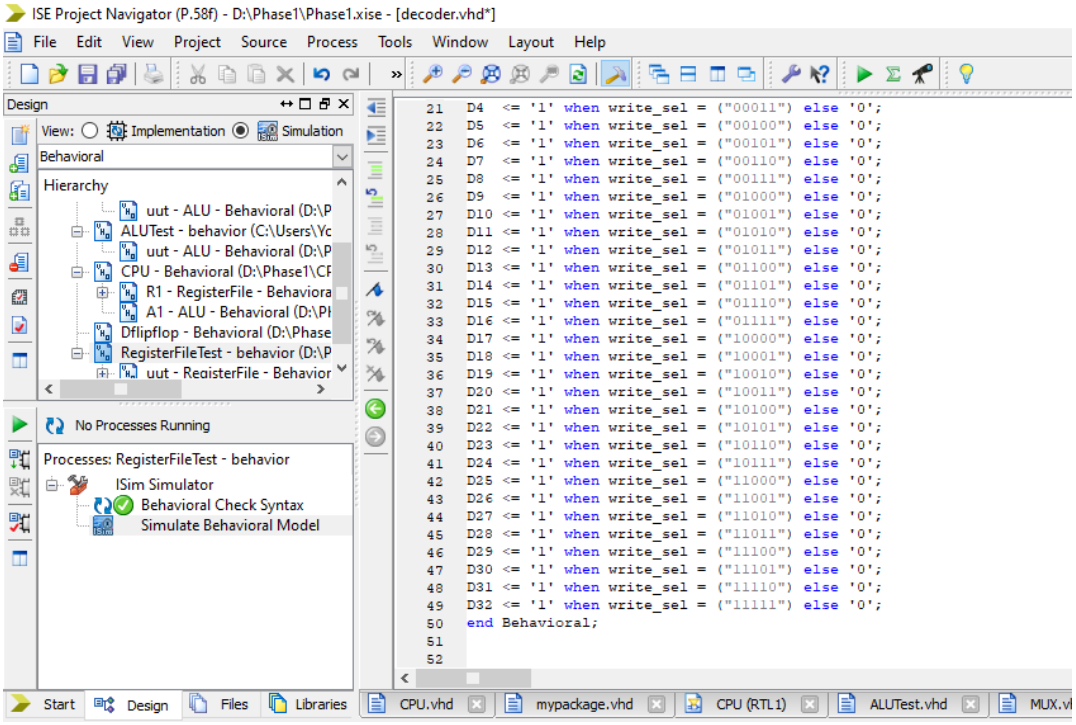
1- Register File

In this project we performed every part in the Register File separately as each one is implemented in a separate module.
Then we add its component in a package then we include this package in the main Register File main module which is called “RegisterFile”

I- decoder

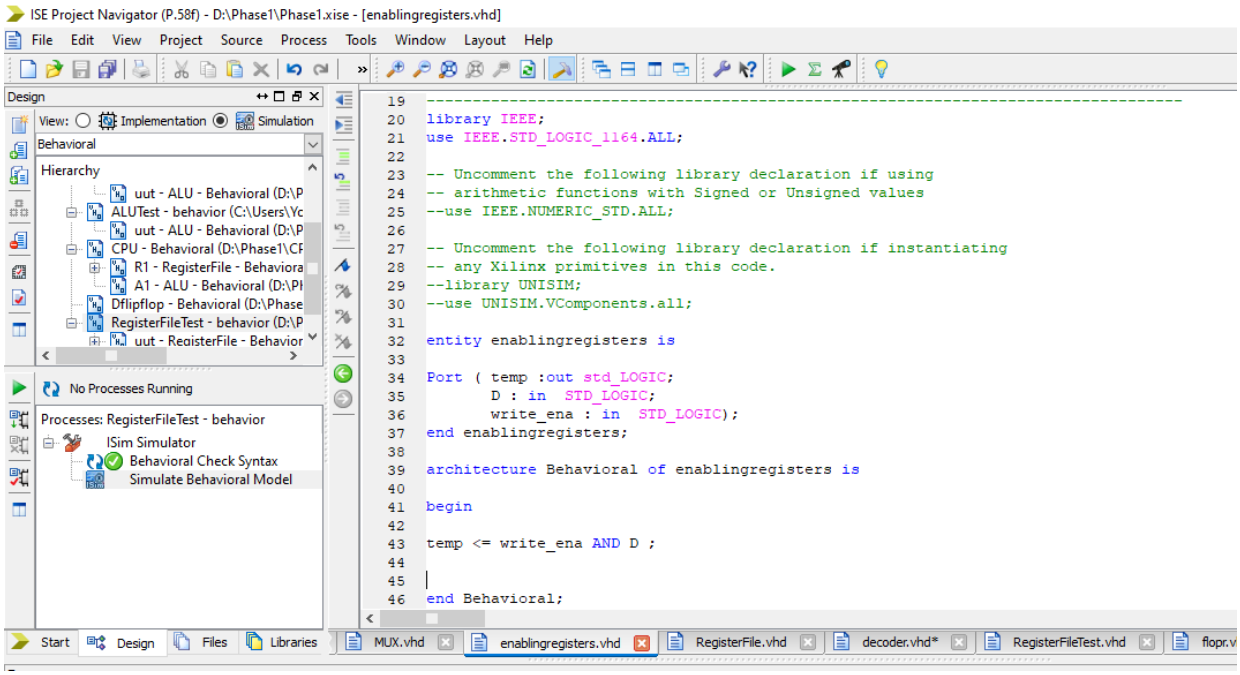


```
entity decoder is

 Port ( D1,D2,D3,D4,D5,D6,D7,D8,D9,D10,D11,D12,D13,D14,D15,D16,D17,D18,D19,D20,D21,D22,D23,D24,D25,D26,D27,D28,D29,D30,D31,D32 : out std
        write_sel : in  STD_LOGIC_VECTOR (4 downto 0));


end decoder;

architecture Behavioral of decoder is

begin

D1  <= '1' when write_sel = ("00000") else '0';
D2  <= '1' when write_sel = ("00001") else '0';
D3  <= '1' when write_sel = ("00010") else '0';
D4  <= '1' when write_sel = ("00011") else '0';
D5  <= '1' when write_sel = ("00100") else '0';
D6  <= '1' when write_sel = ("00101") else '0';
D7  <= '1' when write_sel = ("00110") else '0';
D8  <= '1' when write_sel = ("00111") else '0';
D9  <= '1' when write_sel = ("01000") else '0';
D10 <= '1' when write_sel = ("01001") else '0';
D11 <= '1' when write_sel = ("01010") else '0';
D12 <= '1' when write_sel = ("01011") else '0';
D13 <= '1' when write_sel = ("01100") else '0';
D14 <= '1' when write_sel = ("01101") else '0';
D15 <= '1' when write_sel = ("01110") else '0';
D16 <= '1' when write_sel = ("01111") else '0';
D17 <= '1' when write_sel = ("10000") else '0';
D18 <= '1' when write_sel = ("10001") else '0';
D19 <= '1' when write_sel = ("10010") else '0';
```



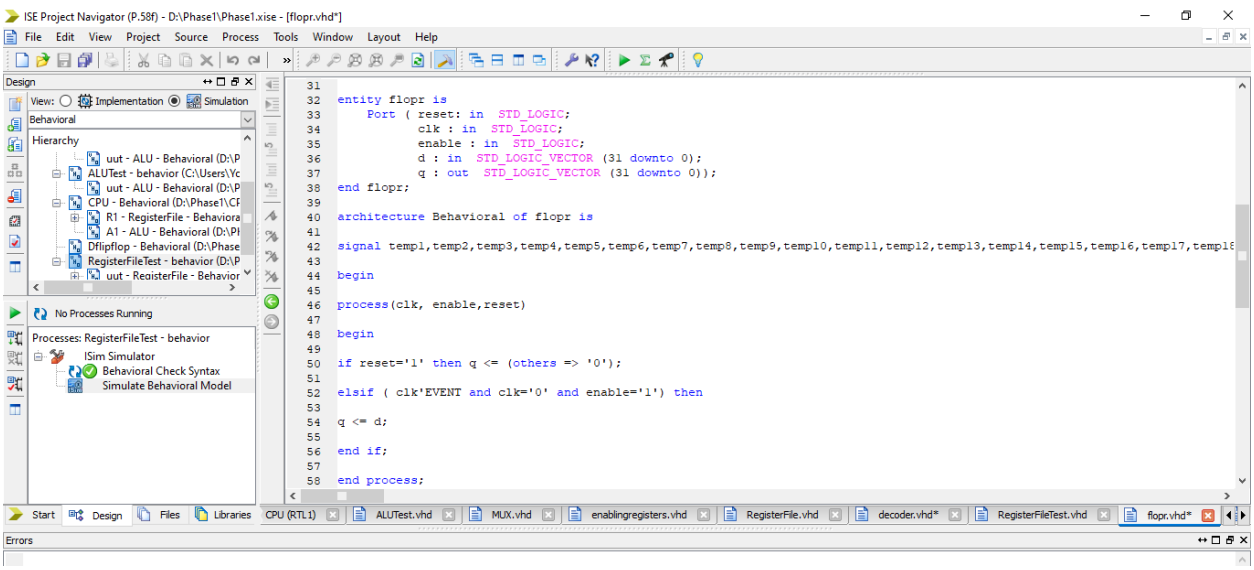
Here we need 32 bit output from the decoder with a selector 5 bits.
so we define 32 d outputs called d1,d2,d3,...etc.
when selector is “00000” d1 is equal to 1 and all other outputs equal 0
when selector is “00001” d2 is equal to 1 and all other outputs equal 0
and so on until we reach d32 which will be 1 when selector “11111”.

II- enabling registers



we make the output of each decoder pass to AND gate with the write enable and then we declare this statement 32 times in RegisterFile main as it will be shown later.

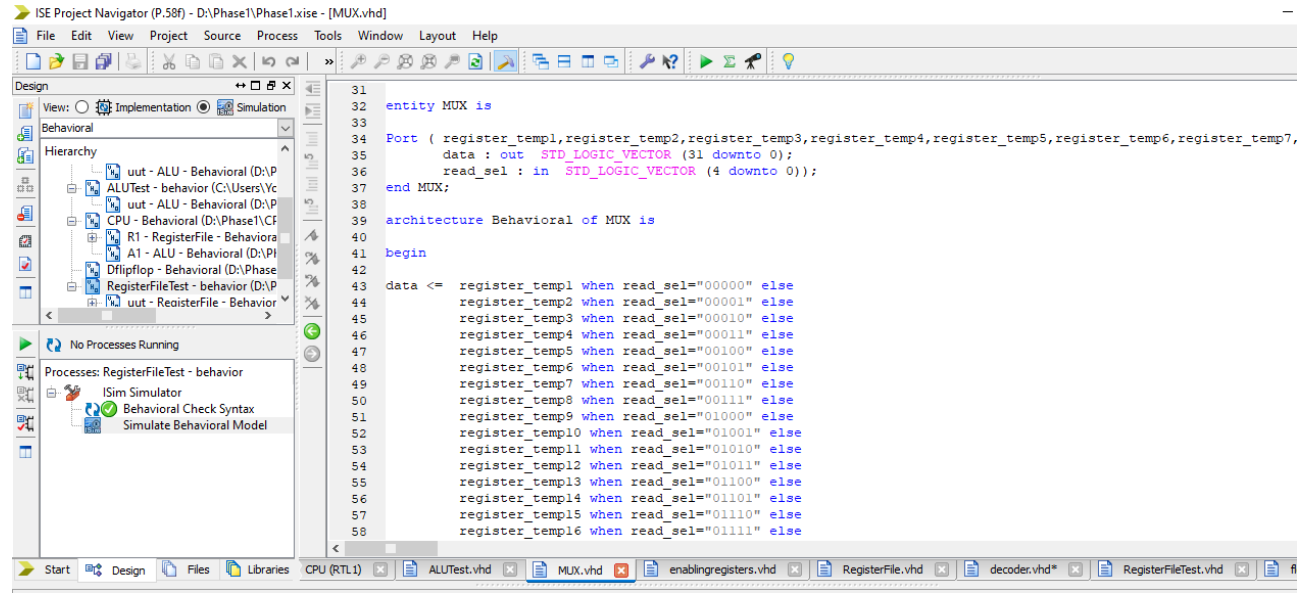
III- registers

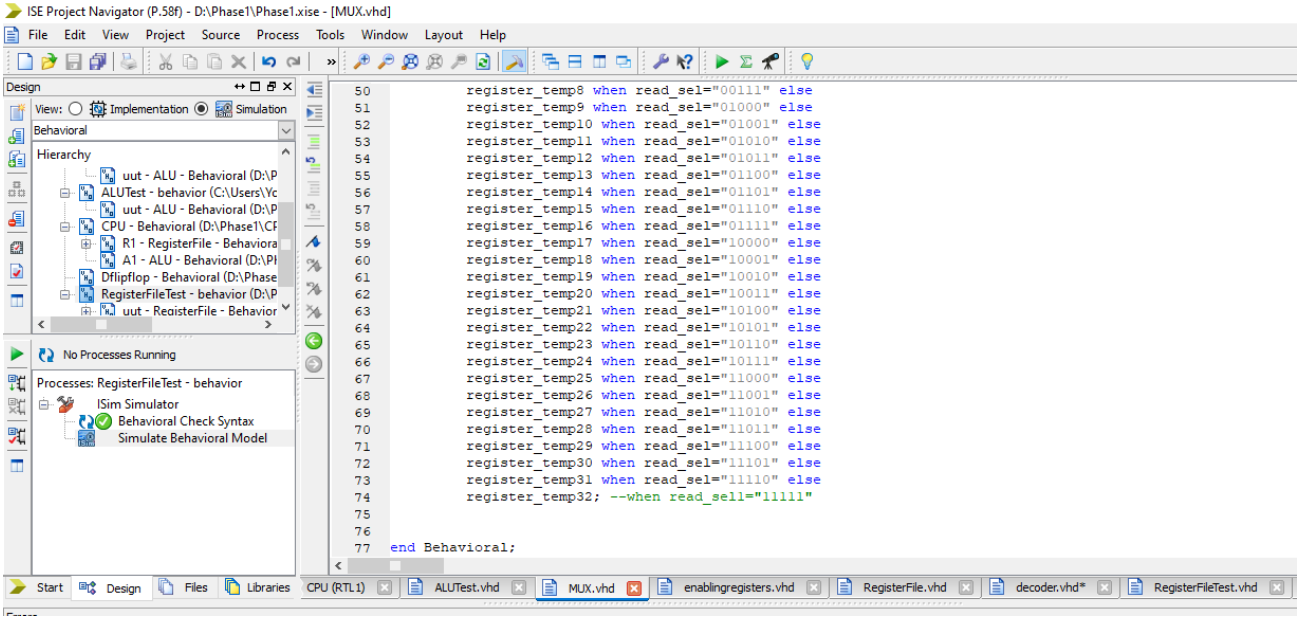


We make the 32-bit flop register like this:
It sees first if the reset is equal to 0 then the q will be otherwise the clock must be at falling edge and the enable equals 1 then the q is equal to d

Finally we will declare it 32 times in the main Register File

IV- 2 MUXs





MUX can be performed by entering 32 inputs and the output is chosen according to the selectors.

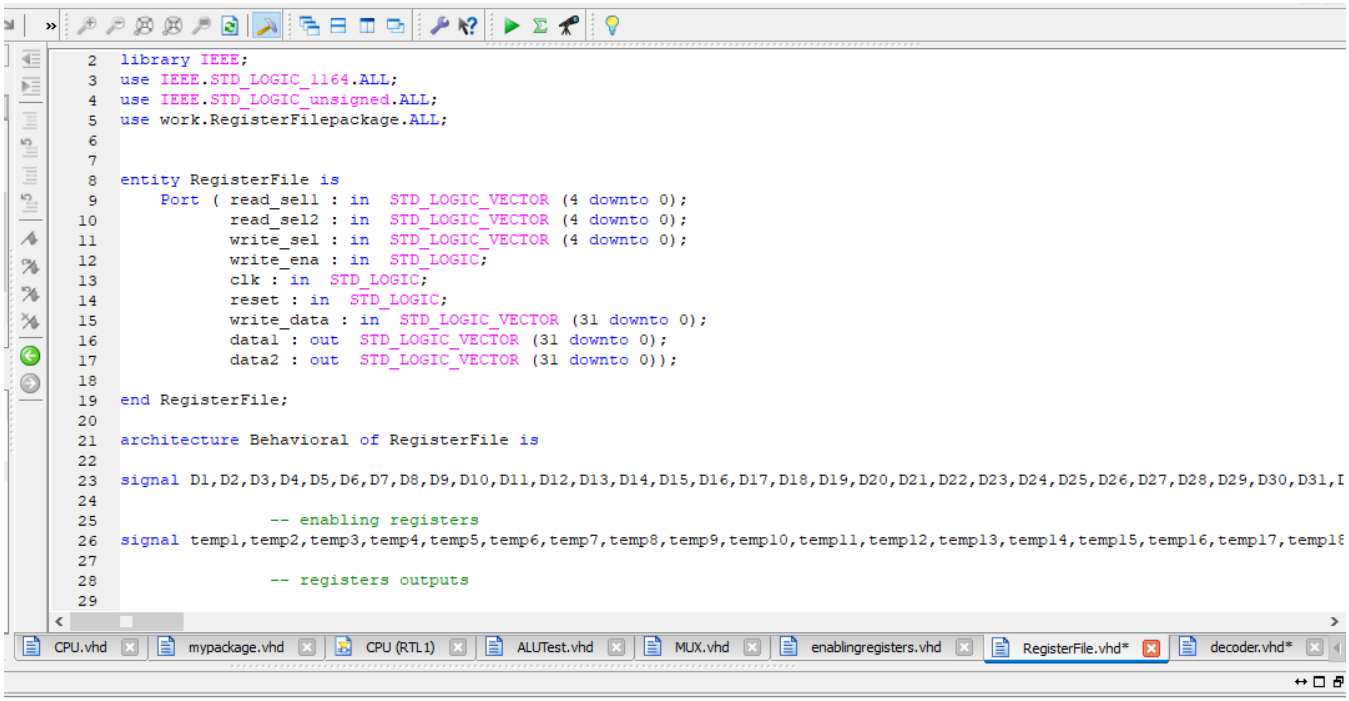
AS MUX output will be register temp1 when selector is “00000” else

equals register temp2 when selector is “00001” else

and so on until we reach register temp32

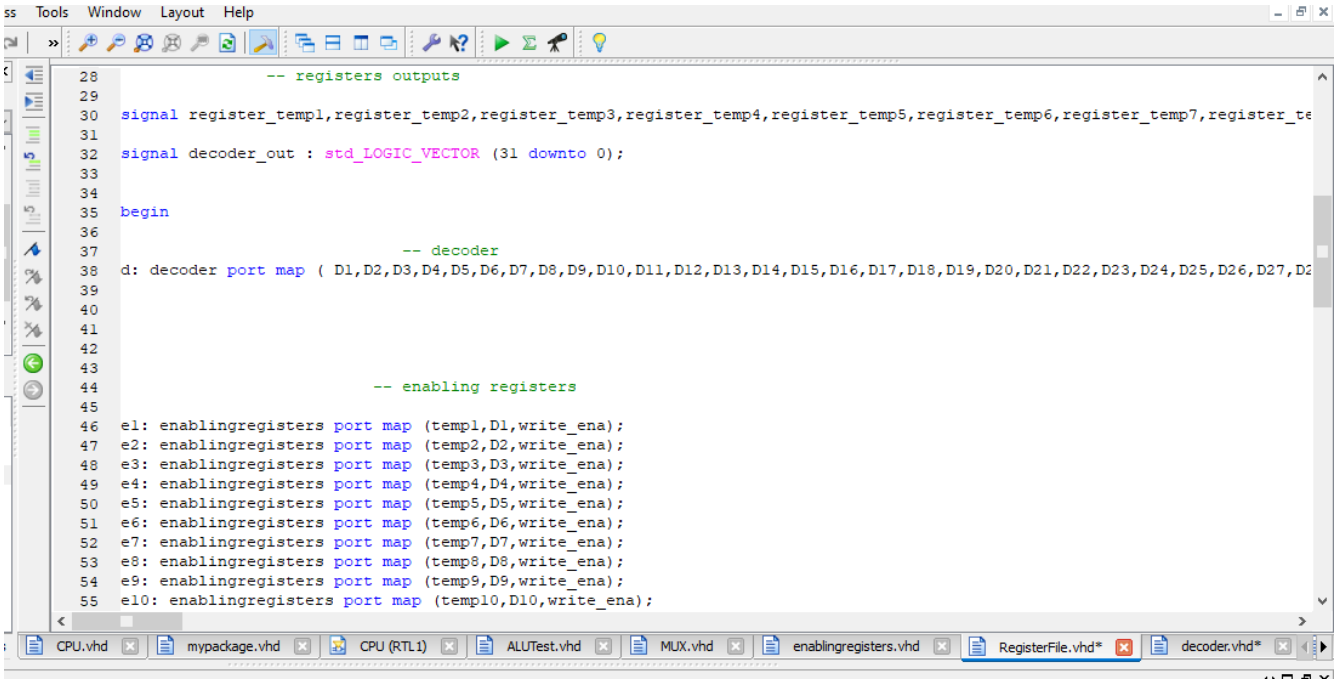
Then we need to show 2 outputs so we will declare 2 MUXs in the main.

RegisterFile main module



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;
use work.RegisterFilepackage.ALL;


entity RegisterFile is
    Port ( read_sel1 : in  STD_LOGIC_VECTOR (4 downto 0);
           read_sel2 : in  STD_LOGIC_VECTOR (4 downto 0);
           write_sel : in  STD_LOGIC_VECTOR (4 downto 0);
           write_ena : in  STD_LOGIC;
           clk : in  STD_LOGIC;
           reset : in  STD_LOGIC;
           write_data : in  STD_LOGIC_VECTOR (31 downto 0);
           data1 : out  STD_LOGIC_VECTOR (31 downto 0);
           data2 : out  STD_LOGIC_VECTOR (31 downto 0));

end RegisterFile;

architecture Behavioral of RegisterFile is

signal D1,D2,D3,D4,D5,D6,D7,D8,D9,D10,D11,D12,D13,D14,D15,D16,D17,D18,D19,D20,D21,D22,D23,D24,D25,D26,D27,D28,D29,D30,D31,D

               -- enabling registers
signal temp1,temp2,temp3,temp4,temp5,temp6,temp7,temp8,temp9,temp10,temp11,temp12,temp13,temp14,temp15,temp16,temp17,temp18

               -- registers outputs

```

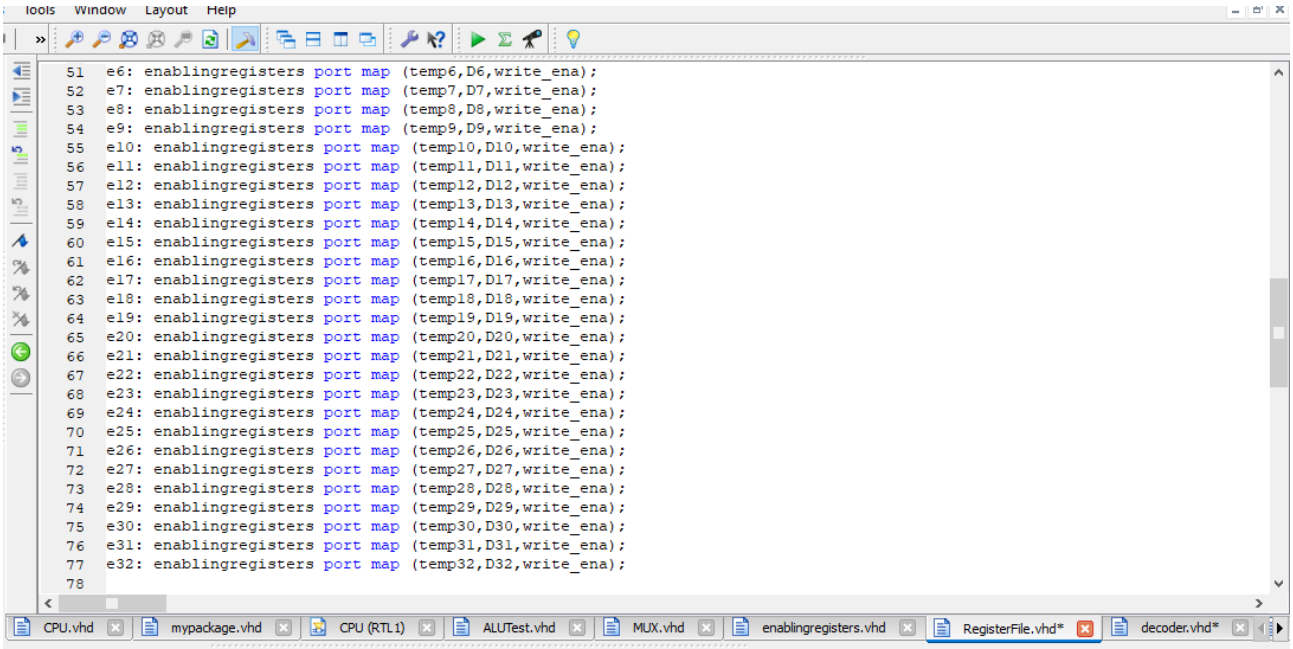
CPU.vhd | mypackage.vhd | CPU (RTL1) | ALUTest.vhd | MUX.vhd | enablingregisters.vhd | RegisterFile.vhd* | decoder.vhd*



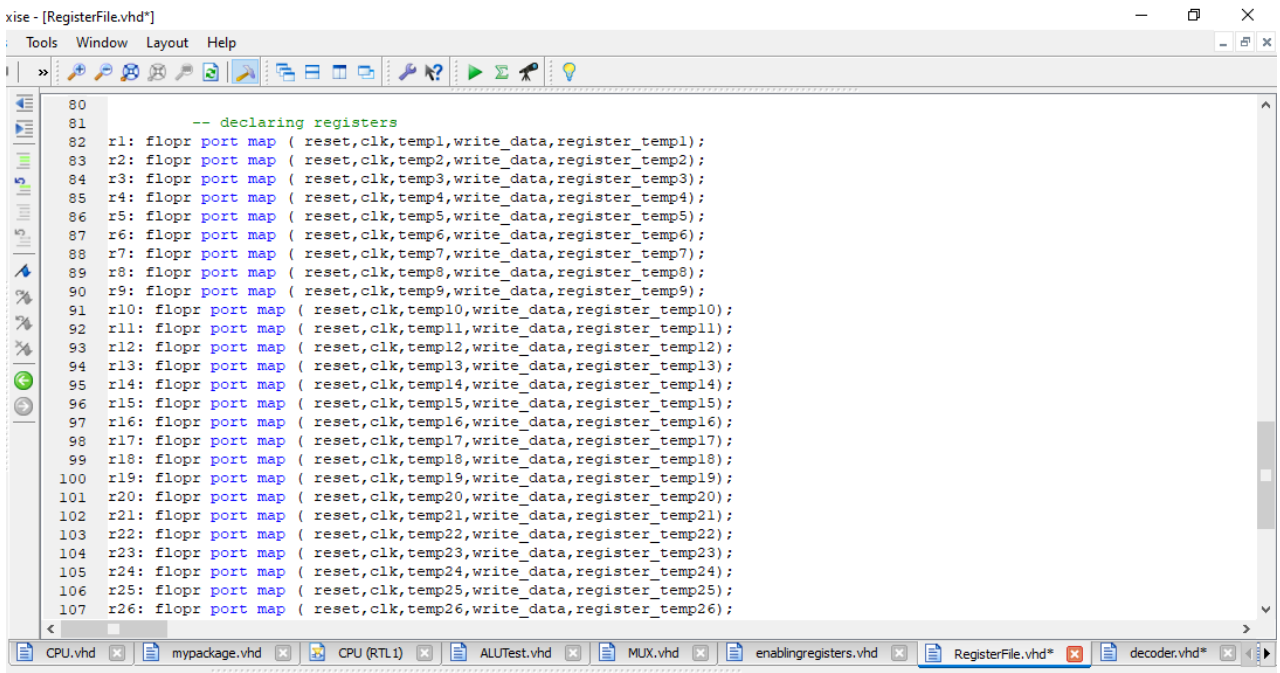
ss Tools Window Layout Help

```
               -- registers outputs

signal register_temp1,register_temp2,register_temp3,register_temp4,register_temp5,register_temp6,register_temp7,register_te

signal decoder_out : std_LOGIC_VECTOR (31 downto 0);


begin

                              -- decoder
d: decoder port map ( D1,D2,D3,D4,D5,D6,D7,D8,D9,D10,D11,D12,D13,D14,D15,D16,D17,D18,D19,D20,D21,D22,D23,D24,D25,D26,D27,D2




                           -- enabling registers

e1: enablingregisters port map (temp1,D1,write_ena);
e2: enablingregisters port map (temp2,D2,write_ena);
e3: enablingregisters port map (temp3,D3,write_ena);
e4: enablingregisters port map (temp4,D4,write_ena);
e5: enablingregisters port map (temp5,D5,write_ena);
e6: enablingregisters port map (temp6,D6,write_ena);
e7: enablingregisters port map (temp7,D7,write_ena);
e8: enablingregisters port map (temp8,D8,write_ena);
e9: enablingregisters port map (temp9,D9,write_ena);
e10: enablingregisters port map (temp10,D10,write_ena);
```

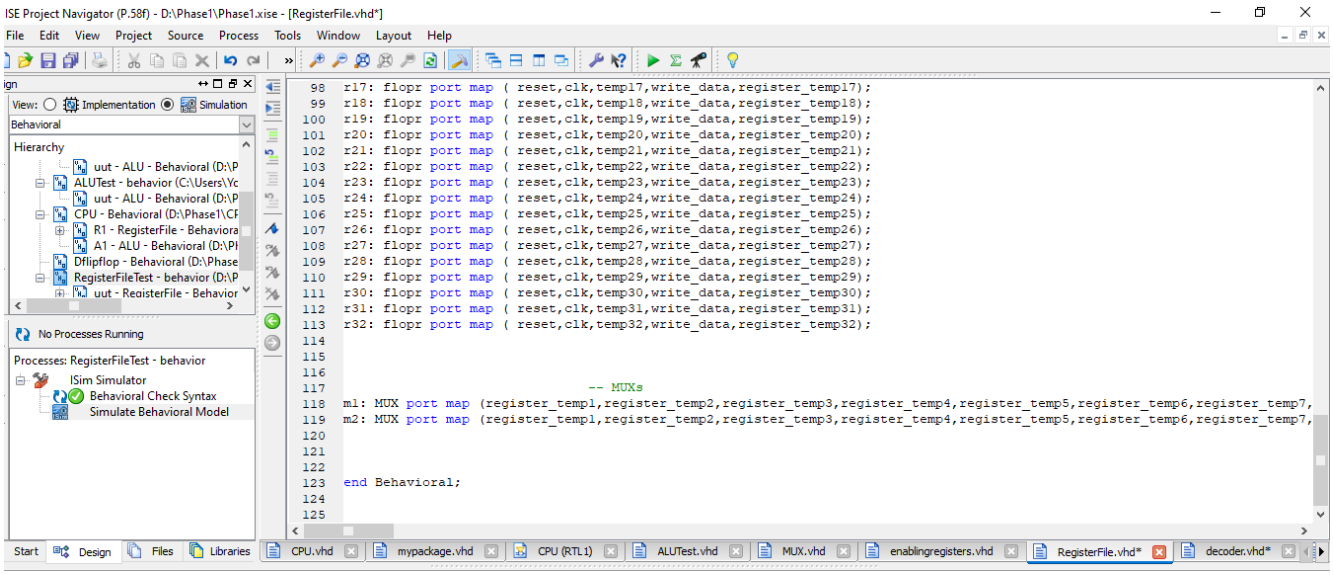
CPU.vhd | mypackage.vhd | CPU (RTL1) | ALUTest.vhd | MUX.vhd | enablingregisters.vhd | RegisterFile.vhd* | decoder.vhd*



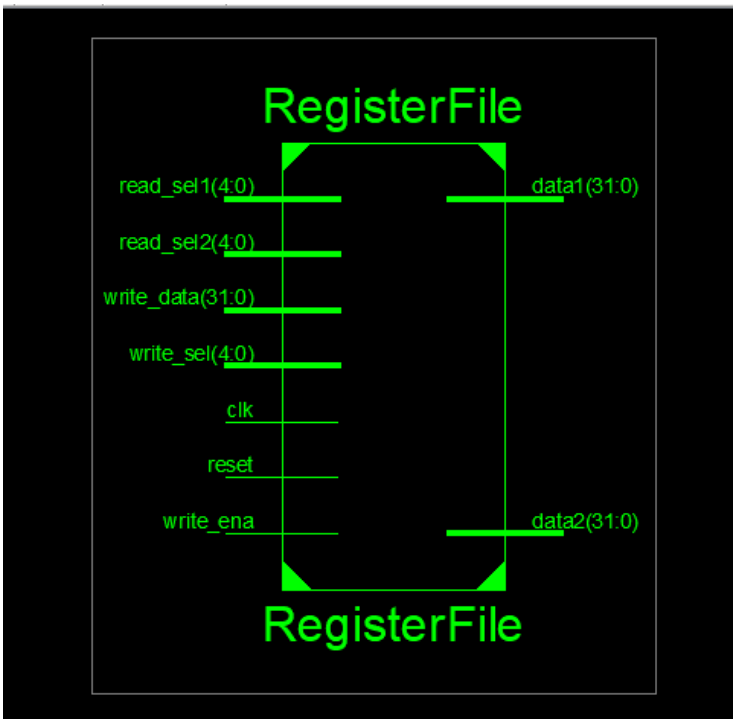
```
51  e6: enablingregisters port map (temp6,D6,write_ena);
52  e7: enablingregisters port map (temp7,D7,write_ena);
53  e8: enablingregisters port map (temp8,D8,write_ena);
54  e9: enablingregisters port map (temp9,D9,write_ena);
55  e10: enablingregisters port map (temp10,D10,write_ena);
56  e11: enablingregisters port map (temp11,D11,write_ena);
57  e12: enablingregisters port map (temp12,D12,write_ena);
58  e13: enablingregisters port map (temp13,D13,write_ena);
59  e14: enablingregisters port map (temp14,D14,write_ena);
60  e15: enablingregisters port map (temp15,D15,write_ena);
61  e16: enablingregisters port map (temp16,D16,write_ena);
62  e17: enablingregisters port map (temp17,D17,write_ena);
63  e18: enablingregisters port map (temp18,D18,write_ena);
64  e19: enablingregisters port map (temp19,D19,write_ena);
65  e20: enablingregisters port map (temp20,D20,write_ena);
66  e21: enablingregisters port map (temp21,D21,write_ena);
67  e22: enablingregisters port map (temp22,D22,write_ena);
68  e23: enablingregisters port map (temp23,D23,write_ena);
69  e24: enablingregisters port map (temp24,D24,write_ena);
70  e25: enablingregisters port map (temp25,D25,write_ena);
71  e26: enablingregisters port map (temp26,D26,write_ena);
72  e27: enablingregisters port map (temp27,D27,write_ena);
73  e28: enablingregisters port map (temp28,D28,write_ena);
74  e29: enablingregisters port map (temp29,D29,write_ena);
75  e30: enablingregisters port map (temp30,D30,write_ena);
76  e31: enablingregisters port map (temp31,D31,write_ena);
77  e32: enablingregisters port map (temp32,D32,write_ena);
78
```

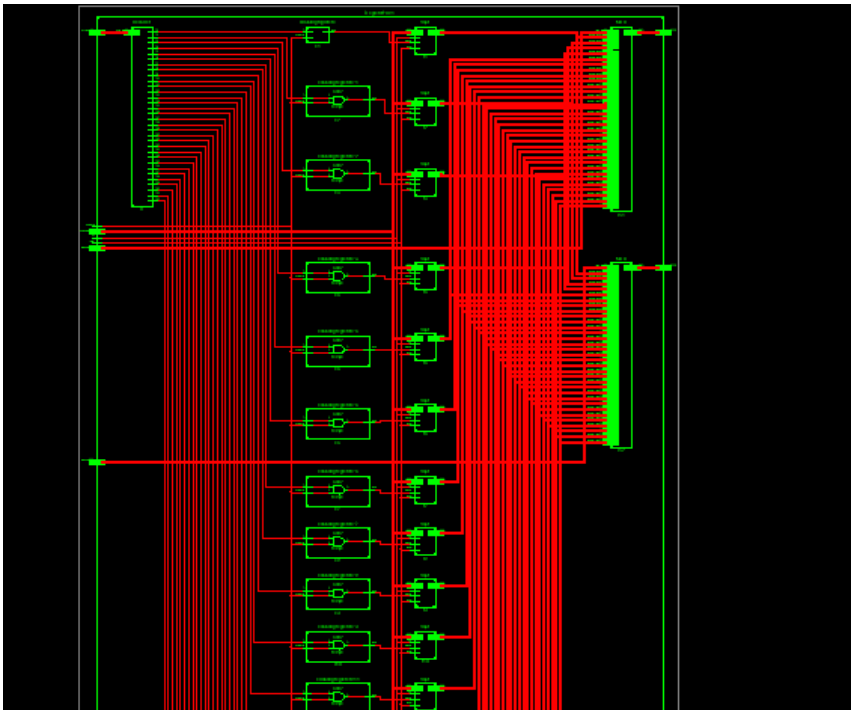


```
80
81          -- declaring registers
82  r1: flopr port map ( reset,clk,temp1,write_data,register_temp1);
83  r2: flopr port map ( reset,clk,temp2,write_data,register_temp2);
84  r3: flopr port map ( reset,clk,temp3,write_data,register_temp3);
85  r4: flopr port map ( reset,clk,temp4,write_data,register_temp4);
86  r5: flopr port map ( reset,clk,temp5,write_data,register_temp5);
87  r6: flopr port map ( reset,clk,temp6,write_data,register_temp6);
88  r7: flopr port map ( reset,clk,temp7,write_data,register_temp7);
89  r8: flopr port map ( reset,clk,temp8,write_data,register_temp8);
90  r9: flopr port map ( reset,clk,temp9,write_data,register_temp9);
91  r10: flopr port map ( reset,clk,temp10,write_data,register_temp10);
92  r11: flopr port map ( reset,clk,temp11,write_data,register_temp11);
93  r12: flopr port map ( reset,clk,temp12,write_data,register_temp12);
94  r13: flopr port map ( reset,clk,temp13,write_data,register_temp13);
95  r14: flopr port map ( reset,clk,temp14,write_data,register_temp14);
96  r15: flopr port map ( reset,clk,temp15,write_data,register_temp15);
97  r16: flopr port map ( reset,clk,temp16,write_data,register_temp16);
98  r17: flopr port map ( reset,clk,temp17,write_data,register_temp17);
99  r18: flopr port map ( reset,clk,temp18,write_data,register_temp18);
100 r19: flopr port map ( reset,clk,temp19,write_data,register_temp19);
101 r20: flopr port map ( reset,clk,temp20,write_data,register_temp20);
102 r21: flopr port map ( reset,clk,temp21,write_data,register_temp21);
103 r22: flopr port map ( reset,clk,temp22,write_data,register_temp22);
104 r23: flopr port map ( reset,clk,temp23,write_data,register_temp23);
105 r24: flopr port map ( reset,clk,temp24,write_data,register_temp24);
106 r25: flopr port map ( reset,clk,temp25,write_data,register_temp25);
107 r26: flopr port map ( reset,clk,temp26,write_data,register_temp26);
```



The RTL Schematic of RegisterFile



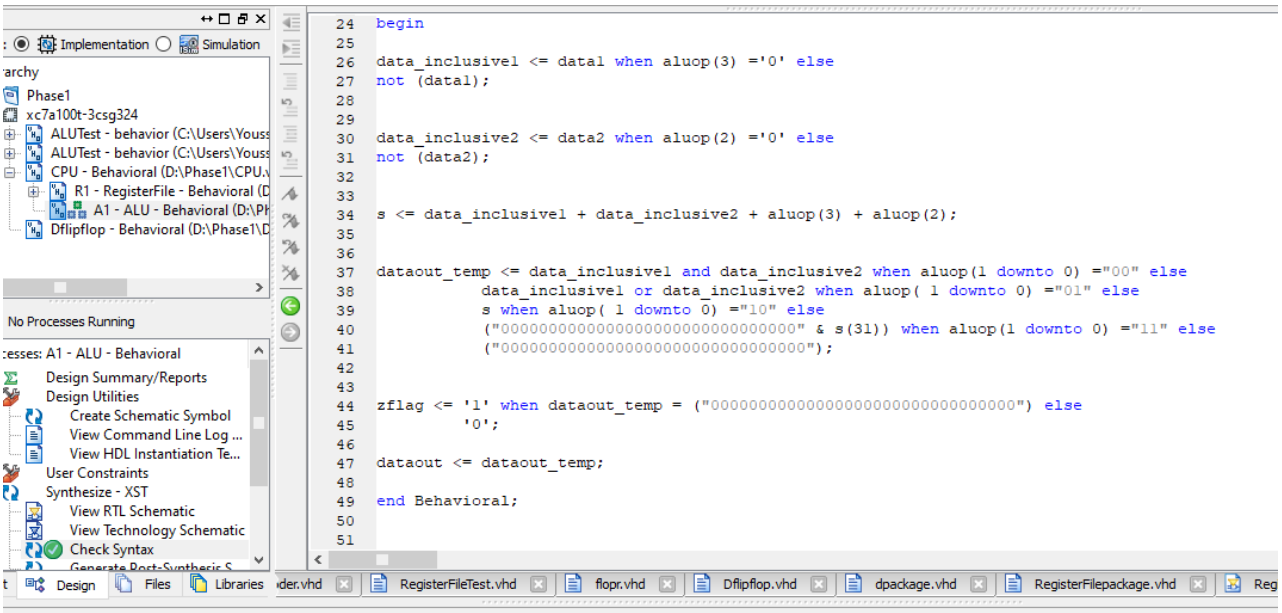


the detailed RegisterFile is too large to be contained in a photo it can be viewed clearly when opening the code on xilinx.

It can appear that it contains a decoder & 2 MUXs & 32 AND gates & 32 Flop register.

2- ALU

```
use IEEE.STD_LOGIC_unsigned.ALL;

entity ALU is
    Port ( data1 : in  STD_LOGIC_VECTOR (31 downto 0);
           data2 : in  STD_LOGIC_VECTOR (31 downto 0);
           aluop : in  STD_LOGIC_VECTOR (3 downto 0);
           dataout : out  STD_LOGIC_VECTOR (31 downto 0);
           zflag : out  STD_LOGIC);
end ALU;

architecture Behavioral of ALU is

signal s: std_LOGIC_vector (31 downto 0);

signal data_inclusive1: std_LOGIC_vector (31 downto 0);
signal data_inclusive2: std_LOGIC_vector (31 downto 0);

signal dataout_temp: std_LOGIC_vector (31 downto 0);

begin

data_inclusive1 <= data1 when aluop(3) ='0' else
not (data1);


data_inclusive2 <= data2 when aluop(2) ='0' else
not (data2);

```




```
begin

data_inclusive1 <= data1 when aluop(3) ='0' else
not (data1);


data_inclusive2 <= data2 when aluop(2) ='0' else
not (data2);


s <= data_inclusive1 + data_inclusive2 + aluop(3) + aluop(2);


dataout_temp <= data_inclusive1 and data_inclusive2 when aluop(1 downto 0) ="00" else
          data_inclusive1 or data_inclusive2 when aluop( 1 downto 0) ="01" else
          s when aluop( 1 downto 0) ="10" else
          ("00000000000000000000000000000000" & s(31)) when aluop(1 downto 0) ="11" else
          ("00000000000000000000000000000000");


zflag <= '1' when dataout_temp = ("00000000000000000000000000000000") else
         '0';

dataout <= dataout_temp;

end Behavioral;

```

We want to make a 32-bit ALU which performs multiple tasks.

• It contains inverters and a multiplexer to invert input B to be not (B) when the F2 control signal is asserted (when statement) and that happens when f (2) is equal to 1.

• It contains inverters and a multiplexer to invert input A to be not (A) when the F2 control signal is asserted (when statement) and that happens when f (3) is equal to 1.

• The arithmetic and logical blocks in the ALU operate on AA and BB. BB is either B or B', depending on F2 && AA is either A or A'.

• If F1:0 = 00, the output multiplexer chooses AA AND BB.

• If F1:0 = 01, the ALU computes AA OR BB.

• If F1:0 = 10, the ALU performs addition or subtraction. Note that F2 is also the carry into the adder. Also remember that B' + 1 = −B in two's complement arithmetic. If F2 = 0, the ALU computes A + B. If F2 = 1, the ALU computes A + B' + 1 = A − B.

- Similarly we can apply the same thing on F3 as it when equals 1, the ALU computes A'+ B + 1 = B − A.

• When F2:0 = 111, the ALU performs the set if less than (SLT) operation. When A < B, Y = 1. Otherwise, Y = 0. In other words, Y is set to 1 if A is less than B.

• Previously s was assigned that s = a + b or s = a - b , when a-b is positive means a greater than b and the most significant bit is '0' so we make the output Y equal 31 bits all 0 and the least significant bit is equal to s(31) which is 0 in this case

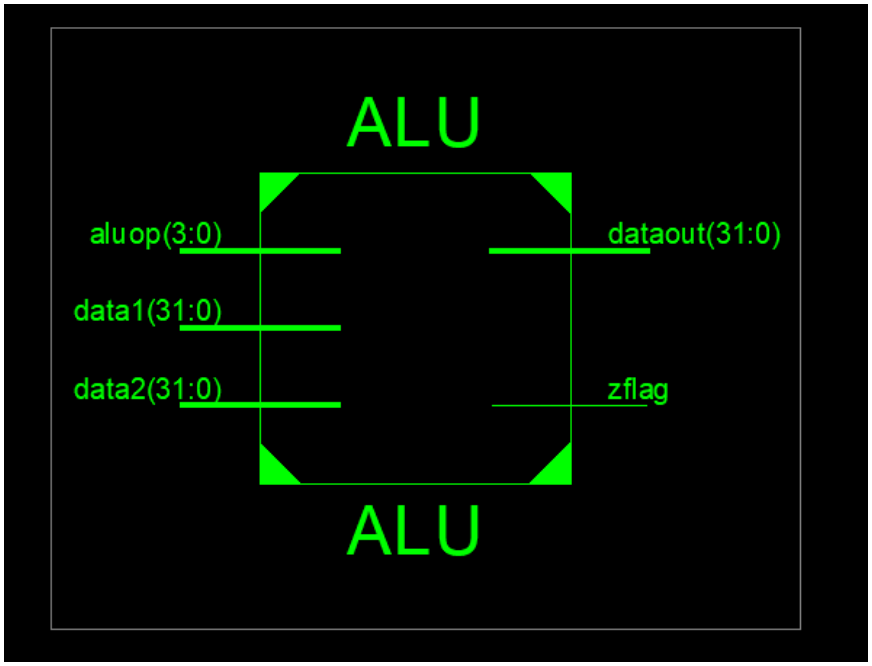
• If a-b is negative means a less than b and the most significant bit is ‘1’ so we make the output Y equal 31 bits all 0 and the least significant bit is equal to s (31) which is 1 in this case.

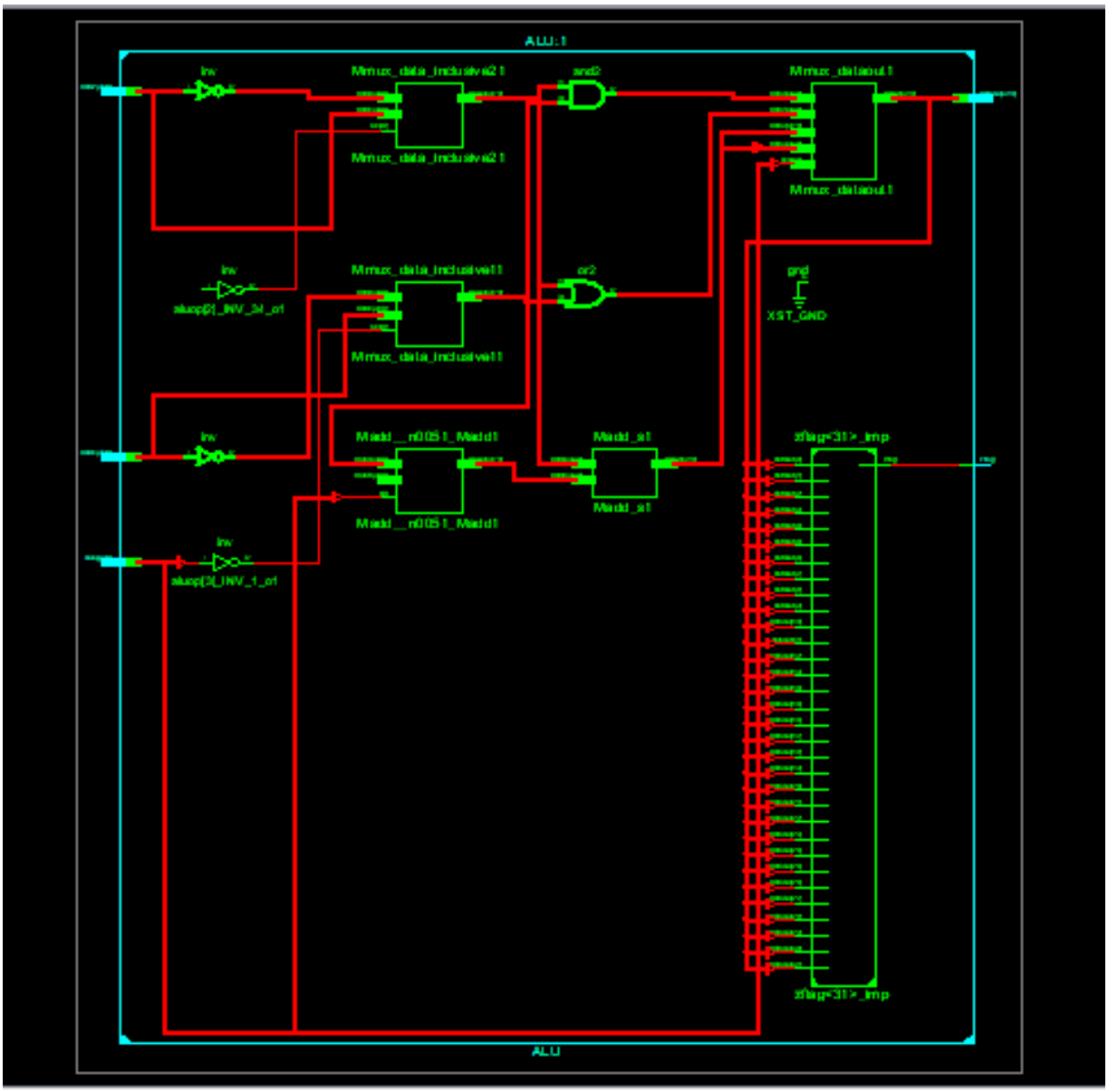
- When F3:0 = 1100, the ALU performs the NOR operation which is equal to A’ AND B’ so it is equivalent to our concept we put in the first place.

• Finally, we have performed the SLT, NOR and all the ALU operations we wanted.

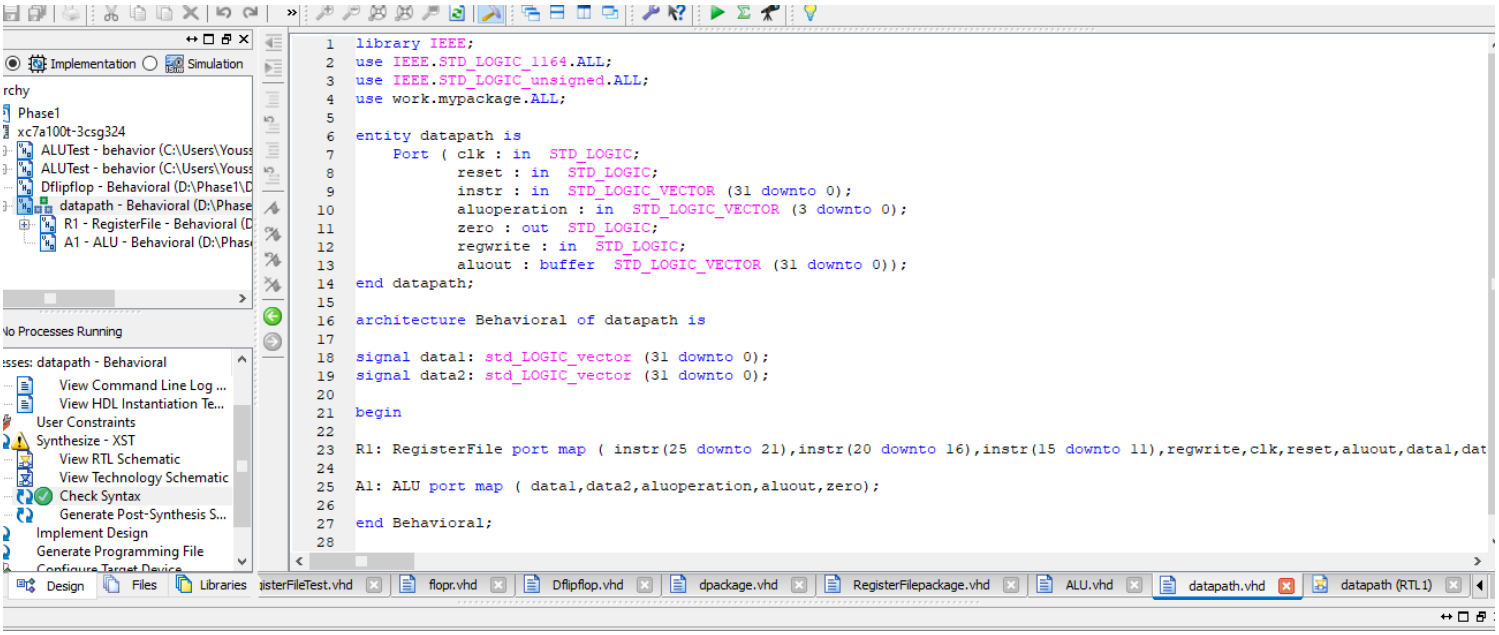
- In our code **data 1 represents A , data 2 represents B , data_inclusive1 represents AA , data_inclusive2 represents BB**

The RTL Schematic of ALU

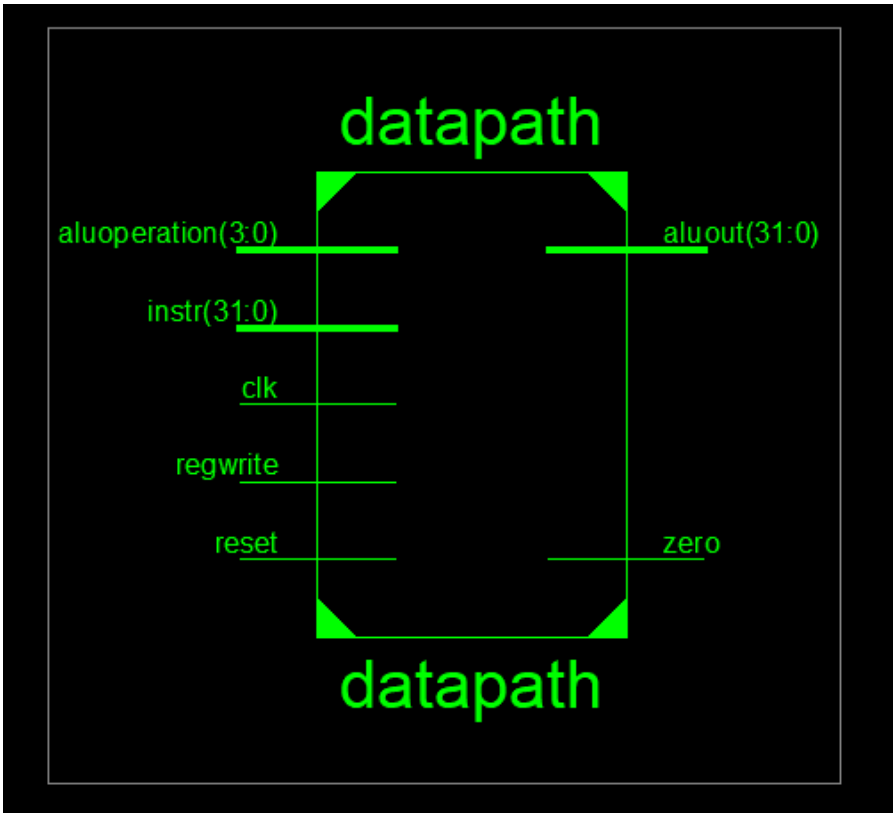


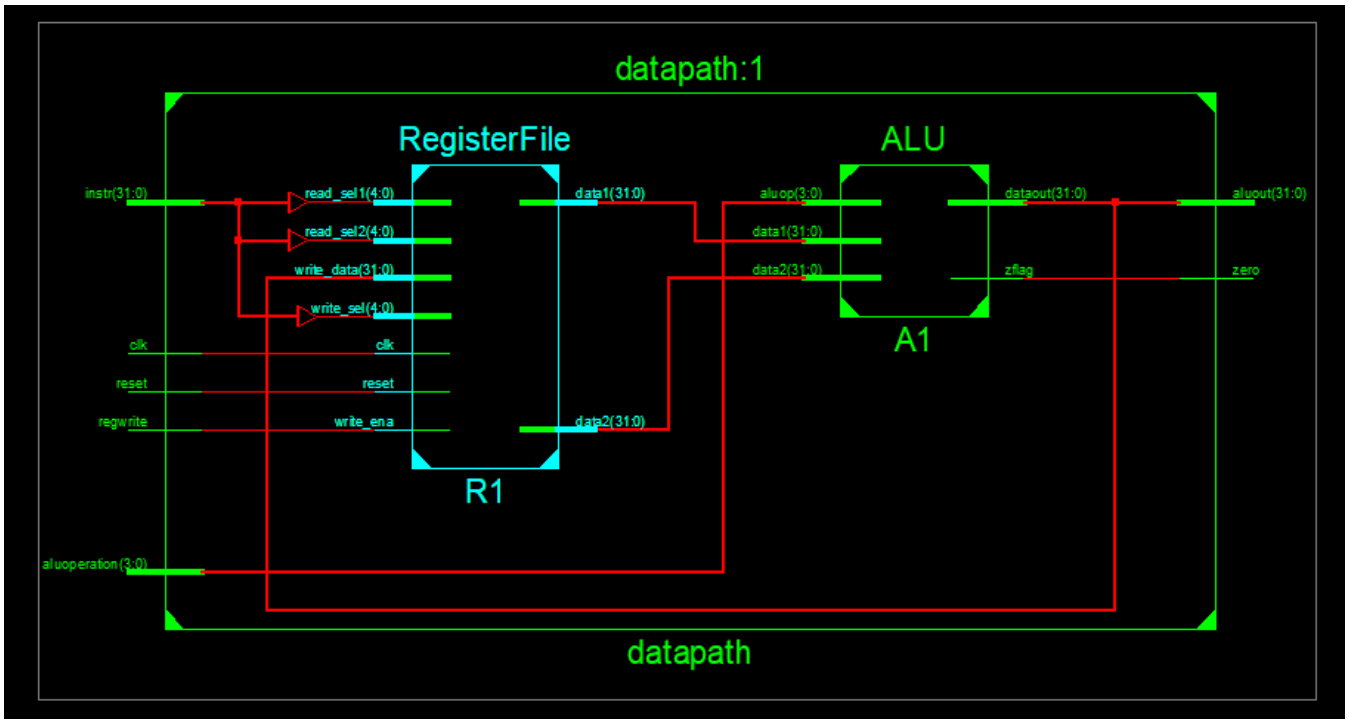

ALU:1
inv
Mmux_data_mdusel21
and2
Mmux_dataout1
Mmux_data_mdusel21
Mmux_dataout1
inv
aluop[1]_INV_24_o1
Mmux_data_mdusel11
or2
gnd
XST_GND
Mmux_data_mdusel11
inv
Maddsub_n0051_Maddsub1
Maddsub_s1
xflags<31>_imp
Maddsub_n0051_Maddsub1
Maddsub_s1
inv
aluop[3]_INV_1_o1
xflags<31>_imp
ALU

3- Data path (phase 1)



The RTL Schematic of ALU




datapath:1
RegisterFile
ALU
instr(31:0)
read_sel1(4:0)
read_sel2(4:0)
write_data(31:0)
write_sel(4:0)
clk
reset
regwrite
clk
reset
write_ena
data1(31:0)
data2(31:0)
R1
aluop(3:0)
data1(31:0)
data2(31:0)
dataout(31:0)
zflag
A1
aluout(31:0)
zero
aluoperation(3:0)
datapath

4- data path (phase 2)

First before we start connecting in the datapath, we will need shifter, adder, mux 2x1, sign extender, flop register(we previously used before in registerFile), ALU, RegisterFile.

I- left shifter

```
1  ----------------------------------------------------------------------------
2  library IEEE;
3  use IEEE.STD_LOGIC_1164.ALL;
4
5  entity sl2 is
6      Port ( a : in  STD_LOGIC_VECTOR (31 downto 0);
7             b : out  STD_LOGIC_VECTOR (31 downto 0));
8  end sl2;
9
10 architecture Behavioral of sl2 is
11
12 begin
13
14 b <= a(29 downto 0) & "00";
15
16 end Behavioral;
17
18
```

II - adder

```
1  ----------------------------------------------------------------
2  library IEEE;
3  use IEEE.STD_LOGIC_1164.ALL;
4  use ieee.std_logic_unsigned.all;
5
6  entity adder is
7      Port ( a : in  STD_LOGIC_VECTOR (31 downto 0);
8             b : in  STD_LOGIC_VECTOR (31 downto 0);
9             s : out  STD_LOGIC_VECTOR (31 downto 0));
10 end adder;
11
12 architecture Behavioral of adder is
13
14 begin
15
16 s <= a + b;
17
18 end Behavioral;
19
```

III- mux 2x1

```
1
2  ----------------------------------------------------------------------------
3  library IEEE;
4  use IEEE.STD_LOGIC_1164.ALL;
5
6
7  entity mux2x1 is
8  Generic (n: integer );
9
10     Port ( s : in  STD_LOGIC;
11            I0 : in  STD_LOGIC_VECTOR (n-1 downto 0);
12            I1 : in  STD_LOGIC_VECTOR (n-1 downto 0);
13            O : out  STD_LOGIC_VECTOR (n-1 downto 0));
14 end mux2x1;
15
16 architecture Behavioral of mux2x1 is
17
18 begin
19
20 O <=  I1 WHEN S = '1' ELSE
21       I0;
22
23 end Behavioral;
24
25
```

IV- Sign extender

```
1
2  ----------------------------------------------------------------------
3  library IEEE;
4  use IEEE.STD_LOGIC_1164.ALL;
5
6  entity signext is
7      Port ( a : in  STD_LOGIC_VECTOR (15 downto 0);
8             b : out  STD_LOGIC_VECTOR (31 downto 0));
9  end signext;
10
11 architecture Behavioral of signext is
12
13 begin
14
15 b <= X"ffff" & a  WHEN a(15) = '1' ELSE
16      X"0000" & a;
17
18
19 end Behavioral;
20
21
```

v- datapath main

```
3  use IEEE.STD_LOGIC_1164.ALL;
4  use work.datapathpackage.ALL;
5
6  entity datapath is
7      port( clk, reset: in STD_LOGIC;
8            readdata: in STD_LOGIC_VECTOR(31 downto 0);
9            instr: in STD_LOGIC_VECTOR(31 downto 0);
10           memtoreg, pcsrc, alusrc, regwrite, regdst, jump: in STD_LOGIC;
11           aluoperation: in STD_LOGIC_VECTOR(3 downto 0);
12           zero: out STD_LOGIC;
13           pc: inout STD_LOGIC_VECTOR(31 downto 0);
14           aluout, writedata: inout STD_LOGIC_VECTOR(31 downto 0));
15 end datapath;
16
17 architecture Behavioral of datapath is
18
19 ---------- next pc
20 signal pcjump, pcnext, pcnextbr, pcplus4,  pcbranch: STD_LOGIC_VECTOR(31 downto 0);
21 signal signimm, signimmsh: STD_LOGIC_VECTOR(31 downto 0);
22 signal srca, srcb, result: STD_LOGIC_VECTOR(31 downto 0);
23
24 ---------- registerfile
25 signal writereg: STD_LOGIC_VECTOR(4 downto 0);
26 signal data1: STD_LOGIC_VECTOR(31 downto 0);
27 signal data2: STD_LOGIC_VECTOR(31 downto 0);
28 signal data2out: STD_LOGIC_VECTOR(31 downto 0);
29 signal writedatainregisterfile: STD_LOGIC_VECTOR(31 downto 0);
30
```

datapath.vhd | mips.vhd | main.vhd | imem.vhd | maindecoder (RTL1) | Controller (RTL1) | adde

```
30
31 begin
32
33 ---------- next pc
34 pcjump <= pcplus4(31 downto 28) & instr(25 downto 0) & "00";
35 pcreg: flopr port map( reset, clk, '1', pcnext, pc);
36 pcadd1: adder port map(pc, X"00000004", pcplus4);
37 immsh: sl2 port map(signimm, signimmsh);
38 pcadd2: adder port map(pcplus4, signimmsh, pcbranch);
39 pcbrmux: mux2x1 generic map(32) port map(pcsrc, pcplus4, pcbranch, pcnextbr);
40 pcmux: mux2x1 generic map(32) port map(jump, pcnextbr, pcjump, pcnext);
41
42
43 ---------- register file
44 reading: mux2x1 generic map(32) port map( memtoreg, aluout, readdata, writedatainregisterfile );
45 writeregister: mux2x1 generic map(5) port map( regdst, instr(20 downto 16), instr(15 downto 11), writereg );
46 regfile: RegisterFile port map( instr(25 downto 21), instr(20 downto 16), writereg, regwrite, clk, reset, writedatainregist
47 extending: signext port map( instr(15 downto 0),signimm );
48 data2select: mux2x1 generic map(32) port map( alusrc, data2, signimm, data2out );
49
50 ---------- ALU
51 alu1: ALU port map( data1, data2out, aluoperation, aluout, zero );
52 writedata <= data2;
53
54 end Behavioral;
55
56
```

datapath.vhd | mips.vhd | main.vhd | imem.vhd | maindecoder (RTL1) | Controller (RTL1) | adder.vhd | sl2.vhd | signe

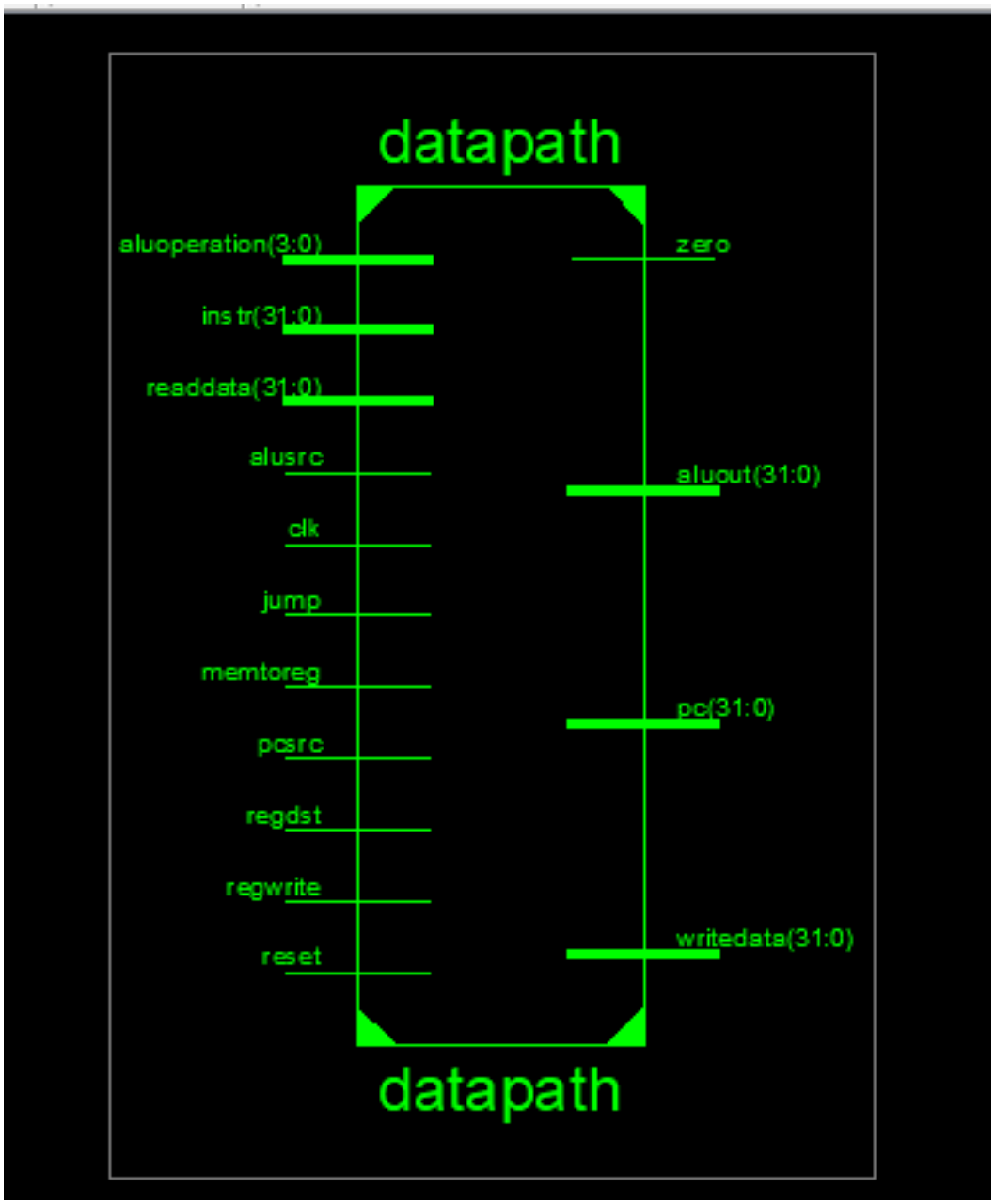
Now after finishing every component we need, we made the datapath main module where we connected every thing according to the datapath we previously explained but without the instruction memory, data memory, controls.

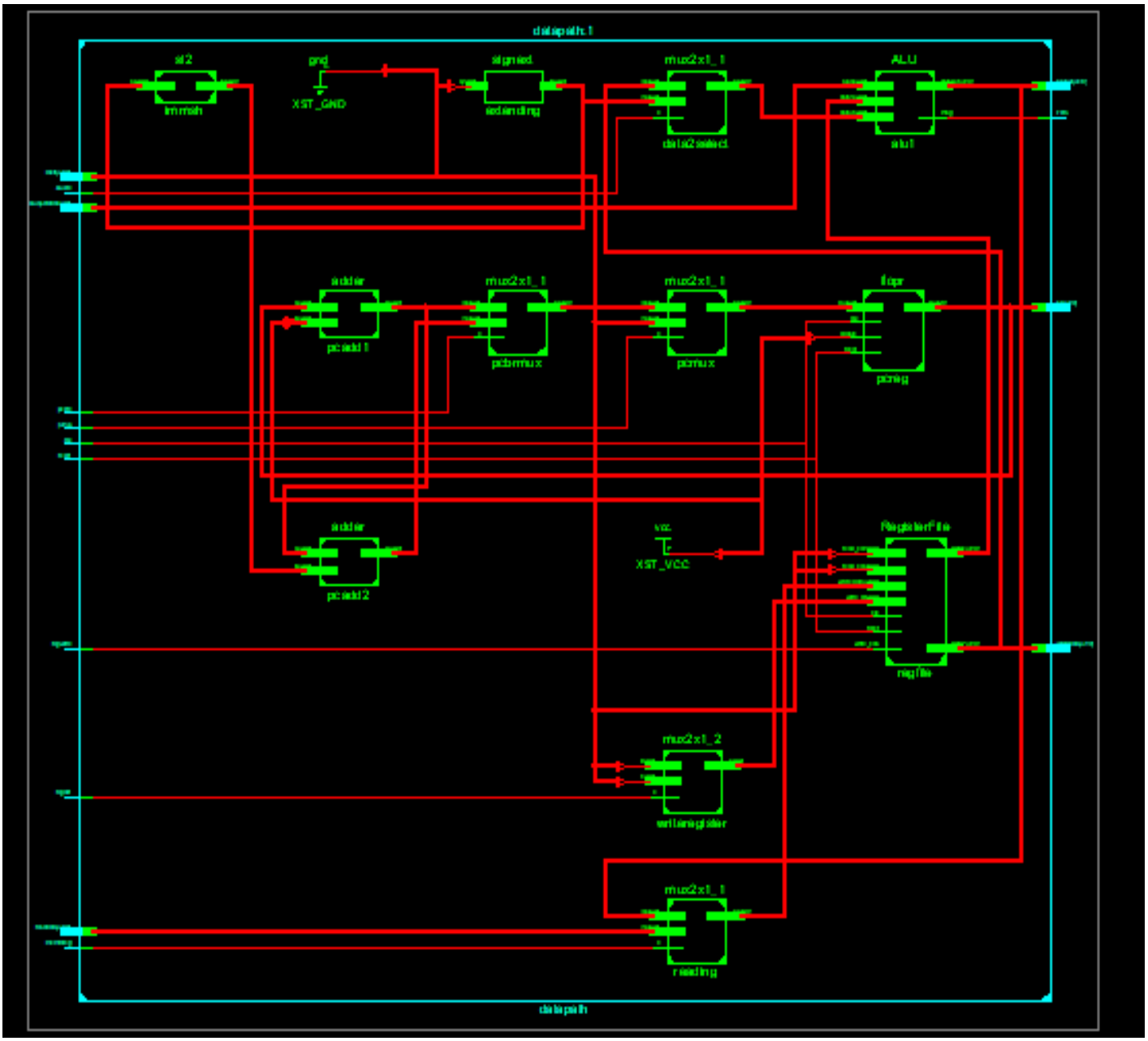
First next PC logic with the jump instruction, we want to make PC which is a flopr that has the next pc as input & the current PC as output that as every instruction the PC is updated. we must add PC with 4 as that happens every instruction. Then the output of sign extender that indicates that a branch instruction occurred is shifted left by 2 then added with the pc +4 and then passed to mux to choose whether to choose which situation occurred.
The jump is made by the taken the instructions (25-0) & the first 2 bits are zeros and the last 4 bits of PC.

Second the Register File logic, we have 2 options to read from memory so we will use mux to choose between ALU output (if it is R-type instruction) or the data from memory (if it is load instruction) . We need another Mux to choose whether to write the instruction(15-11) (if it is R-type instruction) or the instruction(20-16) (if it is I-type instruction). Then the output of the Register file will be data 1 mandatory & data 2 or output of sign extender (if it is I-type instruction).

Third ALU logic, It take the input from data 1 & the output of last mux.

RTL schematic of datapath




datapath:1
sl2
imm8h
gnd
XST_GND
signed
sxtsmallreg
mux2x1_1
data2select
ALU
alu1
adder
pcadd1
mux2x1_1
pcbrmux
mux2x1_1
pcmux
flopr
pcreg
adder
pcadd2
vcc
XST_VCC
RegisterFile
regfile
mux2x1_2
writeregister
mux2x1_1
resultreg
datapath

5- Controller

it contains Main decoder & ALU decoder, they have the selection of each MUX.

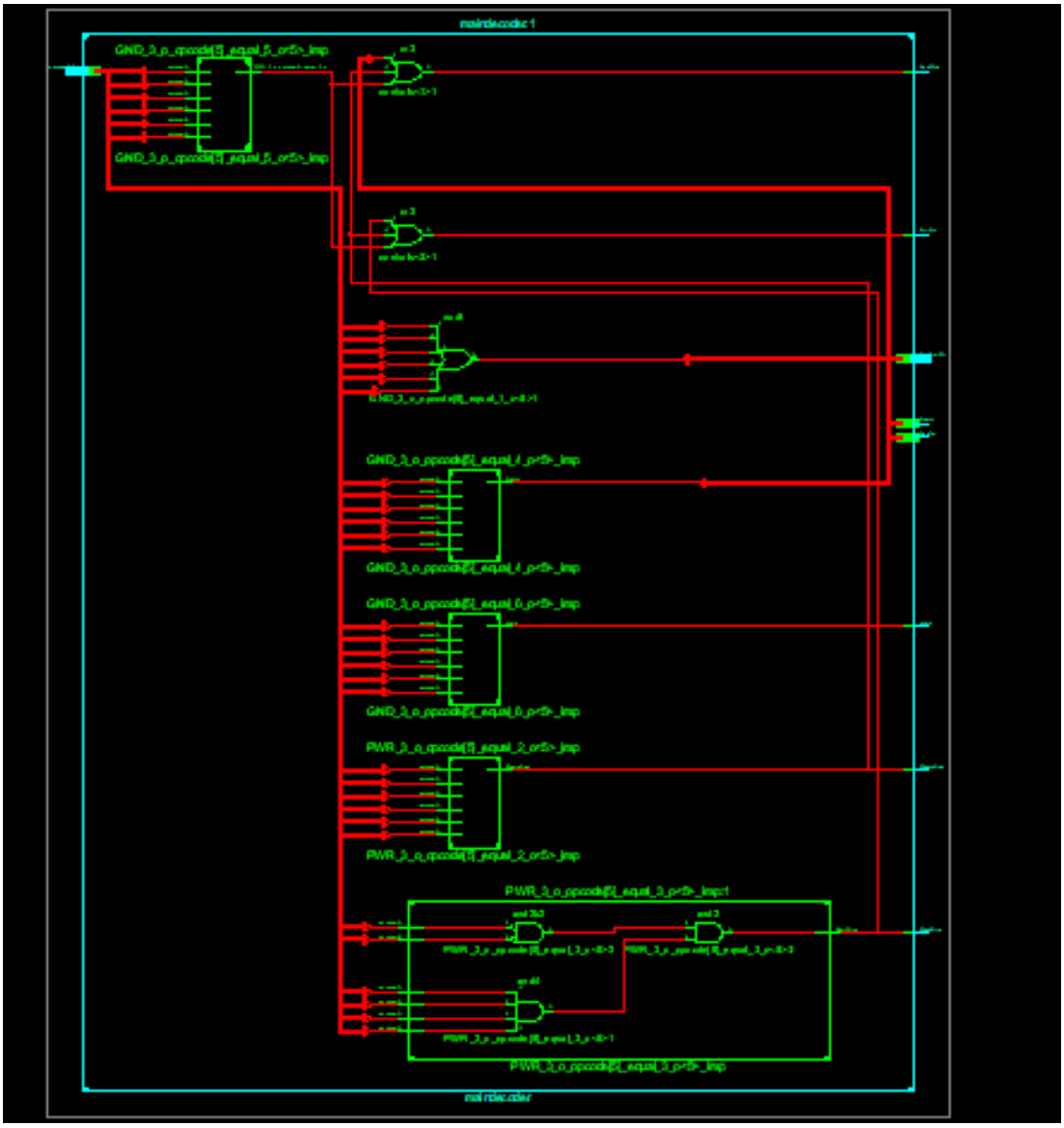
I- Main decoder

```
11          RegWrite : out  STD_LOGIC;
12          Jump : out  STD_LOGIC;
13          ALUOp : out  STD_LOGIC_VECTOR (1 downto 0));
14  end maindecoder;
15
16  architecture Behavioral of maindecoder is
17
18  signal controls : STD_LOGIC_VECTOR ( 8 downto 0 );
19
20  begin
21
22  process ( opcode )
23  begin
24  case opcode is
25                  when "000000" => controls <= "110000010";    ---- R-type instr
26                  when "100011" => controls <= "101001000";    ---- lw
27                  when "101011" => controls <= "001010000";    ---- sw
28                  when "000100" => controls <= "000100001";    ---- beq
29                  when "001000" => controls <= "101000000";    ---- addi
30                  when "000010" => controls <= "000000100";    ---- j
31                  when others => controls <= "---------";
32
33  end case;
34  end process;
35
36  (RegWrite, RegDst, ALUSrc, Branch, MemWrite, MemtoReg, Jump, ALUOp(1), ALUOp(0)) <= controls;
37
38  end Behavioral;
```

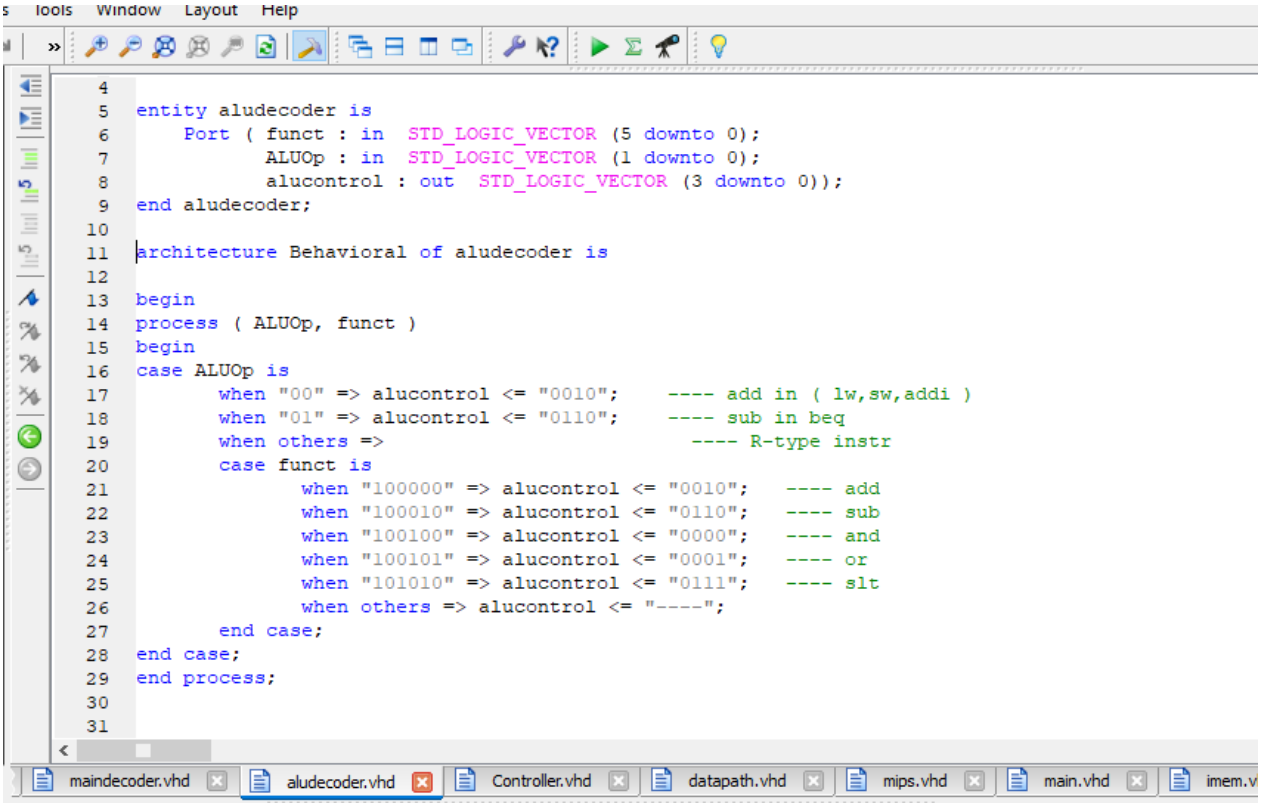
maindecoder.vhd | aludecoder.vhd | Controller.vhd | datapath.vhd | mips.vhd | main.vhd | imem.vhd

we have different cases that shows which selector have 1 or 0 according to the opcode from each instruction as illustrated in the table previously which are the last 6 bits.

RTL schematic of Main decoder



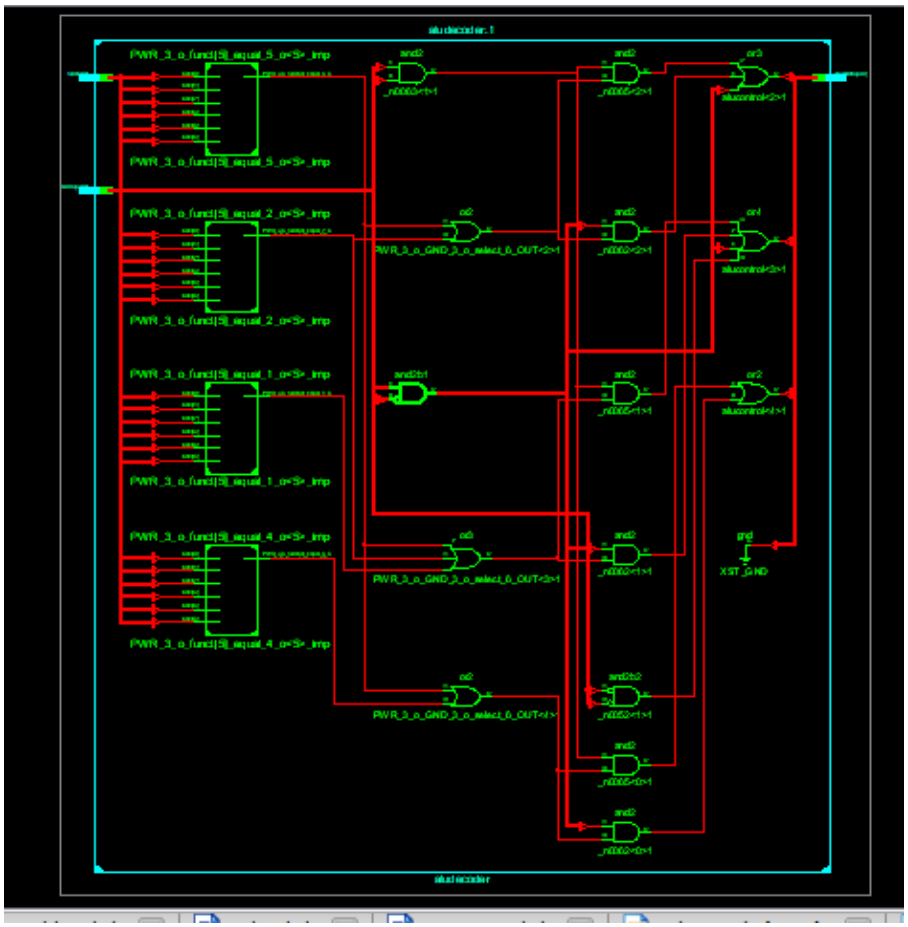
II- ALU decoder



If it is R-Type is takes the first 6 bits to check the function otherwise if it is lw,sw,addi we will need also add.

If it is beq we will need sub.

RTL schematic of ALU decoder



III- Controller

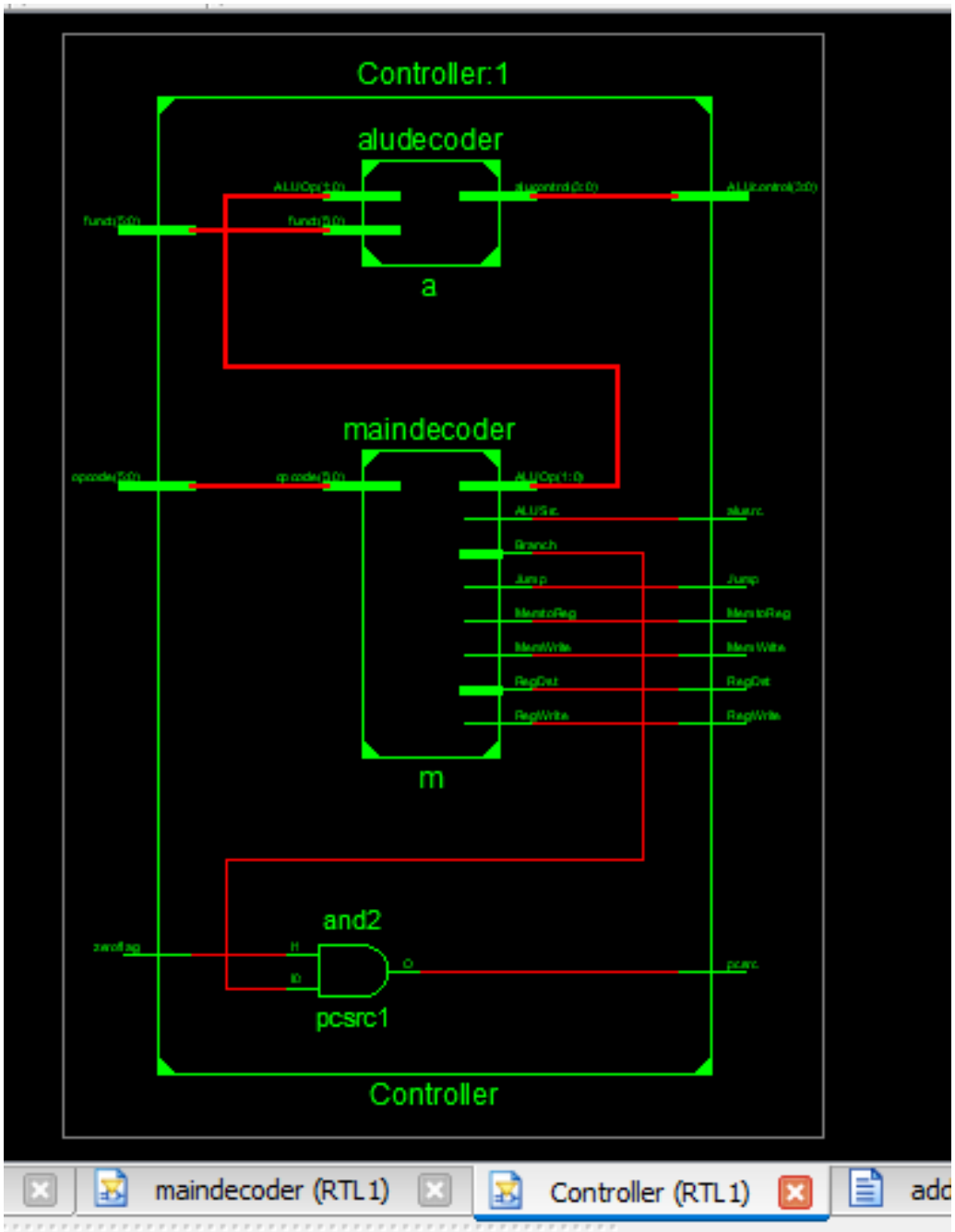
```
 6  entity Controller is
 7      Port ( opcode : in  STD_LOGIC_VECTOR (5 downto 0);
 8             funct : in  STD_LOGIC_VECTOR (5 downto 0);
 9             zeroflag : in  STD_LOGIC;
10             MemtoReg : out  STD_LOGIC;
11             MemWrite : out  STD_LOGIC;
12             pcsrc : out  STD_LOGIC;
13             alusrc : out  STD_LOGIC;
14             RegDst : out  STD_LOGIC;
15             RegWrite : out  STD_LOGIC;
16             Jump : out  STD_LOGIC;
17             ALUcontrol : out  STD_LOGIC_VECTOR (3 downto 0));
18  end Controller;
19
20  architecture Behavioral of Controller is
21
22  signal ALUOp : STD_LOGIC_VECTOR (1 downto 0);
23  signal Branch: STD_LOGIC;
24
25  begin
26
27  m: maindecoder port map ( opcode, MemtoReg, MemWrite, Branch, alusrc, RegDst, RegWrite, Jump, ALUOp );
28  a: aludecoder port map ( funct, ALUOp, ALUcontrol );
29
30  pcsrc <= Branch AND zeroflag;
31
32
33  end Behavioral;
```

mux2x1.vhd | maindecoder.vhd | aludecoder.vhd | Controller.vhd | datapath.vhd | mips.vhd | main.vhd

we will connect the main decoder with ALU decoder.

we must pass the ALUop to the ALU decoder from the main decoder to check which operation needed in each instruction.

RTL Schematic of Controller



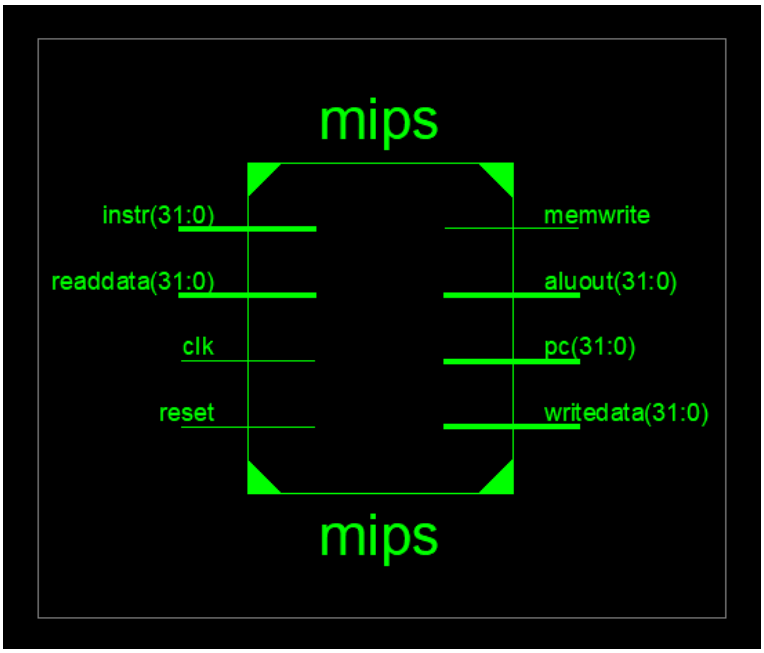
6- MIPS

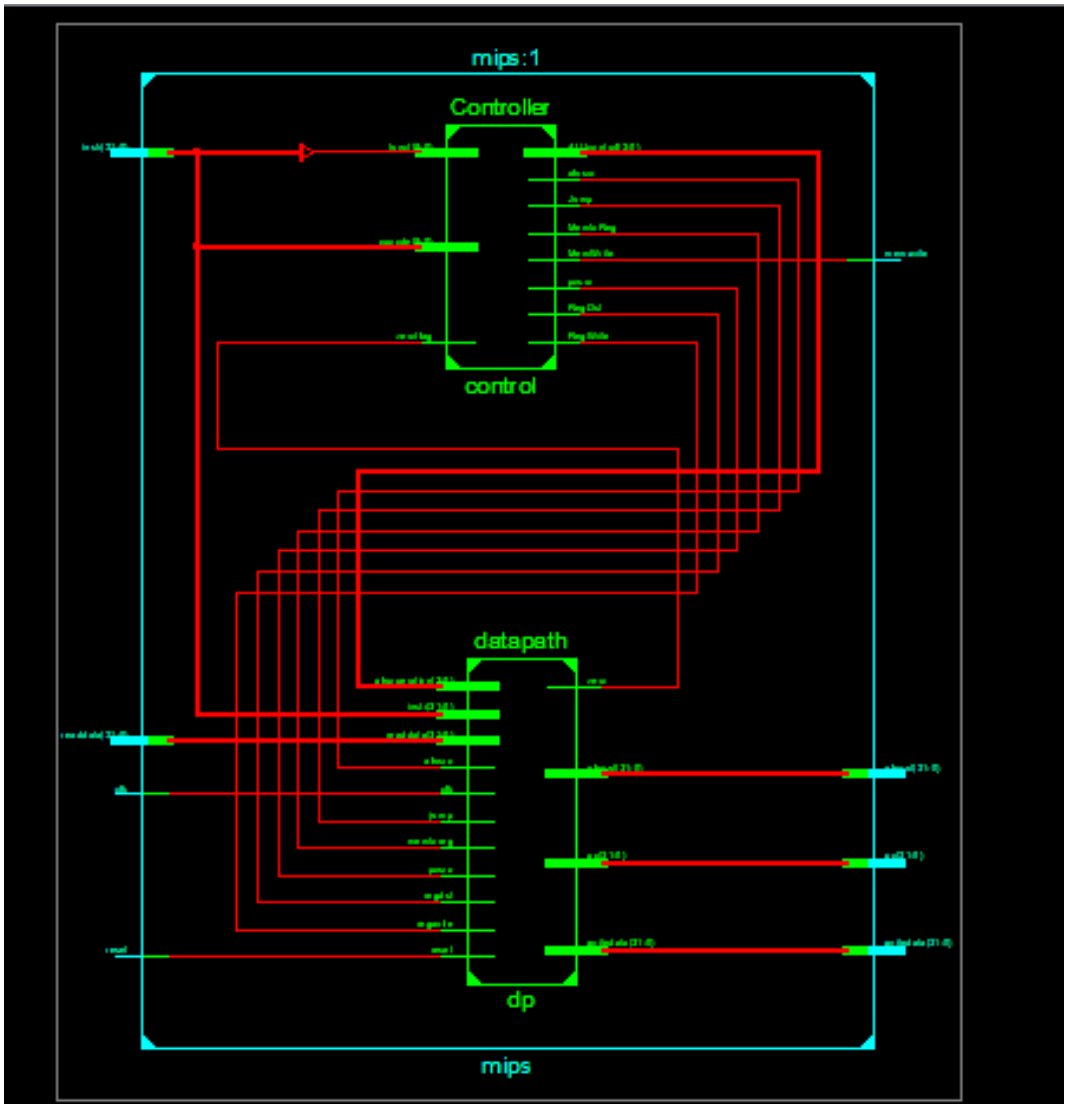
Here we will connect the controller with datapath

```
3  use IEEE.STD_LOGIC_1164.ALL;
4  use work.mipsprocessor.ALL;
5
6
7  entity mips is
8      Port ( clk : in  STD_LOGIC;
9             reset : in  STD_LOGIC;
10            pc : inout  STD_LOGIC_VECTOR (31 downto 0);
11            instr : in  STD_LOGIC_VECTOR (31 downto 0);
12            memwrite : out  STD_LOGIC;
13            aluout : inout  STD_LOGIC_VECTOR (31 downto 0);
14            writedata : inout  STD_LOGIC_VECTOR (31 downto 0);
15            readdata : in  STD_LOGIC_VECTOR (31 downto 0));
16 end mips;
17
18 architecture Behavioral of mips is
19
20 signal memtoreg, alusrc, regdst, regwrite, jump, pcsrc : STD_LOGIC;
21 signal zero : STD_LOGIC;
22 signal alucontrol : STD_LOGIC_VECTOR (3 downto 0);
23
24 begin
25
26 control: Controller port map ( instr(31 downto 26), instr(5 downto 0), zero, memtoreg, memwrite, pcsrc, alusrc, regdst, reg
27
28 dp: datapath port map ( clk, reset, readdata, instr, memtoreg, pcsrc, alusrc, regwrite, regdst, jump, alucontrol, zero, pc,
29
30 end Behavioral;
```

ath.vhd | mips.vhd | main.vhd | imem.vhd | maindecoder (RTL1) | Controller (RTL1) | adder.vhd | sl2.vhd | signext.vhd

RTL Schematic of MIPS




mips:1
Controller
control
datapath
dp
mips

7- Instruction memory

: - [imem.vhd]

Tools Window Layout Help

```
 7  use IEEE.std_logic_textio.all;
 8  library STD;
 9  use STD.textio.all;
10  entity imem is -- instruction memory
11  port(a: in STD_LOGIC_VECTOR(5 downto 0);
12  rd: out STD_LOGIC_VECTOR(31 downto 0));
13  end;
14
15  architecture behave of imem is
16  begin
17  process is
18  file mem_file: TEXT;
19  variable L: line;
20  variable ch: character;
21  variable i, index, result: integer;
22  type ramtype is array (63 downto 0) of STD_LOGIC_VECTOR(31 downto 0);
23  variable mem: ramtype;
24  begin
25  -- initialize memory from file
26  for i in 0 to 63 loop -- set all contents low
27  mem(i) := (others => '0');
28  end loop;
29  index := 0;
30  FILE_OPEN (mem_file, "D:\phase2\memfile.dat", READ_MODE);
31  while not endfile(mem_file) loop
32  readline(mem_file, L);
33  result := 0;
34  for i in 1 to 8 loop
```

imem.vhd | maindecoder (RTL1) | Controller (RTL1) | adder.vhd | sl2.vhd | signext.vhd

```
28  end loop;
29  index := 0;
30  FILE_OPEN (mem_file, "D:\phase2\memfile.dat", READ_MODE);
31  while not endfile(mem_file) loop
32  readline(mem_file, L);
33  result := 0;
34  for i in 1 to 8 loop
35  read (L, ch);
36  if '0' <= ch and ch <= '9' then
37  result := character'pos(ch) - character'pos('0');
38  elsif 'a' <= ch and ch <= 'f' then
39  result := character'pos(ch) - character'pos('a')+10;
40  else report "Format error on line" & integer'
41  image(index) severity error;
42  end if;
43  mem(index)(35-i*4 downto 32-i*4):= std_logic_vector(to_unsigned(result,4));
44  end loop;
45  index := index + 1;
46  end loop;
47  -- read memory
48  for i in 1 to 1000 loop
49  rd <= mem(CONV_INTEGER(a));
50  wait on a;
51  end loop;
52  end process;
53
54  end;
55
```

8- data memory

e - [dmem.vhd]

Tools Window Layout Help

```
3  use IEEE.STD_LOGIC_SIGNED.all;
4  use IEEE.STD_LOGIC_ARITH.all;
5  use ieee.numeric_std.all;
6
7  entity dmem is -- data memory
8  port(clk, we: in STD_LOGIC;
9  a, wd: in STD_LOGIC_VECTOR (31 downto 0);
10 rd: out STD_LOGIC_VECTOR (31 downto 0));
11 end;
12 architecture behave of dmem is
13 begin
14 process  is
15 type ramtype is array (63 downto 0) of STD_LOGIC_VECTOR(31 downto 0);
16 variable mem: ramtype;
17 begin
18 -- read or write memory
19 for i in 1 to 1000 loop
20 if rising_edge(clk) then
21 if (we='1') then
22 mem (CONV_INTEGER('0'&a(7 downto 2))):= wd;
23 end if;
24 end if;
25 rd <= mem (CONV_INTEGER('0'&a (7 downto 2)));
26 wait on clk, a;
27 end loop;
28 end process;
29 end;
30
```

9- main module

we connect the mips with imem, dmem.

```
6  entity main is
7      Port ( clk: in STD_LOGIC;
8             reset: in STD_LOGIC;
9             Writedata,dataadr: out STD_LOGIC_VECTOR(31 downto 0);
10            memwrite: out STD_LOGIC);
11 end main;
12
13 architecture Behavioral of main is
14
15 signal memwrites : STD_LOGIC;
16 signal pc, pcout, instr, readdata, dataadrs, writedatas : STD_LOGIC_VECTOR (31 downto 0);
17
18
19 begin
20
21 pcout <= "00" & pc(31 downto 2);
22
23 iml: imem port map( pcout(5 downto 0), instr );
24
25 mipsl: mips port map ( clk, reset, pc, instr, memwrites, dataadrs, writedatas, readdata );
26
27 dml: dmem port map ( clk, memwrites, dataadrs, writedatas, readdata);
28
29 Writedata <= writedatas;
30 dataadr <= dataadrs;
31 memwrite <= memwrites;
32
33 end Behavioral;
```

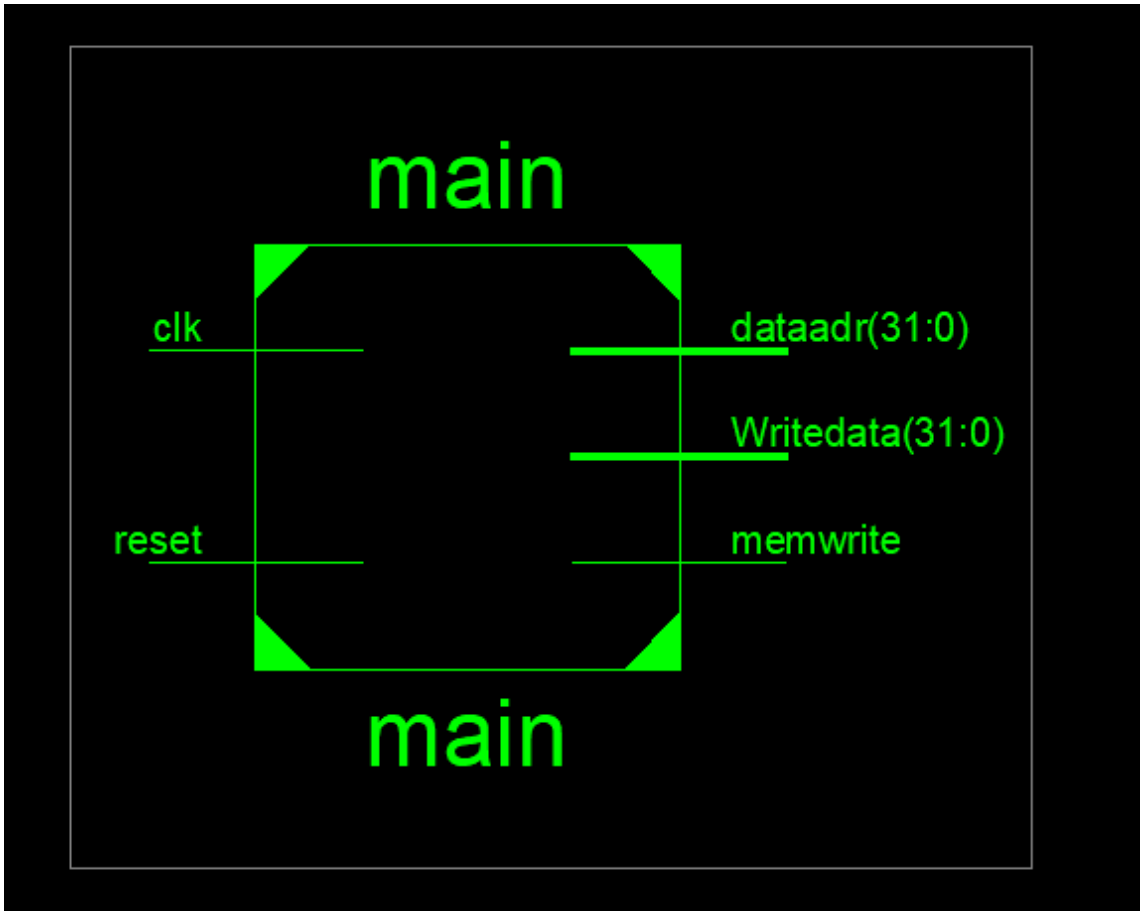
Controller.vhd | datapath.vhd | mips.vhd | main.vhd | imem.vhd | maindecoder (RTL1) | Controller (RTL

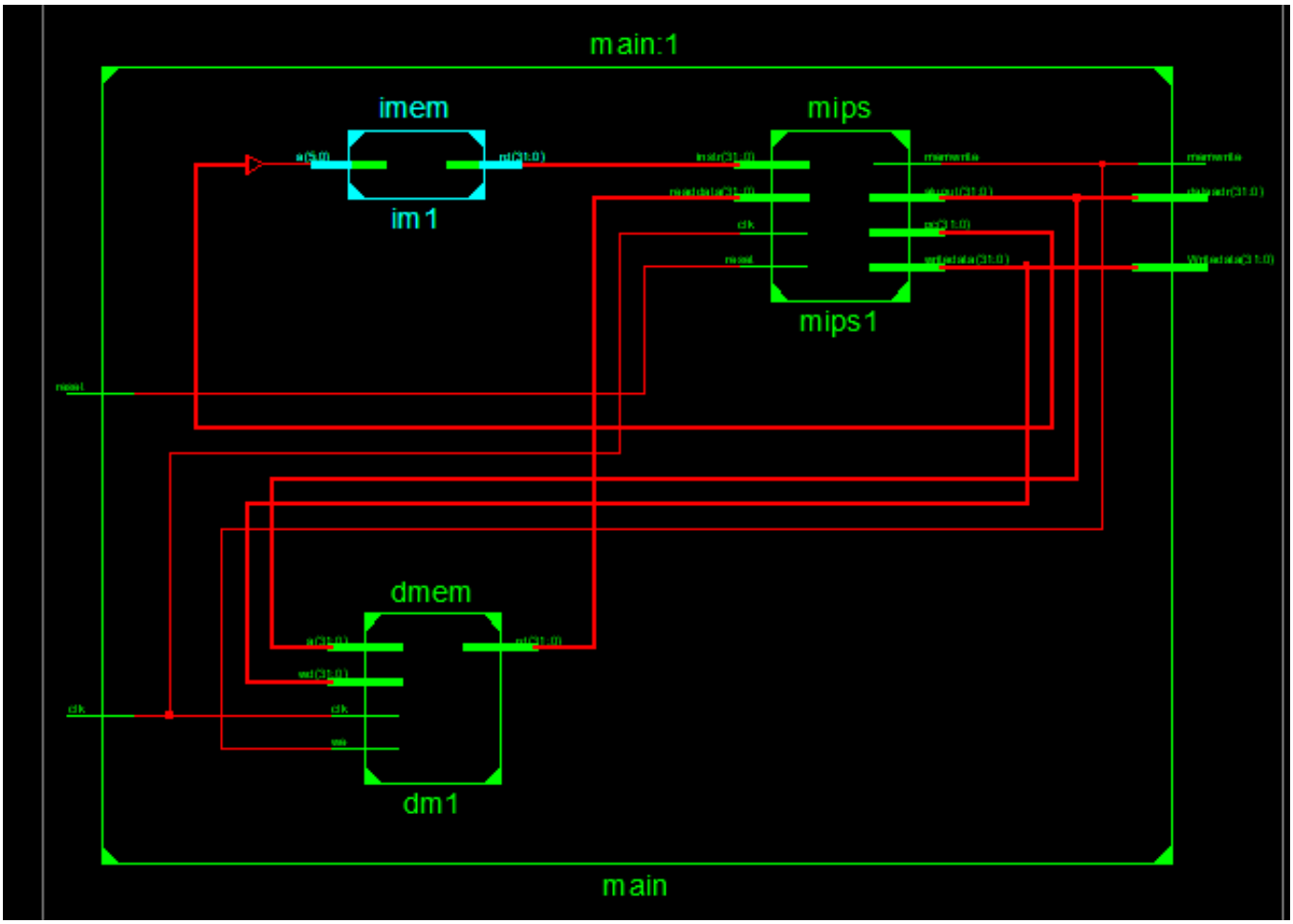
we shift the pc by 2 as we are still in the current pc, then we take the first 6 bits as indicator to be an input in the read address of the instruction memory.

Then we take the instruction from imem to the mips.

Finally we take the memwrite from the mips to the data memory, ALU out & read data 2 from the Register File as in case of store instruction.

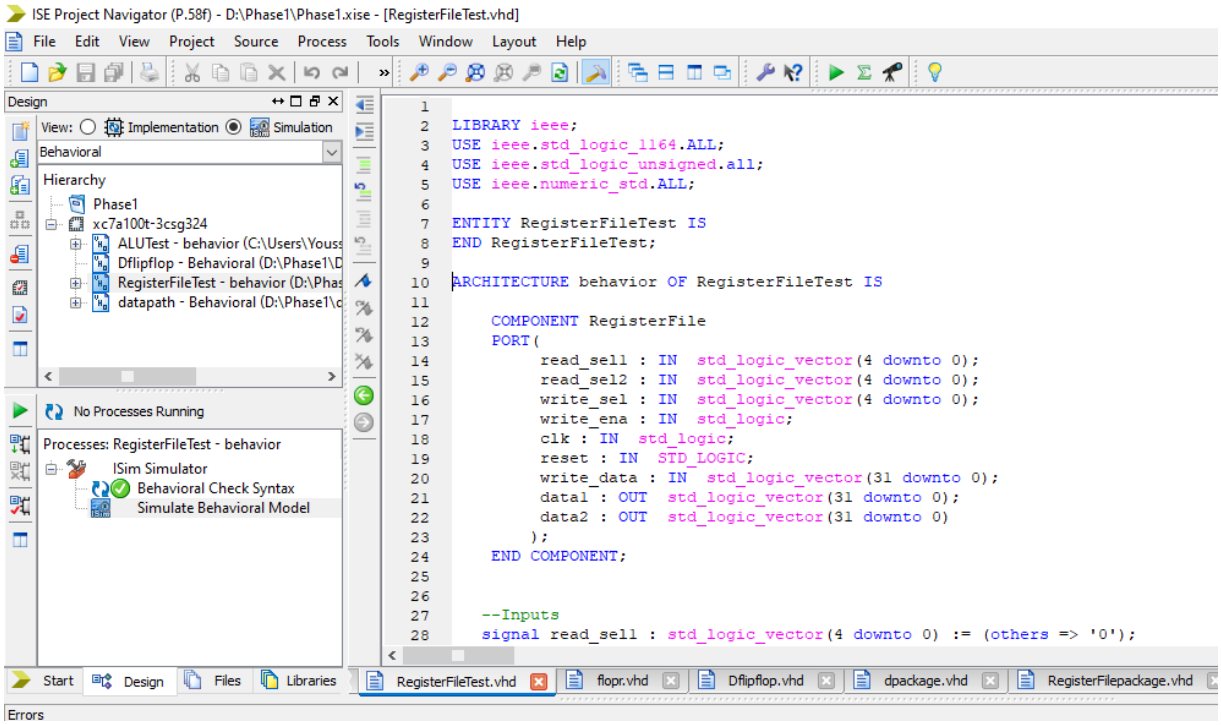
RTL Schematic of main module

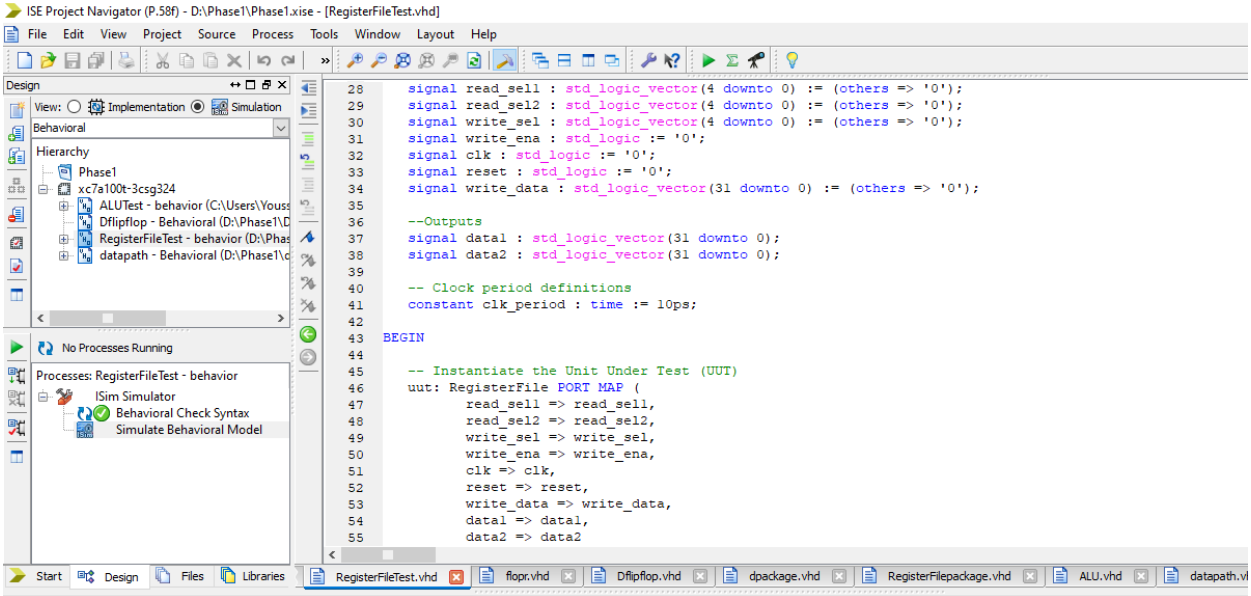


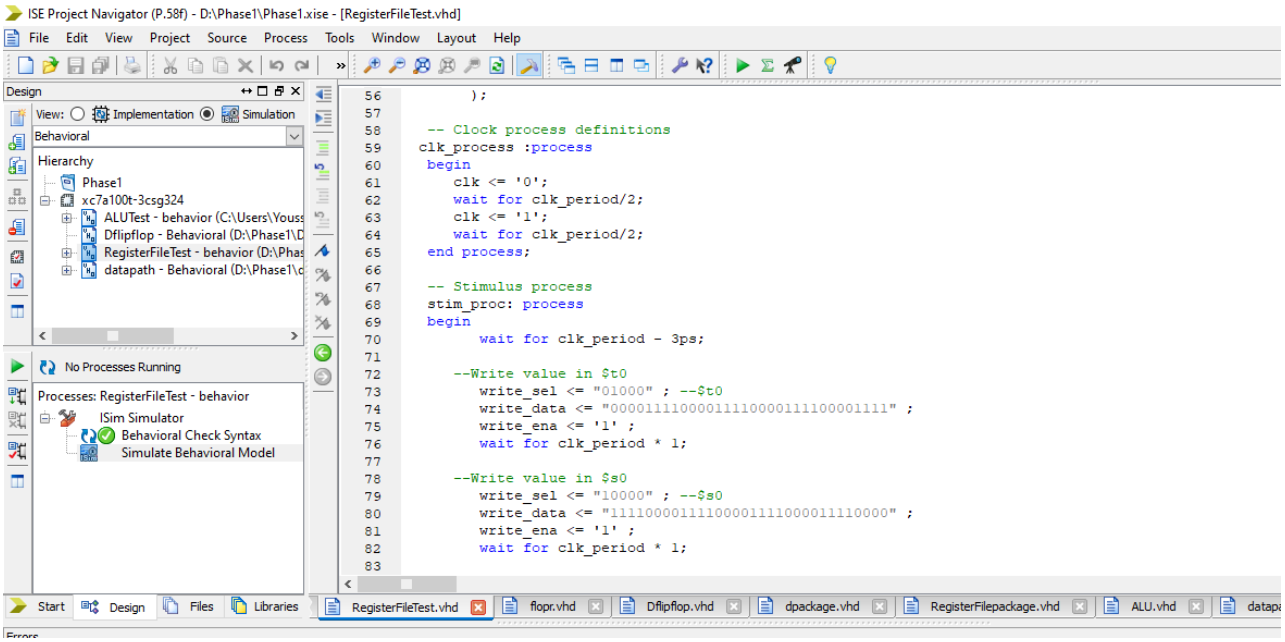

main:1
imem
im1
mips
mips1
dmem
dm1
main

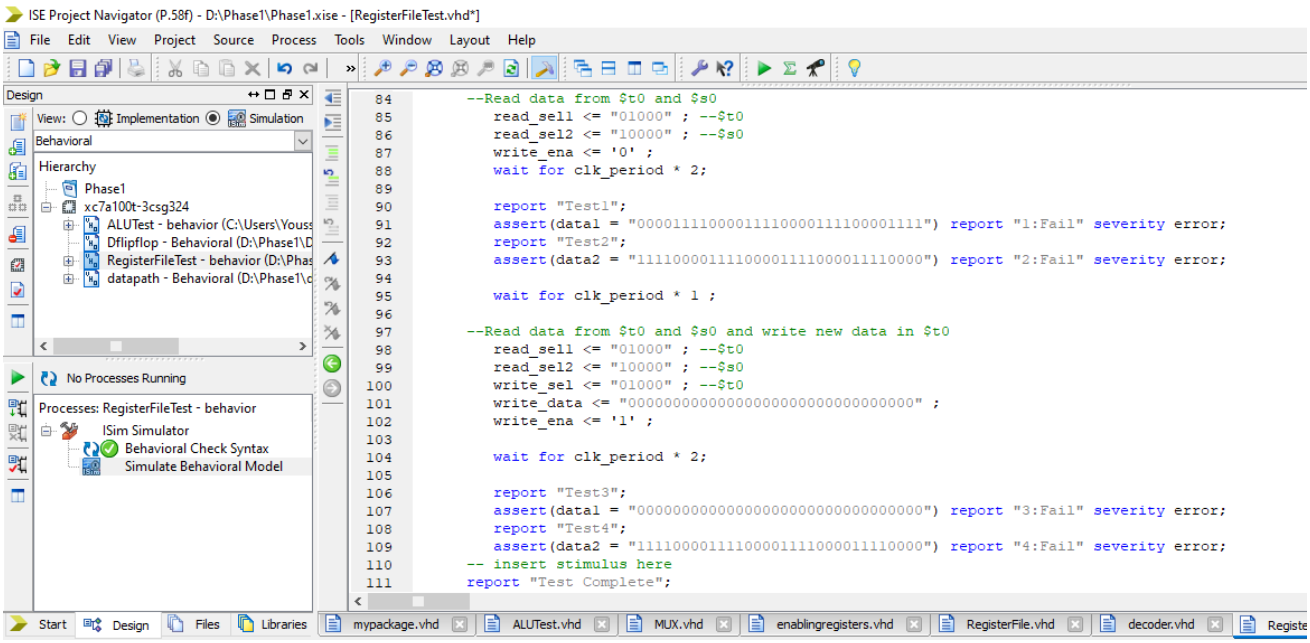
Testing Codes

1- RegisterFile

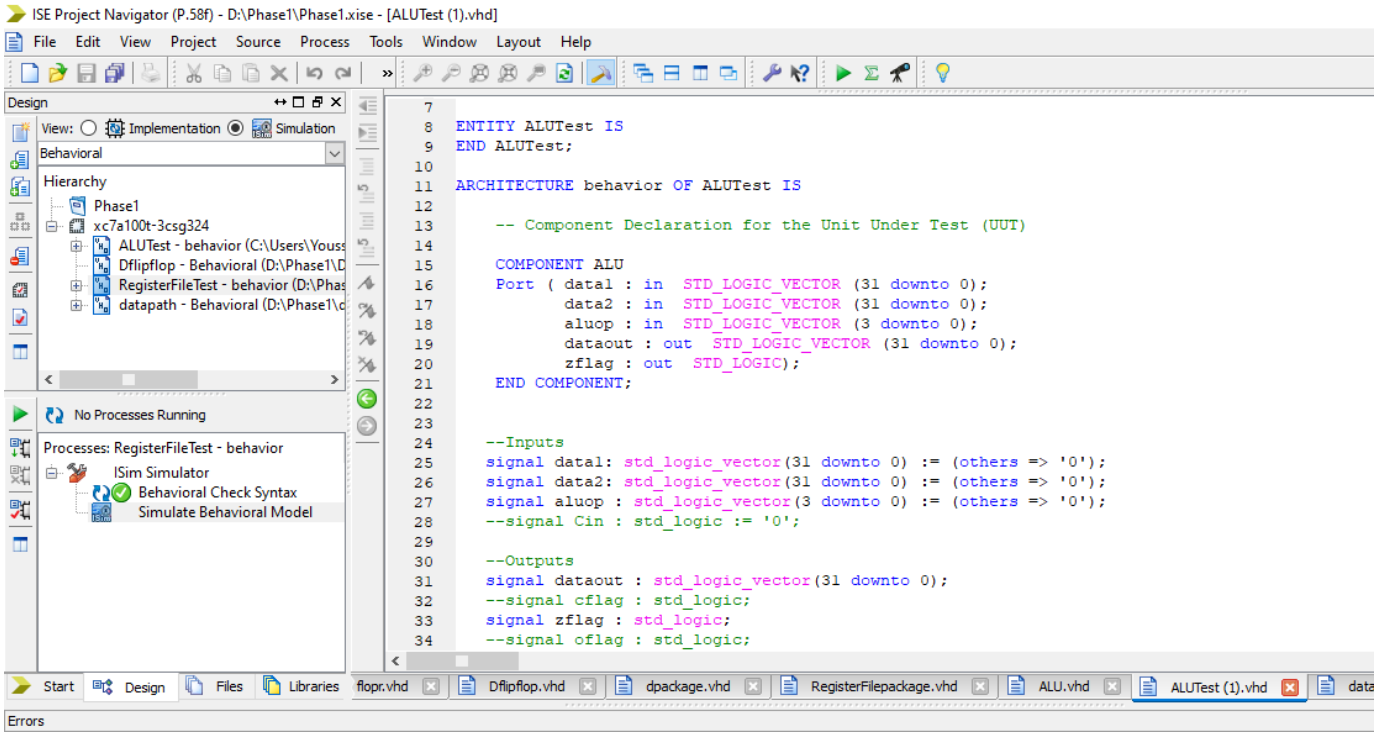


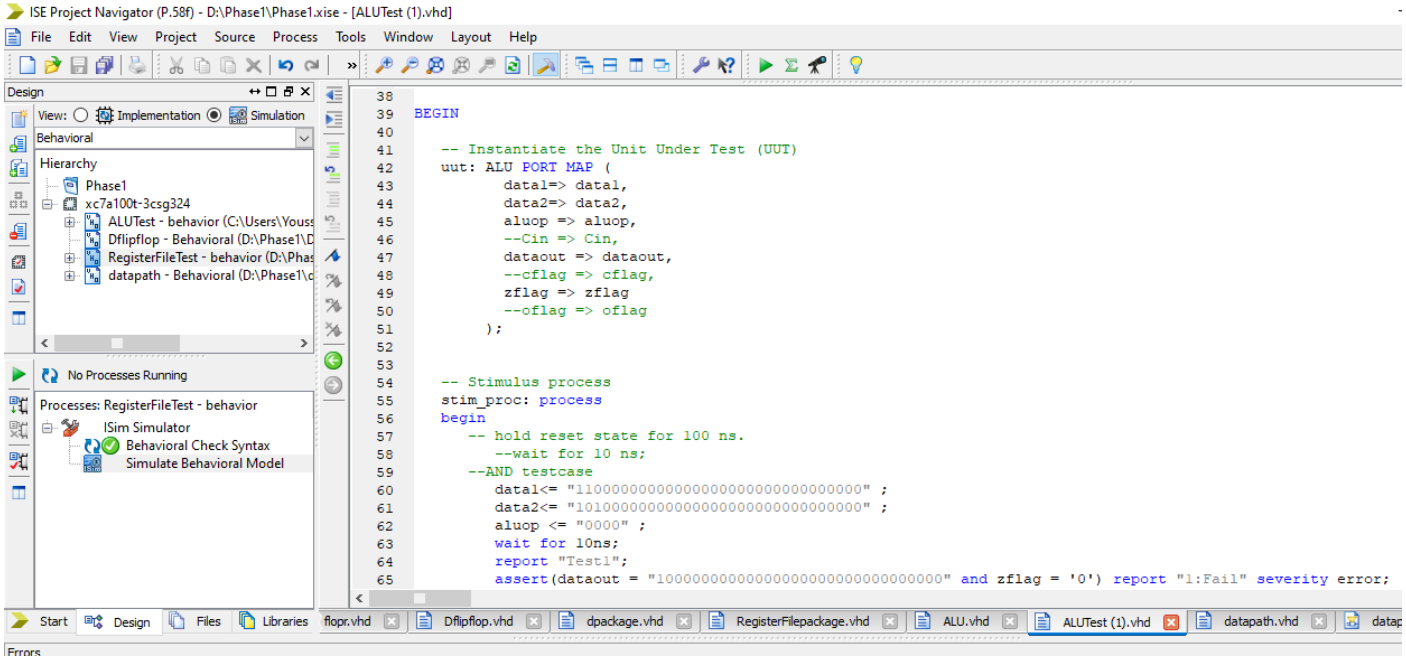


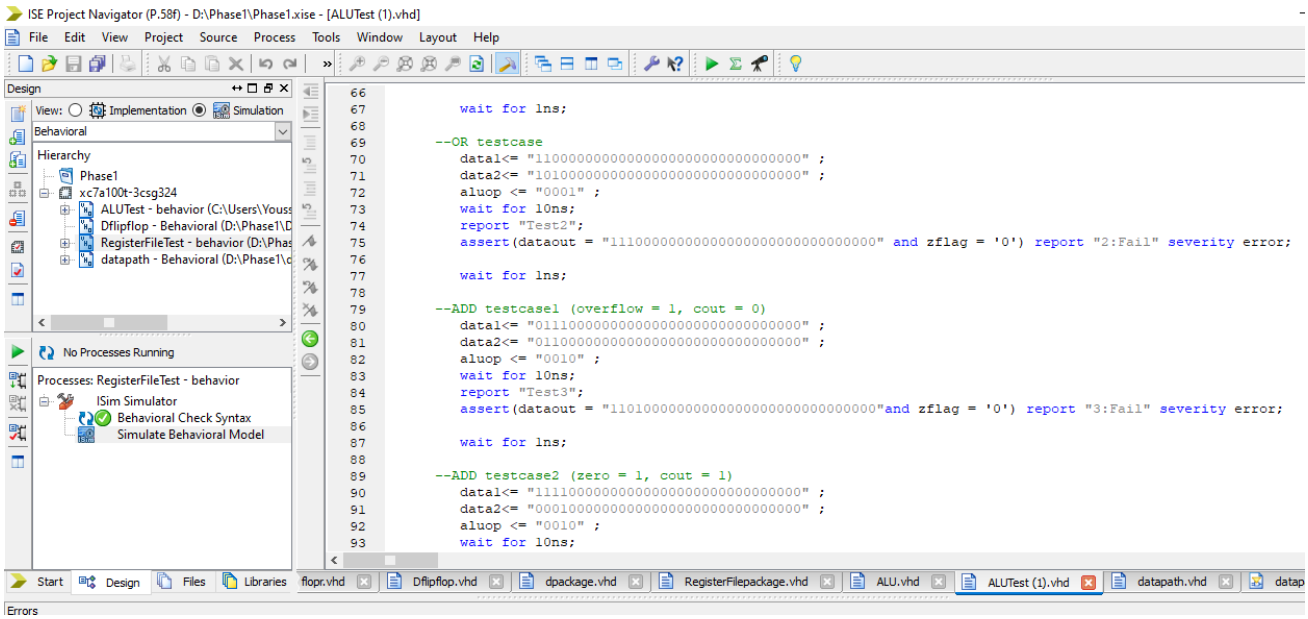

ISE Project Navigator (P.58f) - D:\Phase1\Phase1.xise - [RegisterFileTest.vhd]
File Edit View Project Source Process Tools Window Layout Help
Design
View: Implementation Simulation
Behavioral
Hierarchy
Phase1
xc7a100t-3csg324
ALUTest - behavior (C:\Users\Youss
Dflipflop - Behavioral (D:\Phase1\D
RegisterFileTest - behavior (D:\Phas
datapath - Behavioral (D:\Phase1\c
No Processes Running
Processes: RegisterFileTest - behavior
ISim Simulator
Behavioral Check Syntax
Simulate Behavioral Model
56);
57
58 -- Clock process definitions
59 clk_process :process
60 begin
61 clk <= '0';
62 wait for clk_period/2;
63 clk <= '1';
64 wait for clk_period/2;
65 end process;
66
67 -- Stimulus process
68 stim_proc: process
69 begin
70 wait for clk_period - 3ps;
71
72 --Write value in $t0
73 write_sel <= "01000" ; --$t0
74 write_data <= "00001111000011110000111100001111" ;
75 write_ena <= '1' ;
76 wait for clk_period * 1;
77
78 --Write value in $s0
79 write_sel <= "10000" ; --$s0
80 write_data <= "11110000111100001111000011110000" ;
81 write_ena <= '1' ;
82 wait for clk_period * 1;
83
Start Design Files Libraries
RegisterFileTest.vhd flopr.vhd Dflipflop.vhd dpackage.vhd RegisterFilepackage.vhd ALU.vhd
Errors

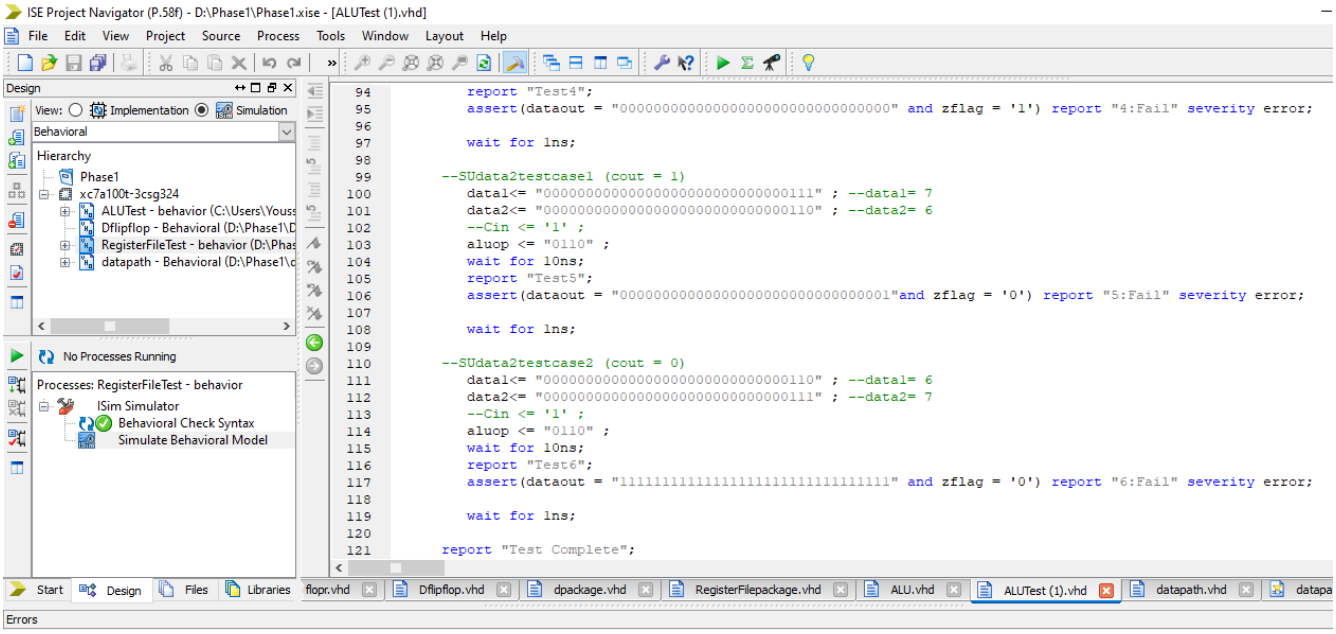

ISE Project Navigator (P.58f) - D:\Phase1\Phase1.xise - [RegisterFileTest.vhd*]
File Edit View Project Source Process Tools Window Layout Help
Design
View: Implementation Simulation
Behavioral
Hierarchy
Phase1
xc7a100t-3csg324
ALUTest - behavior (C:\Users\Youss
Dflipflop - Behavioral (D:\Phase1\D
RegisterFileTest - behavior (D:\Phas
datapath - Behavioral (D:\Phase1\c
No Processes Running
Processes: RegisterFileTest - behavior
ISim Simulator
Behavioral Check Syntax
Simulate Behavioral Model
84 --Read data from $t0 and $s0
85 read_sel1 <= "01000" ; --$t0
86 read_sel2 <= "10000" ; --$s0
87 write_ena <= '0' ;
88 wait for clk_period * 2;
89
90 report "Test1";
91 assert(data1 = "00001111000011110000111100001111") report "1:Fail" severity error;
92 report "Test2";
93 assert(data2 = "11110000111100001111000011110000") report "2:Fail" severity error;
94
95 wait for clk_period * 1 ;
96
97 --Read data from $t0 and $s0 and write new data in $t0
98 read_sel1 <= "01000" ; --$t0
99 read_sel2 <= "10000" ; --$s0
100 write_sel <= "01000" ; --$t0
101 write_data <= "00000000000000000000000000000000" ;
102 write_ena <= '1' ;
103
104 wait for clk_period * 2;
105
106 report "Test3";
107 assert(data1 = "00000000000000000000000000000000") report "3:Fail" severity error;
108 report "Test4";
109 assert(data2 = "11110000111100001111000011110000") report "4:Fail" severity error;
110 -- insert stimulus here
111 report "Test Complete";
Start Design Files Libraries
mypackage.vhd ALUTest.vhd MUX.vhd enablingregisters.vhd RegisterFile.vhd decoder.vhd

2- ALU


ISE Project Navigator (P.58f) - D:\Phase1\Phase1.xise - [ALUTest (1).vhd]
File Edit View Project Source Process Tools Window Layout Help
Design
View: Implementation Simulation
Behavioral
Hierarchy
Phase1
xc7a100t-3csg324
ALUTest - behavior (C:\Users\Youss
Dflipflop - Behavioral (D:\Phase1\D
RegisterFileTest - behavior (D:\Phas
datapath - Behavioral (D:\Phase1\c
No Processes Running
Processes: RegisterFileTest - behavior
ISim Simulator
Behavioral Check Syntax
Simulate Behavioral Model
7 ENTITY ALUTest IS
8 END ALUTest;
9
10 ARCHITECTURE behavior OF ALUTest IS
11
12 -- Component Declaration for the Unit Under Test (UUT)
13
14 COMPONENT ALU
15 Port (datal : in STD_LOGIC_VECTOR (31 downto 0);
16 data2 : in STD_LOGIC_VECTOR (31 downto 0);
17 aluop : in STD_LOGIC_VECTOR (3 downto 0);
18 dataout : out STD_LOGIC_VECTOR (31 downto 0);
19 zflag : out STD_LOGIC);
20 END COMPONENT;
21
22
23 --Inputs
24 signal datal: std_logic_vector(31 downto 0) := (others => '0');
25 signal data2: std_logic_vector(31 downto 0) := (others => '0');
26 signal aluop : std_logic_vector(3 downto 0) := (others => '0');
27 --signal Cin : std_logic := '0';
28
29 --Outputs
30 signal dataout : std_logic_vector(31 downto 0);
31 --signal cflag : std_logic;
32 signal zflag : std_logic;
33 --signal oflag : std_logic;
34
Start Design Files Libraries
flopr.vhd Dflipflop.vhd dpackage.vhd RegisterFilepackage.vhd ALU.vhd ALUTest (1).vhd
Errors


ISE Project Navigator (P.58f) - D:\Phase1\Phase1.xise - [ALUTest (1).vhd]
File Edit View Project Source Process Tools Window Layout Help
Design
View: Implementation Simulation
Behavioral
Hierarchy
Phase1
xc7a100t-3csg324
ALUTest - behavior (C:\Users\Youss
Dflipflop - Behavioral (D:\Phase1\D
RegisterFileTest - behavior (D:\Phas
datapath - Behavioral (D:\Phase1\c
No Processes Running
Processes: RegisterFileTest - behavior
ISim Simulator
Behavioral Check Syntax
Simulate Behavioral Model
38
39 BEGIN
40
41 -- Instantiate the Unit Under Test (UUT)
42 uut: ALU PORT MAP (
43 datal=> datal,
44 data2=> data2,
45 aluop => aluop,
46 --Cin => Cin,
47 dataout => dataout,
48 --cflag => cflag,
49 zflag => zflag
50 --oflag => oflag
51);
52
53
54 -- Stimulus process
55 stim_proc: process
56 begin
57 -- hold reset state for 100 ns.
58 --wait for 10 ns;
59 --AND testcase
60 datal<= "11000000000000000000000000000000" ;
61 data2<= "10100000000000000000000000000000" ;
62 aluop <= "0000" ;
63 wait for 10ns;
64 report "Test1";
65 assert(dataout = "10000000000000000000000000000000" and zflag = '0') report "1:Fail" severity error;
Start Design Files Libraries
flopr.vhd Dflipflop.vhd dpackage.vhd RegisterFilepackage.vhd ALU.vhd ALUTest (1).vhd datapath.vhd
Errors


ISE Project Navigator (P.58f) - D:\Phase1\Phase1.xise - [ALUTest (1).vhd]
File Edit View Project Source Process Tools Window Layout Help
Design
View: Implementation Simulation
Behavioral
Hierarchy
Phase1
xc7a100t-3csg324
ALUTest - behavior (C:\Users\Youss
Dflipflop - Behavioral (D:\Phase1\D
RegisterFileTest - behavior (D:\Phas
datapath - Behavioral (D:\Phase1\c
No Processes Running
Processes: RegisterFileTest - behavior
ISim Simulator
Behavioral Check Syntax
Simulate Behavioral Model
66 wait for 1ns;
67
68 --OR testcase
69 data1<= "11000000000000000000000000000000" ;
70 data2<= "10100000000000000000000000000000" ;
71 aluop <= "0001" ;
72 wait for 10ns;
73 report "Test2";
74 assert(dataout = "11100000000000000000000000000000" and zflag = '0') report "2:Fail" severity error;
75
76 wait for 1ns;
77
78 --ADD testcase1 (overflow = 1, cout = 0)
79 data1<= "01110000000000000000000000000000" ;
80 data2<= "01100000000000000000000000000000" ;
81 aluop <= "0010" ;
82 wait for 10ns;
83 report "Test3";
84 assert(dataout = "11010000000000000000000000000000"and zflag = '0') report "3:Fail" severity error;
85
86 wait for 1ns;
87
88 --ADD testcase2 (zero = 1, cout = 1)
89 data1<= "11110000000000000000000000000000" ;
90 data2<= "00010000000000000000000000000000" ;
91 aluop <= "0010" ;
92 wait for 10ns;
93
Start Design Files Libraries
flopr.vhd Dflipflop.vhd dpackage.vhd RegisterFilepackage.vhd ALU.vhd ALUTest (1).vhd datapath.vhd datap
Errors


ISE Project Navigator (P.58f) - D:\Phase1\Phase1.xise - [ALUTest (1).vhd]
File Edit View Project Source Process Tools Window Layout Help
Design
View: Implementation Simulation
Behavioral
Hierarchy
Phase1
xc7a100t-3csg324
ALUTest - behavior (C:\Users\Youss
Dflipflop - Behavioral (D:\Phase1\D
RegisterFileTest - behavior (D:\Phas
datapath - Behavioral (D:\Phase1\c
No Processes Running
Processes: RegisterFileTest - behavior
ISim Simulator
Behavioral Check Syntax
Simulate Behavioral Model
94 report "Test4";
95 assert(dataout = "00000000000000000000000000000000" and zflag = '1') report "4:Fail" severity error;
96
97 wait for 1ns;
98
99 --SUdata2testcase1 (cout = 1)
100 data1<= "00000000000000000000000000000111" ; --data1= 7
101 data2<= "00000000000000000000000000000110" ; --data2= 6
102 --Cin <= '1' ;
103 aluop <= "0110" ;
104 wait for 10ns;
105 report "Test5";
106 assert(dataout = "00000000000000000000000000000001"and zflag = '0') report "5:Fail" severity error;
107
108 wait for 1ns;
109
110 --SUdata2testcase2 (cout = 0)
111 data1<= "00000000000000000000000000000110" ; --data1= 6
112 data2<= "00000000000000000000000000000111" ; --data2= 7
113 --Cin <= '1' ;
114 aluop <= "0110" ;
115 wait for 10ns;
116 report "Test6";
117 assert(dataout = "11111111111111111111111111111111" and zflag = '0') report "6:Fail" severity error;
118
119 wait for 1ns;
120
121 report "Test Complete";
Start Design Files Libraries
flopr.vhd Dflipflop.vhd dpackage.vhd RegisterFilepackage.vhd ALU.vhd ALUTest (1).vhd datapath.vhd datapa
Errors

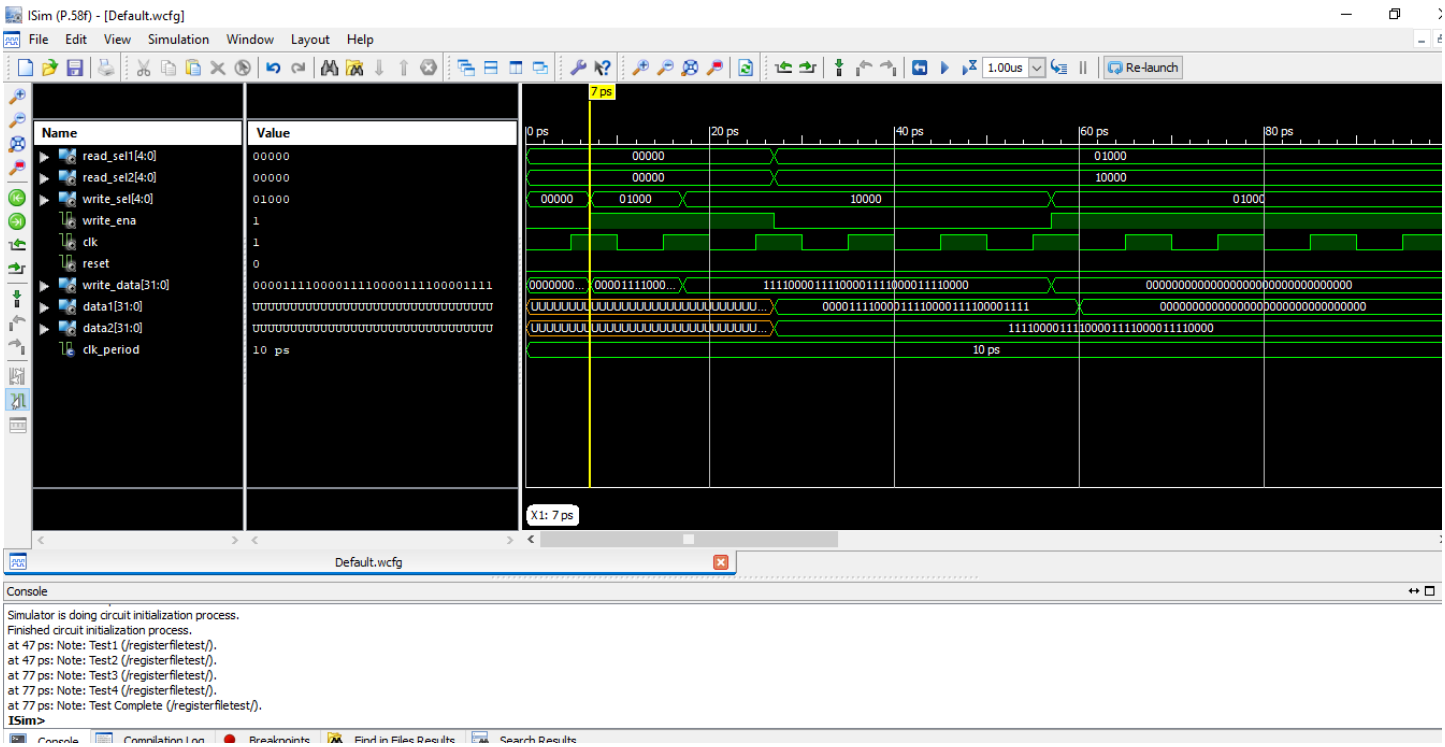
3- Phase 2 test

```
10  architecture test of testbench is
11  component main
12  port(clk, reset: in STD_LOGIC;
13  writedata, dataadr: out STD_LOGIC_VECTOR(31 downto 0);
14  memwrite: out STD_LOGIC);
15  end component;
16  signal writedata, dataadr: STD_LOGIC_VECTOR(31 downto 0);
17  signal clk, reset, memwrite: STD_LOGIC;
18
19  begin
20  -- instantiate device to be tested
21  dut: main port map(clk, reset, writedata, dataadr, memwrite);
22
23  -- Generate clock with 10 ns period
24  process begin
25  clk <= '1';
26  wait for 5 ns;
27  clk <= '0';
28  wait for 5 ns;
29  end process;
30
31  -- Generate reset for first two clock cycles
32  process begin
33  reset <= '1';
34  wait for 22 ns;
35  reset <= '0';
36  wait;
37  end process;
```

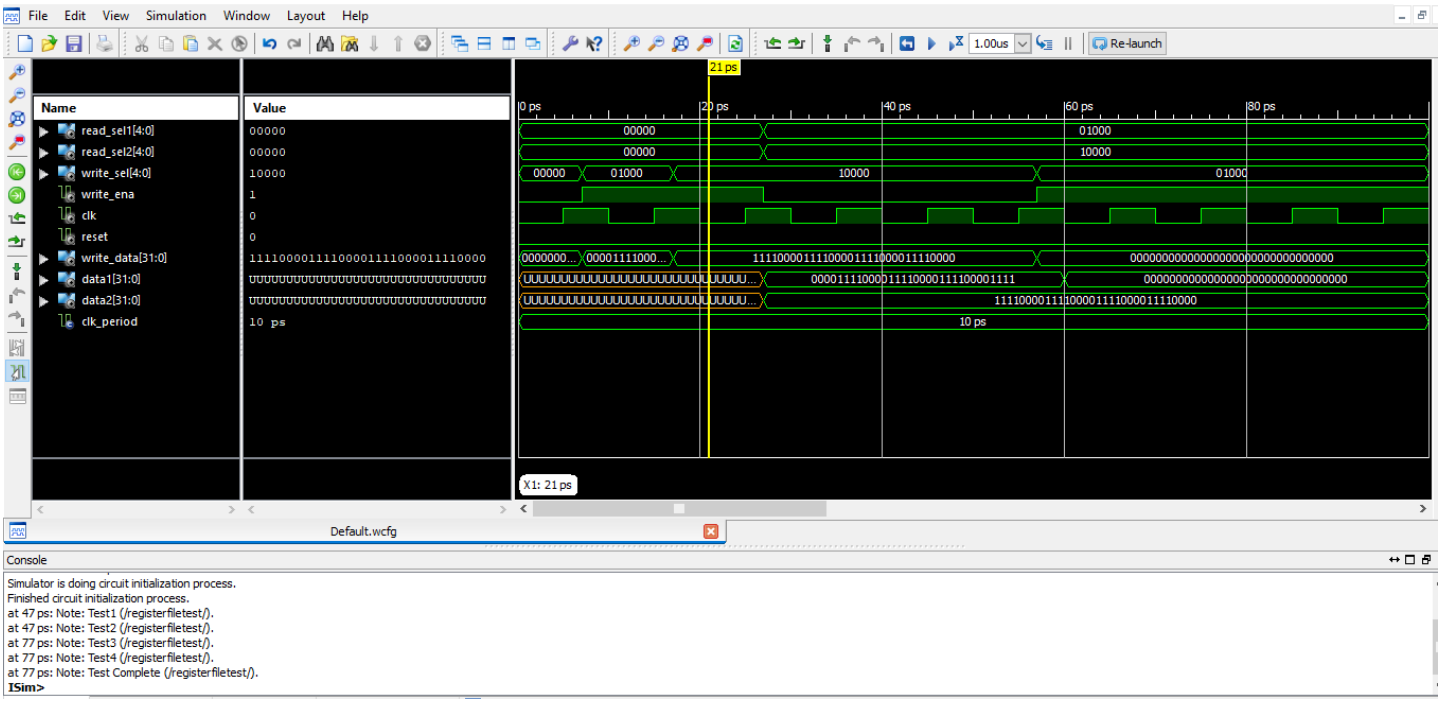
```
24  process begin
25  clk <= '1';
26  wait for 5 ns;
27  clk <= '0';
28  wait for 5 ns;
29  end process;
30
31  -- Generate reset for first two clock cycles
32  process begin
33  reset <= '1';
34  wait for 22 ns;
35  reset <= '0';
36  wait;
37  end process;
38
39  -- check that 7 gets written to address 84 at end of program
40  process(clk) begin
41  if (clk'event and clk = '0' and memwrite = '1') then
42  if (CONV_INTEGER(dataadr) = 84 and CONV_INTEGER(writedata) = 7) then
43  report "NO ERRORS: Simulation succeeded" severity failure;
44  elsif (CONV_INTEGER(dataadr) = 84) then
45  report "Simulation failed" severity failure;
46  end if;
47  end if;
48
49  end process;
50  end;
51
```

Simulation of blocks

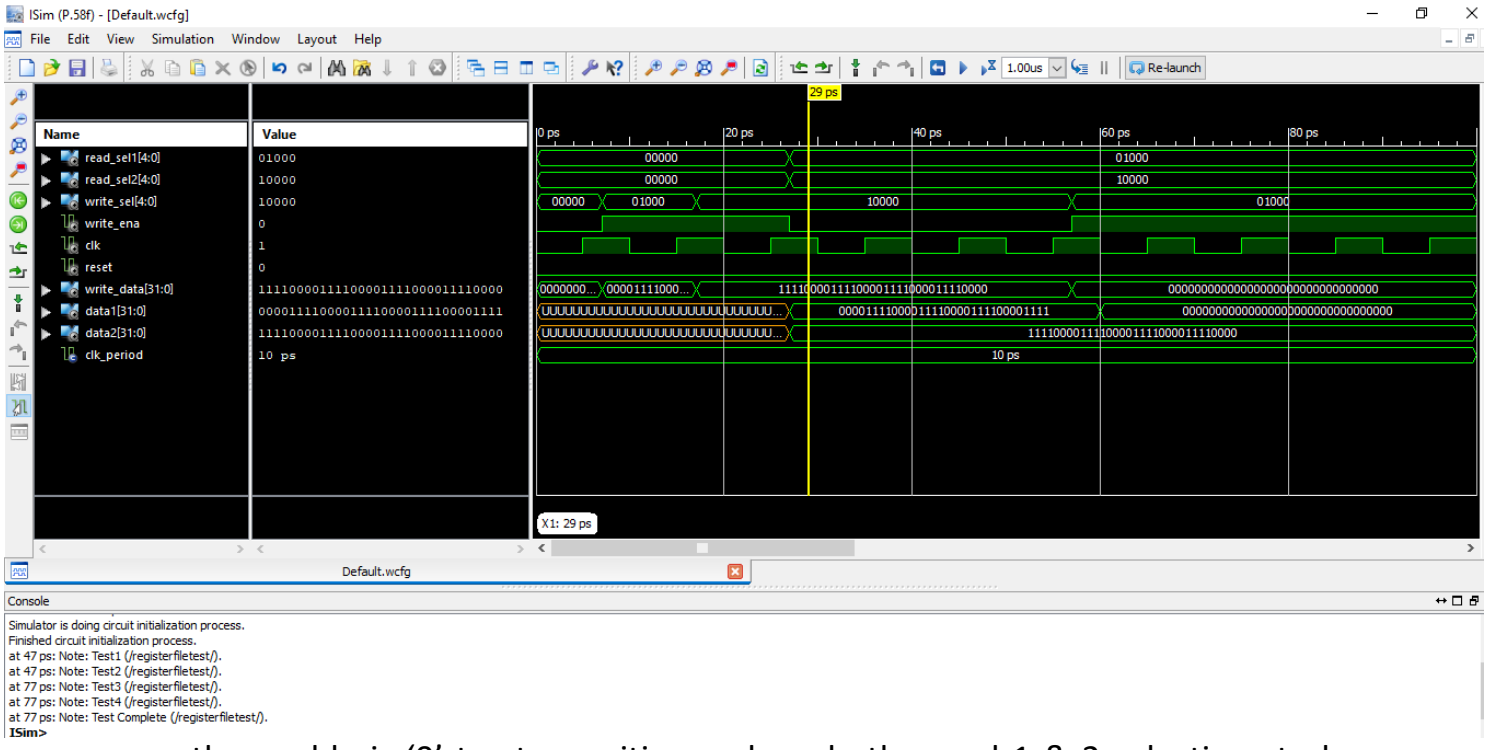
1- RegisterFile



First step in simulation was choosing register “01000” and making enable ‘1’ to write the data “00001111000011110000111100001111”

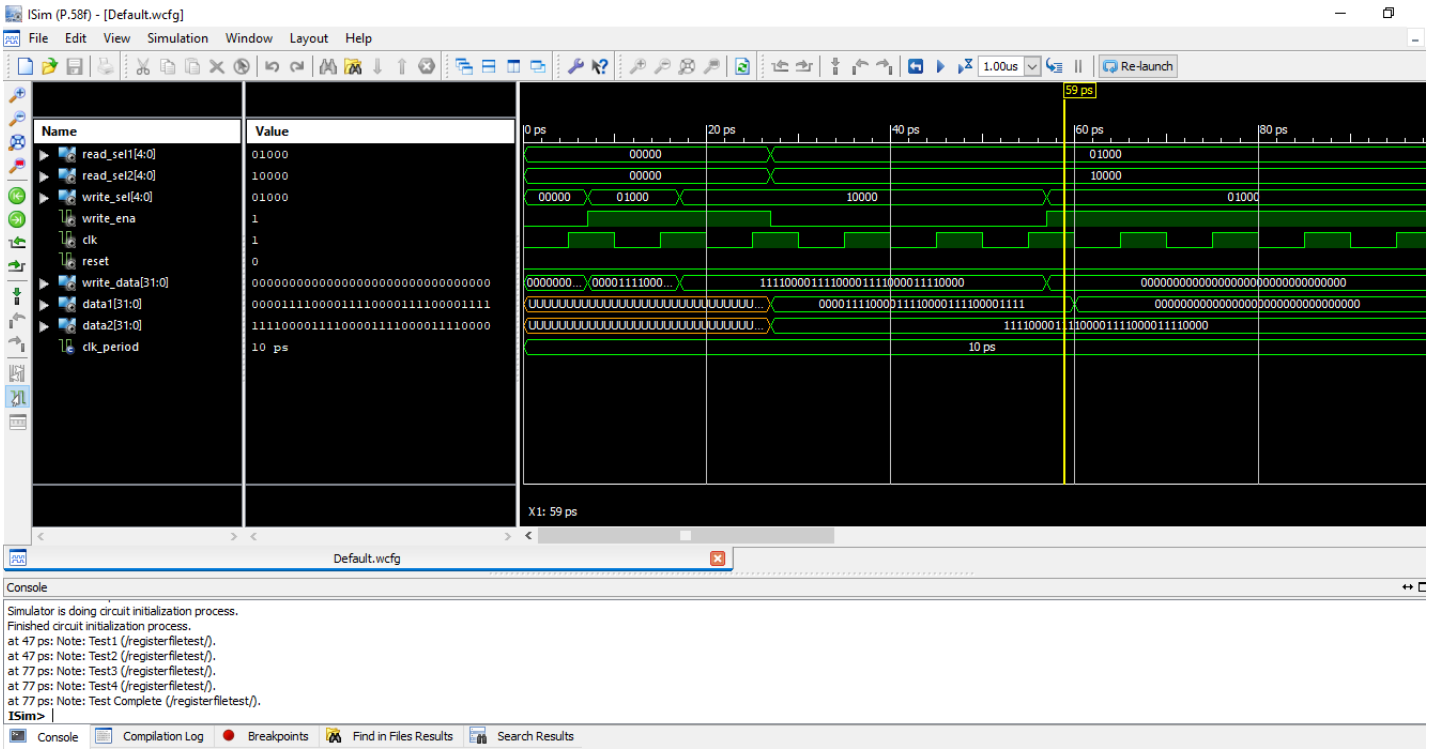


second step in simulation was choosing register “10000” and making enable ‘1’ to write the data “11110000111100001111000011110000”

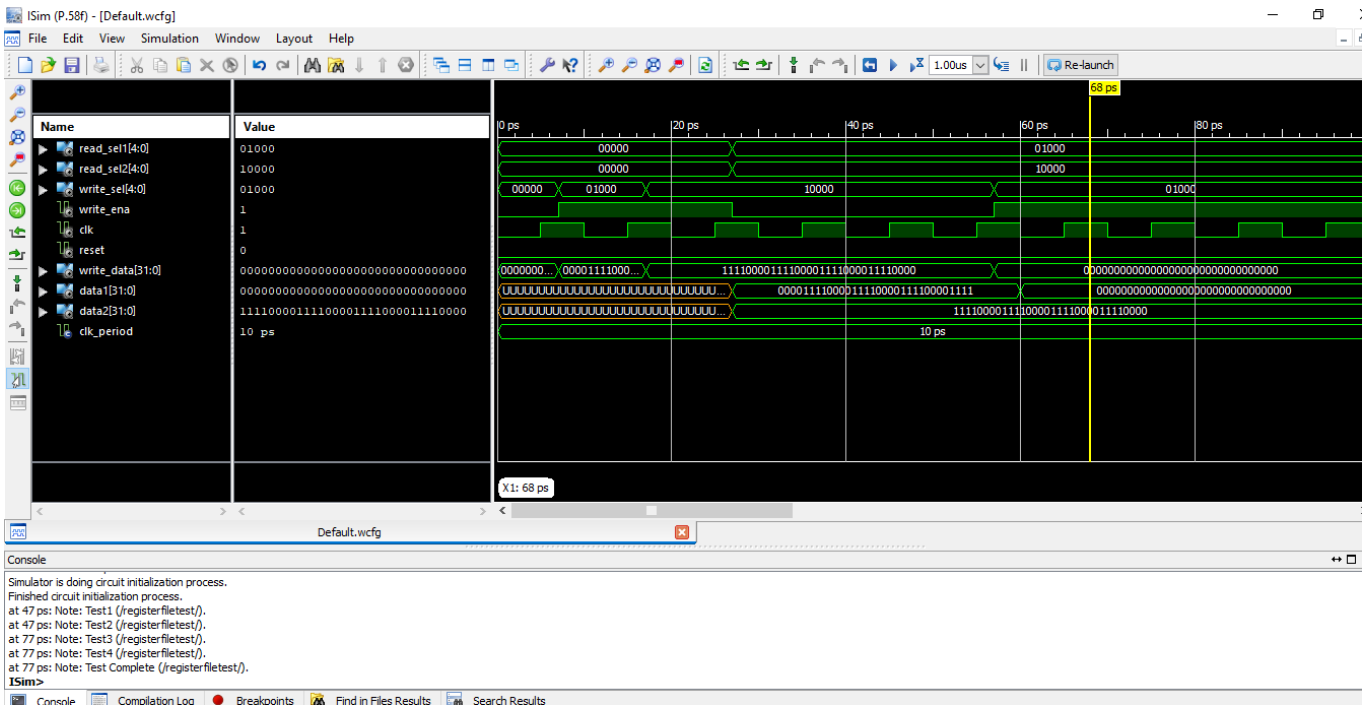


now the enable is ‘0’ to stop writing and made the read 1 & 2 selections to be “01000” & “10000” to read from the registers we just wrote in. data 1 & 2 equal

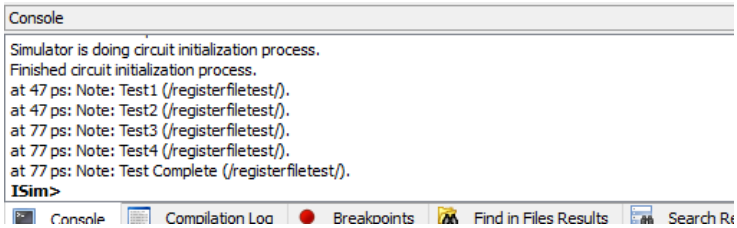
“00001111000011110000111100001111” & “11110000111100001111000011110000” are the same as we wrote them in these registers which verifies passing test cases 1 & 2.



now we change the data in register “01000” to be "00000000000000000000000000000000"

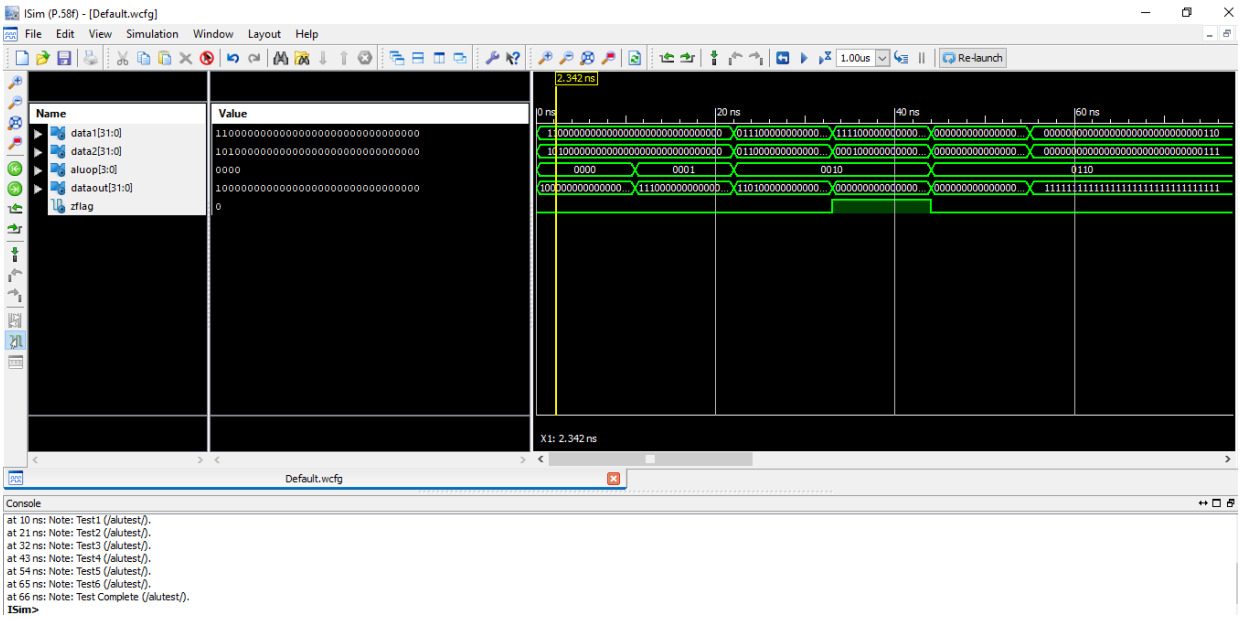


now the enable is ‘0’ again to stop writing and the read 1 & 2 selections are still “01000” & “10000”. data 1 equals "00000000000000000000000000000000" that we just wrote & data 2 remained as it is which verifies testcases 3 & 4.



Here it shows all 4 testcases passed which verifies the RegisterFile.

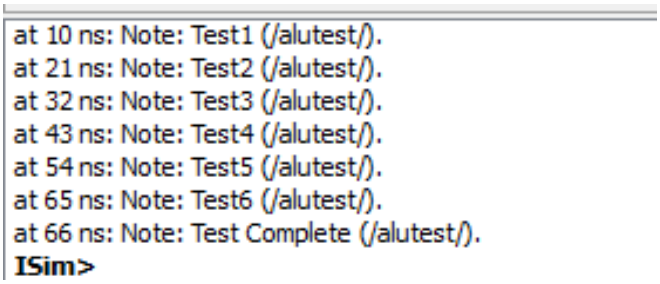
2- ALU



ALUop = “0000” which is A AND B

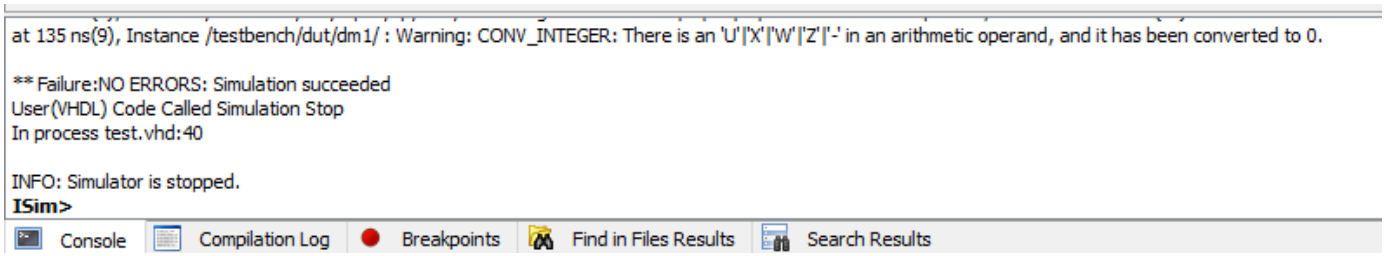
the output verifies the AND validity.

Similarly All testcases are passed in the ALU and can be checked from the Xilinx simulation.



Here it shows all 6 testcases passed which verifies the ALU.

3- Phase 2 Test



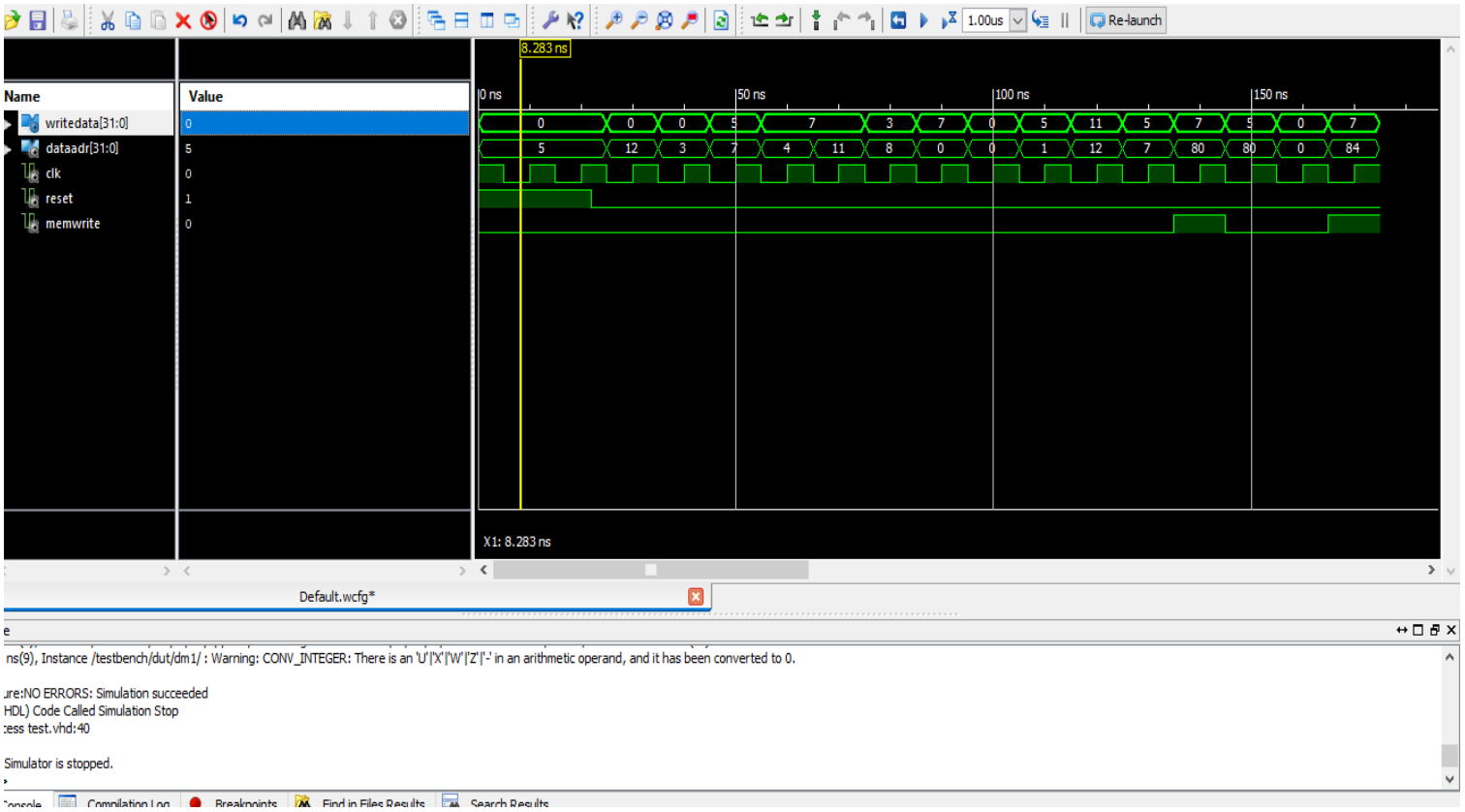
It shows that there is no errors & simulation succeeded

memfile.dat

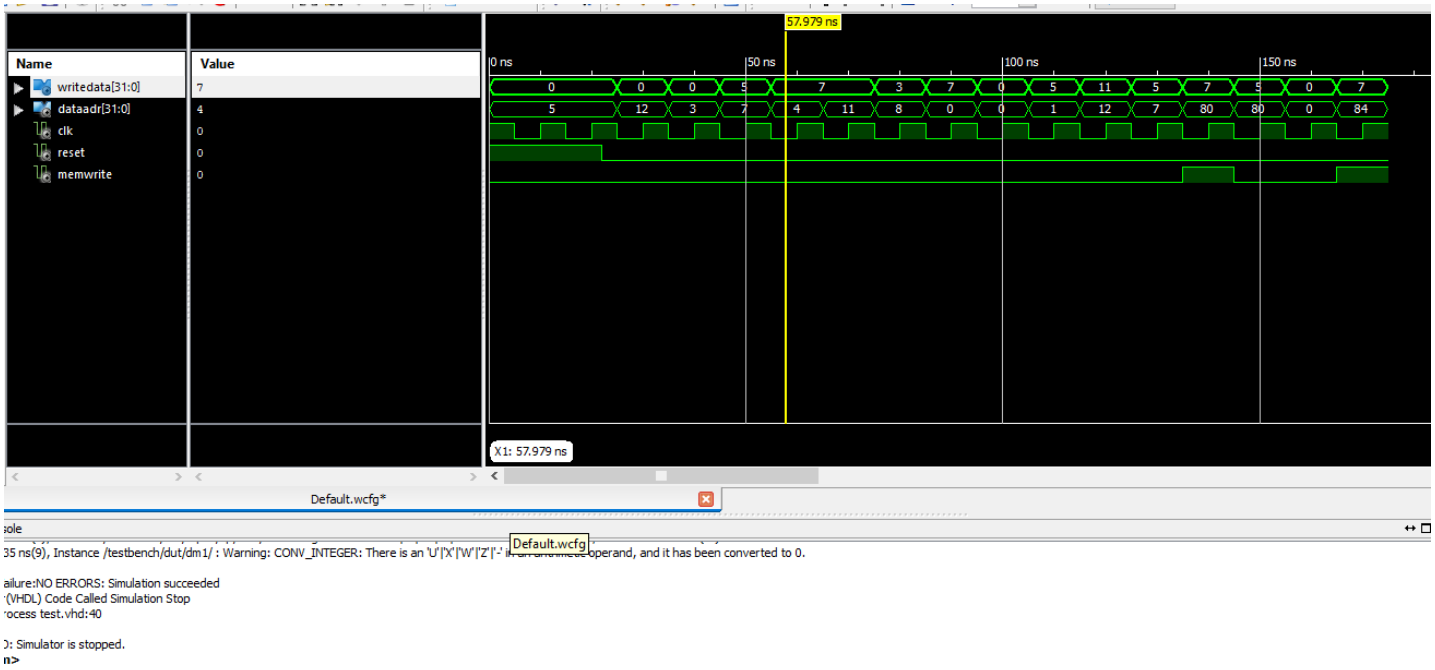
it contains the instructions that are put in instruction memory to check

```
20020005
2003000c
2067fff7
00e22025
00642824
00a42820
10a7000a
0064202a
10800001
20050000
00e2202a
00853820
00e23822
ac670044
8c020050
08000011
20020001
ac020054
```

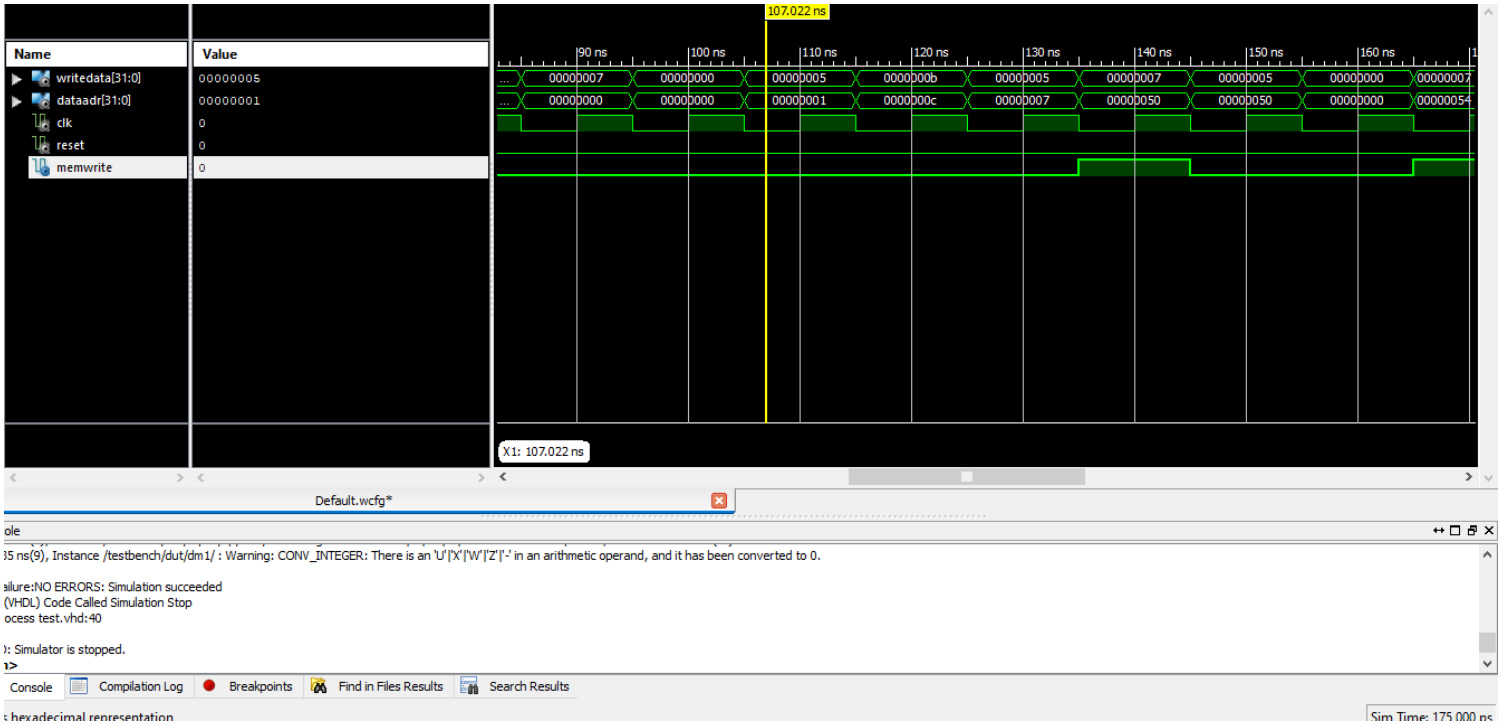
(Instruction 1)



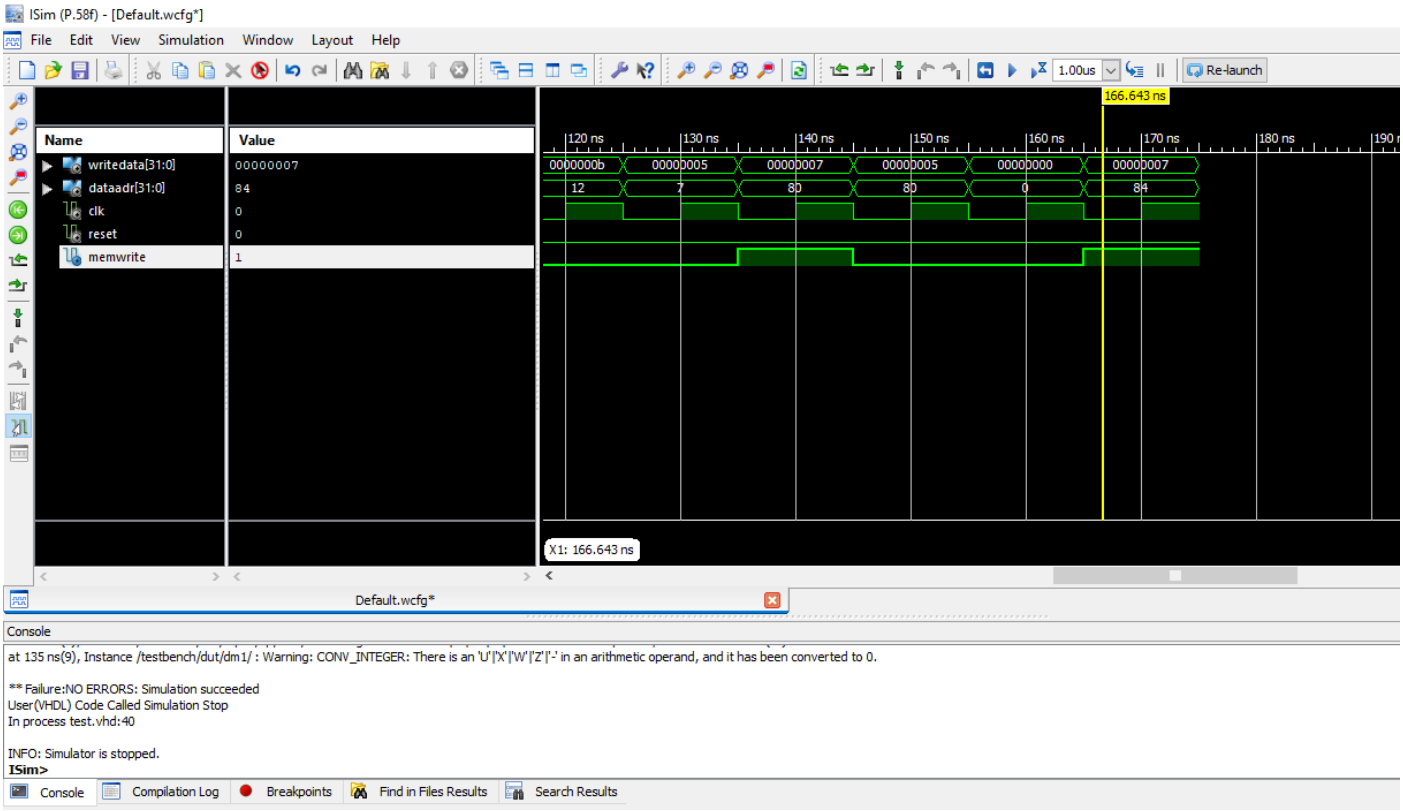
(INSTRUCTIONS 6-10: 2ND 5 CYCLES)



(INSTRUCTIONS 11-15: 3RD 5 CYCLES)



Last INSTRUCTION: CYCLE # 16



this shows last instruction has 7 in write data

84 is its data address

Conclusion

- Using xilinx and VHDL for ise design is very significant and helped in implementing different circuits.

- Using of clock, process and IF statements helped making sequential circuits.

- Using its features like Testing helps you make sure that the outputs are as desired which verifies the circuit.

- Both the DR's slides & the lab manual was very helpful and helped in every aspect in the project.

- We've learned how to trace our mistakes & solve them either by the RTL schematic or for more complex circuits that might not be understood fully from the Schematic, we can make testcases using simulation & verify our answers.

References

1-COMPUTER ORGANIZATION AND DESIGN Fifth edition by DAVID A. PATTERSON and JOHN L. HENNESSY, 20 APRIL 2024